



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO
INGENIERÍA DE TECNOLOGÍAS DE TELECOMUNICACIÓN

Desarrollo de una aplicación móvil para el control y monitorización de robots basados en ROS 2

Autor

Enrique Gómez Pacheco

Directores

Juan José Ramos Muñoz



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, septiembre de 2026



Desarrollo de una aplicación móvil para el control y monitorización de robots basados en ROS 2.

Desarrollo de la aplicación R2Pilot.

Autor

Enrique Gómez Pacheco

Director

Juan José Ramos Muñoz



DEPARTAMENTO DE TEORÍA DE LA SEÑAL, TELEMÁTICA Y
COMUNICACIONES

—
Granada, septiembre de 2026

Desarrollo de una aplicación móvil para el control y monitorización de robots basados en ROS 2

Enrique Gómez Pacheco

Palabras clave: robótica móvil, ROS 2, aplicación móvil, control, monitorización, DDS, Android, arquitectura cliente-servidor, protocolo.

Resumen

En el ámbito de la robótica móvil, la intervención y supervisión por parte de un operador humano siguen siendo necesarias en numerosas tareas de desarrollo, experimentación y pruebas. En este contexto, este Trabajo de Fin de Grado presenta el diseño y desarrollo de R2Pilot, una aplicación móvil nativa para dispositivos Android orientada al control y la monitorización remota de robots basados en ROS 2. La aplicación busca proporcionar una interfaz portátil que permita centralizar la interacción con el robot sin depender de las herramientas de control habituales en un ordenador.

Para ello, se ha desarrollado una arquitectura cliente-servidor formada por una aplicación Android y un servidor ejecutado en el robot, comunicados mediante un protocolo propio basado en WebSockets para el intercambio bidireccional de órdenes e información. El servidor permite interactuar con ROS 2 (*Robot Operating System 2*), un *middleware* ampliamente utilizado en robótica para la comunicación entre los distintos componentes del sistema. La aplicación integra diferentes funcionalidades de control y monitorización, entre las que se incluyen la teleoperación de la base móvil mediante un joystick virtual, el control de las articulaciones, la visualización de cámaras y datos procedentes de sensores, así como la ejecución de movimientos y rutinas predefinidas.

El sistema se ha diseñado siguiendo un enfoque modular que permite adaptar la aplicación a las características específicas de diferentes plataformas robóticas. Para evaluar la solución desarrollada, se han realizado pruebas en entornos de simulación y sobre un robot físico, analizando su funcionamiento y la comunicación entre los distintos componentes. Los resultados obtenidos muestran la viabilidad de utilizar una aplicación móvil como interfaz para el control y la monitorización de sistemas robóticos basados en ROS 2.

Development of a mobile application for the control and monitoring of ROS 2-based robots.

Enrique Gómez Pacheco

Keywords: mobile robotics, ROS 2, mobile application, control, monitoring, DDS, Android, client-server architecture, protocol

Abstract

In the field of mobile robotics, intervention and supervision by a human operator remain necessary for numerous development, experimentation and testing tasks. In this context, this Final Year Project presents the design and development of R2Pilot, a native mobile application for Android devices designed for the remote control and monitoring of ROS 2-based robots. The application aims to provide a portable interface that centralises interaction with the robot without relying on the usual control tools found on a computer.

To this end, a client-server architecture has been developed, consisting of an Android application and a server running on the robot, communicating through a custom WebSocket-based protocol for the bidirectional exchange of commands and information. The server enables interaction with ROS 2 (*Robot Operating System 2*), a middleware widely used in robotics for communication between the different components of the system. The application integrates various control and monitoring functionalities, including mobile base teleoperation through a virtual joystick, joint control, visualization of camera feeds and sensor data, as well as the execution of predefined movements and routines.

The system has been designed using a modular approach that allows the application to be adapted to the specific characteristics of different robotic platforms. To evaluate the solution developed, tests have been carried out in simulation environments and on a physical robot, analysing its operation and the communication between the various components. The results obtained demonstrate the feasibility of using a mobile application as an interface for the control and monitoring of ROS 2-based robotic systems.

Yo, **Enrique Gómez Pacheco**, alumno de la titulación INGENIERÍA DE TECNOLOGÍAS DE TELECOMUNICACIÓN de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 79111322Q, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Enrique Gómez Pacheco

Granada a 4 de septiembre de 2026.

D. **Juan José Ramos Muñoz**, Profesor del Área de telemática del Departamento de Teoría de la Señal, Telemática y Comunicaciones de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Desarrollo de una aplicación móvil para el control y monitorización de robots basados en ROS 2***, ha sido realizado bajo su supervisión por **Enrique Gómez Pacheco**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a 4 de septiembre de 2026.

El director:

Juan José Ramos Muñoz

Declaración del Uso Responsable de la Inteligencia Artificial Generativa

Durante el desarrollo de este Trabajo Fin de Grado, se ha hecho uso de herramientas de inteligencia artificial generativa Google Gemini en dos ámbitos. Se han utilizado como apoyo para la revisión y mejora de la redacción de diversos apartados de la memoria, así como para el desarrollo del código de la aplicación. En este último ámbito, las herramientas se han empleado principalmente para la generación de bloques de código repetitivos y estructurales, y como apoyo en el diseño y representación de la interfaz de usuario.

En todos los casos, estas herramientas se han utilizado únicamente como apoyo al desarrollo del trabajo. El contenido generado ha sido revisado, analizado y comprendido por el autor antes de su incorporación al proyecto.

Fdo: Enrique Gómez Pacheco

Granada a 4 de septiembre de 2026.

Agradecimientos

En primer lugar, quiero agradecer a mi familia, especialmente a mis padres y a mi hermana, ya que son las personas que me han acompañado día a día. Gracias por todos los medios, la confianza y el cariño que me habéis dado y, en general, por todo lo que me habéis enseñado. Sin vosotros, no podría haber llegado hasta aquí.

A mis amigos del instituto, que, a pesar de haber tomado caminos distintos, siempre hemos mantenido la amistad, nos hemos visto crecer juntos y hemos compartido grandes momentos, de esos necesarios para desconectar, pudiendo contar con vuestro apoyo siempre que ha sido necesario.

A los amigos que he hecho durante estos años de universidad, que han hecho que este camino sea mucho mas fácil, tanto dentro como fuera de la universidad. Gracias por todo lo que me habéis enseñado y ayudado, que me ha permitido crecer tanto académicamente como personalmente.

Y por último, pero no por ello menos importante, a mi pareja, Mabilia, que ha sido un apoyo imprescindible, no solo en este proyecto, sino durante todos estos años de carrera, tanto en lo académico como, sobre todo, en lo personal. Muchas gracias por todos estos años de comprensión, cariño y diversión. Sin ti, este viaje no hubiese sido lo mismo. Sin duda, eres lo mejor que me llevo de la carrera.

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Estructura de la memoria	4
2. Estado del arte	5
2.1. Control y monitorización en robótica móvil	5
2.2. Ecosistema ROS 2 y el estándar DDS	6
2.3. Aplicaciones móviles	11
2.4. Proyectos similares	11
3. Planificación, metodología y presupuesto	13
3.1. Metodología de trabajo	13
3.2. Planificación temporal	14
3.3. Plan de gestión de riesgos y contingencias	16
3.4. Estimación de costes	16
4. Diseño y desarrollo del protocolo	19
4.1. Diseño del protocolo	19
4.2. Desarrollo del protocolo	31
5. Diseño y desarrollo de la aplicación	39
5.1. Descripción del problema y objetivos	39
5.2. Análisis de requisitos	40
5.3. Entorno de desarrollo y herramientas	44
5.4. Diseño de la arquitectura del sistema	45
5.5. Diseño gráfico	49
5.6. Diseño y desarrollo de las funcionalidades de la aplicación . .	53
6. Evaluación y resultados	59
6.1. Resultados finales y evaluación técnica	59
6.2. Evaluación del Middleware de ROS 2 (DDS)	61
6.3. Análisis y optimización del tráfico de red generado	63

6.4. Evaluación de requisitos de red	69
6.5. Validación de seguridad y tolerancia a fallos	72
7. Conclusión y Trabajo futuro	73
7.1. Conclusiones	73
7.2. Trabajo futuro	75
Bibliografía	76
Anexos	87
A. Manual de instalación y ejecución de la aplicación	88
A.1. Requisitos previos y de seguridad	88
A.2. Instalación y ejecución del backend	89
A.3. Instalación y ejecución del frontend	89
A.4. Puesta en funcionamiento	90
B. Diagramas de estados	91
C. Diagramas de secuencia de mensajes	96
D. Grafos de alcanzabilidad	99
D.1. Máquina de Estados Principal	99
D.2. Subestado: Movimiento	100
D.3. Subestado: Monitorización	100
E. Wireframes	104
E.1. Conexión y Configuración	104
E.2. Control del robot	105
E.3. Monitorización del robot	106

Índice de figuras

2.1. Arquitectura de ROS 2.	8
2.2. Nodos publicadores enviando el mensaje a través del <i>topic</i> . .	9
2.3. Nodos cliente y servidor de un <i>service</i>	10
2.4. Nodos cliente y servidor de un <i>action</i>	10
3.1. Diagrama de Gantt con la planificación temporal final del proyecto.	15
5.1. Diagrama de bloques de la arquitectura del servidor.	48
5.2. Diagrama de bloques de la arquitectura del cliente.	50
5.3. Logotipo e icono de la aplicación.	52
5.4. Paletas de colores de la aplicación.	52
5.5. Conjunto de pantallas e interfaces de la aplicación.	53
6.1. Consumo de ancho de banda del tráfico DDS (primeros 30 s). .	62
6.2. Comparativa del número total de paquetes RTPS durante el descubrimiento.	62
6.3. Consumo de ancho de banda durante el estado de reposo (60 segundos).	63
6.4. Tráfico generado con la aplicación en reposo.	65
6.5. Tráfico generado por las funcionalidades de la aplicación. . .	66
6.6. Análisis de frecuencias de teleoperación.	67
6.7. Análisis de frecuencias del sensor IMU.	68
6.8. Gráfica comparativa del submuestreo del LiDAR.	69
6.9. Interfaz gráfica del LiDAR con diferentes niveles de muestreo. .	69
6.10. Gráfica escenario A.	70
6.11. Gráfica escenario B.	71
6.12. Gráfica RTT.	71
B.1. Diagrama de estados del cliente.	92
B.2. Ampliación del diagrama de estados del cliente para visualizar la parte superior.	93
B.3. Ampliación del diagrama de estados del cliente para visualizar la parte inferior.	94

B.4. Diagrama de estados del servidor.	95
C.1. Resumen de los diagramas de secuencia de los mensajes del protocolo R2Pilot (Parte 1).	97
C.1. Resumen de los diagramas de secuencia de los mensajes del protocolo R2Pilot (Continuación).	98
D.1. Grafo de alcanzabilidad principal.	101
D.2. Grafo de alcanzabilidad del subestado “Movimiento”.	102
D.3. Grafo de alcanzabilidad del subestado “Monitorización”.	103
E.1. Bocetos interfaces de conexión y configuración.	104
E.2. Bocetos interfaces de control.	105
E.3. Bocetos pantalla dividida.	105
E.4. Bocetos interfaces de monitorización.	106

Índice de tablas

3.1. Principales riesgos identificados durante el desarrollo.	16
3.2. Estimación de costes de personal.	17
3.3. Estimación de amortización de hardware.	18
3.4. Estimación de costes indirectos.	18
3.5. Presupuesto total estimado del proyecto.	18
4.1. Evaluación de protocolos de comunicación.	22
4.2. Resumen de los mensajes del protocolo R2Pilot según su blo- que funcional.	25
4.3. Evaluación de formatos de serialización de datos.	29
4.4. Definición de los campos de la cabecera (Header) del protocolo.	31
4.5. Estructura de datos del Payload según el tipo de mensaje. . .	32
4.6. Módulos de software implementados para el procesamiento del protocolo.	35
5.1. Requisitos Funcionales (RF) de la aplicación.	41
5.2. Requisitos No Funcionales (RNF) de la aplicación.	42
5.3. Estructura de módulos y archivos del servidor (Backend). . .	47
5.4. Estructura de componentes del cliente móvil (Frontend - MVVM).	49
D.1. Leyenda para el grafo principal (figura D.1).	99
D.2. Leyenda para el subestado Movimiento (figura D.2).	100
D.3. Leyenda para el subestado Monitorización (figura D.3). . . .	100

Capítulo 1

Introducción

En este capítulo se introduce el contexto en el que surge el proyecto, así como las principales motivaciones que han llevado a su desarrollo y los objetivos que se pretenden alcanzar. Por último, se presenta la estructura de la memoria y se explica brevemente el contenido de los capítulos que la componen.

1.1. Contexto

Los robots son sistemas capaces de percibir su entorno, procesar la información obtenida y actuar mediante la integración de sensores, algoritmos y actuadores. Dentro de este ámbito se encuentran los robots móviles, caracterizados por su capacidad de desplazamiento autónomo y utilizados en tareas que requieren movilidad e interacción con distintos entornos [1, 2].

Aunque estos robots pueden realizar determinadas tareas de forma autónoma, la intervención de un operador sigue siendo necesaria cuando las condiciones del entorno o de la tarea requieren decisiones que el robot no puede tomar por sí mismo. En estos casos, el operador puede controlarlo de forma remota mediante el envío de órdenes y supervisar su funcionamiento a partir de la información recibida. La eficacia de esta interacción depende, entre otros factores, de la información disponible y de la calidad de la comunicación. Por ello, una interfaz que integre simultáneamente las funciones de control y monitorización puede facilitar esta interacción, especialmente en tareas de desarrollo, experimentación, pruebas y supervisión [3].

El desarrollo de estos sistemas se ve facilitado por plataformas software como ROS 2 (*Robot Operating System 2*), un conjunto de bibliotecas y herramientas de código abierto para aplicaciones robóticas [4]. Su arquitectura permite organizar las funcionalidades del robot en componentes independientes que se comunican entre sí, facilitando tanto su desarrollo como la

integración de aplicaciones externas de control y monitorización [5].

Estas aplicaciones pueden proporcionar interfaces adaptadas al operador, y los dispositivos móviles constituyen una alternativa interesante por su portabilidad y conectividad, permitiendo interactuar con el robot mientras el usuario se desplaza por su entorno. Sin embargo, su utilización requiere establecer una comunicación con el sistema robótico y adaptar sus funcionalidades e información a las características de la interfaz móvil. Estas necesidades, junto con la complejidad de los sistemas robóticos, constituyen el punto de partida de este proyecto.

1.2. Motivación

El origen de este proyecto parte de una necesidad práctica detectada por el tutor en el trabajo habitual con robots en el laboratorio. En particular, el laboratorio singular WIN-NET-LAB (*Wireless and INnovative NETworks LAB*) de la Universidad de Granada, que cuenta con diferentes plataformas robóticas, entre ellas el robot TIAGo de PAL Robotics, utilizadas en tareas de desarrollo, experimentación y pruebas [6].

Este contexto puso de manifiesto la utilidad de disponer de una herramienta accesible desde dispositivos móviles que facilitara la interacción con estos sistemas, motivando el desarrollo de una aplicación móvil orientada al control y la monitorización de robots basados en ROS 2. A ello se suma el reducido número de soluciones móviles de este tipo, como se analizará posteriormente en el capítulo 2.

Desde el inicio también se consideró importante que la solución no estuviera limitada a un único robot. Aunque para el desarrollo se utilice el robot TIAGo, la herramienta se ha planteado de forma que pueda adaptarse a otros robots basados en ROS 2 y utilizarse en diferentes entornos de investigación.

Desde el punto de vista académico, el proyecto ha supuesto una oportunidad para ampliar los conocimientos adquiridos durante el grado y abordar tecnologías no trabajadas previamente. Los conocimientos en programación y diseño de protocolos se han aplicado y profundizado durante el desarrollo, mientras que ha sido necesario adquirir experiencia en tecnologías como Kotlin, ROS 2, DDS (Data Distribution Service) y WebSocket. Además, el uso de Docker, simuladores y robots reales ha permitido aplicar estos conocimientos en un entorno que se acerca al profesional.

1.3. Objetivos

A partir del contexto y las motivaciones planteadas en las secciones anteriores, el objetivo principal de este Trabajo de Fin de Grado es desarrollar y evaluar una aplicación móvil para Android que permita controlar y monitorizar robots basados en ROS 2 de forma general, independientemente de la plataforma utilizada. Además, se busca que su diseño modular facilite la realización de adaptaciones o la incorporación de funcionalidades específicas cuando sea necesario ajustar la aplicación a las características de un robot concreto.

Para alcanzar este objetivo, se han definido los siguientes objetivos específicos:

- **OE1. Analizar** las necesidades relacionadas con el control y la monitorización de robots basados en ROS 2, y establecer los requisitos funcionales y no funcionales que debe cumplir la aplicación.
- **OE2. Definir** una solución que permita la interacción remota entre la aplicación móvil y el robot, garantizando el intercambio bidireccional de órdenes e información.
- **OE3. Integrar** las capacidades de control, monitorización y gestión proporcionadas por ROS 2 con la aplicación móvil.
- **OE4. Desarrollar** una aplicación Android que proporcione una interfaz eficiente y cómoda para interactuar con las diferentes capacidades del robot.
- **OE5. Diseñar** una solución modular que permita su uso con diferentes plataformas robóticas basadas en ROS 2 y facilite su adaptación a las características y necesidades específicas de cada robot.
- **OE6. Evaluar** el funcionamiento, rendimiento y robustez de la solución mediante pruebas realizadas en diferentes configuraciones y plataformas basadas en ROS 2.

El proyecto también plantea una serie de objetivos personales orientados a ampliar y poner en práctica los conocimientos y competencias adquiridos durante el grado:

- **Adquirir y aplicar** conocimientos sobre el desarrollo de aplicaciones móviles nativas para Android mediante Kotlin y sobre el diseño de interfaces de usuario.
- **Profundizar** en el funcionamiento de ROS 2 y sus mecanismos de comunicación, especialmente DDS.

- **Aplicar** metodologías y herramientas propias de proyectos de ingeniería, como Docker, entornos simulados y reales, así como mejorar las capacidades de planificación y gestión de un proyecto de larga duración.

1.4. Estructura de la memoria

La memoria se estructura en siete capítulos, además de la bibliografía y los anexos. El capítulo 1 introduce el contexto, las motivaciones y los objetivos del proyecto. El resto de la memoria se organiza de la siguiente forma:

- **Capítulo 2. Estado del arte:** se presenta el contexto tecnológico necesario para comprender el desarrollo del proyecto, introduciendo los principales conceptos y tecnologías en los que se basa. Además, se analizan diferentes proyectos relacionados.
- **Capítulo 3. Planificación y estimación de costes:** se recoge la planificación temporal del proyecto, la estimación de costes y los principales riesgos identificados.
- **Capítulo 4. Diseño y desarrollo del protocolo:** se presenta el diseño e implementación del protocolo de comunicación desarrollado para conectar la aplicación móvil con el robot, incluyendo su funcionamiento, estados y formato de los mensajes.
- **Capítulo 5. Diseño y desarrollo de la aplicación:** se presentan los requisitos del sistema y las decisiones adoptadas para su diseño y desarrollo. Se describe la arquitectura de la solución, el diseño de la interfaz y la estructura de sus principales componentes, así como la implementación de las distintas funcionalidades.
- **Capítulo 6. Evaluación:** se analiza el funcionamiento de la solución mediante pruebas de rendimiento, comunicaciones, tráfico de red y tolerancia ante fallos.
- **Capítulo 7. Conclusiones y trabajo futuro:** se recogen las principales conclusiones del proyecto y las posibles líneas de mejora y ampliación de la solución.

Capítulo 2

Estado del arte

Este capítulo presenta el contexto tecnológico del proyecto y analiza las principales soluciones disponibles. Para ello, se abordan los fundamentos del control y la monitorización de robots móviles, ROS 2 y su sistema de comunicaciones DDS, así como Android como plataforma de desarrollo móvil. También se revisan herramientas y proyectos similares para conocer sus características y limitaciones y tomar como referencia las soluciones existentes.

La información empleada procede principalmente de documentación técnica y publicaciones tecnológicas en inglés, debido a la mayor disponibilidad de recursos actualizados en este idioma. Para garantizar su fiabilidad, se han priorizado fuentes oficiales y de referencia, como la documentación de Open Robotics, las especificaciones de fabricantes, por ejemplo de implementaciones DDS, los repositorios originales de GitHub y las publicaciones de autores y organizaciones reconocidas.

2.1. Control y monitorización en robótica móvil

Un robot móvil es un sistema capaz de desplazarse e interactuar con el entorno, lo que le permite trabajar en espacios desconocidos y cambiantes. A diferencia de los robots industriales tradicionales, que operan desde una posición fija, los robots móviles pueden percibir su entorno y actuar de forma autónoma o ser guiados por un operador humano. Por ello, se utilizan en ámbitos como la logística, la industria y la investigación, donde su control remoto resulta especialmente útil para explorar entornos, realizar pruebas o validar el comportamiento del robot antes de utilizar sistemas autónomos [7, 8, 9].

Para controlar un robot a distancia de forma segura, el operador necesita recibir información sobre su estado y entorno. Esta información se obtiene mediante distintos sensores, entre los que destacan los sensores de distancia,

como el *LiDAR*, las cámaras y los sensores de estado y movimiento, como las unidades de medición inercial (IMU), o los sensores de fuerza/par. Cada uno proporciona información diferente que debe estar disponible para el operador durante el control [10, 11, 12].

Por tanto, se requiere una comunicación bidireccional entre el robot y la estación de control. El robot debe transmitir continuamente información de sus sensores y recibir las órdenes del operador para ejecutarlas correctamente, todo ello en tiempo real.

Esta comunicación hace que la red deba cumplir unos requisitos exigentes, principalmente en dos aspectos [13]:

- **Alto ancho de banda:** debe existir capacidad suficiente para transmitir simultáneamente vídeo, datos de sensores y comandos de control sin saturar la red.
- **Baja latencia (tiempo real):** es un aspecto crítico, ya que un retardo elevado puede provocar que el robot continúe ejecutando una acción antes de recibir una nueva orden, especialmente en situaciones de frenado o cambio de dirección. Además, el operador puede recibir información desactualizada del estado del robot, dificultando su control y aumentando el riesgo de errores o colisiones.

2.2. Ecosistema ROS 2 y el estándar DDS

En esta sección se presentan el ecosistema ROS 2 y su estándar de comunicaciones subyacente, DDS (*Data Distribution Service*), que constituyen una de las principales infraestructuras empleadas actualmente en robótica y que, además, son las utilizadas en los robots objeto de este proyecto.

2.2.1. Arquitectura de ROS 2

ROS 2 (*Robot Operating System 2*) es un *middleware* robótico de código abierto ampliamente utilizado en el desarrollo de sistemas robóticos modernos. Concretamente, se trata de un conjunto de librerías y herramientas de *software* diseñadas para facilitar la creación de aplicaciones robóticas, mediante la provisión de una infraestructura que permite la comunicación, coordinación y gestión de los distintos componentes físicos y lógicos que conforman un robot. El desarrollo de ROS 2 se organiza mediante distintas distribuciones, como *Foxy*, *Galactic* o *Humble*, que incorporan nuevas mejoras y se adaptan a la evolución de los sistemas operativos. [14, 15, 16]

Uno de los aspectos fundamentales de ROS 2 es su arquitectura distribuida, en la que el sistema se organiza en múltiples nodos independientes

que se ejecutan de manera concurrente y se comunican entre sí. Esta división en nodos permite estructurar el sistema complejo en componentes más pequeños, facilitando así el análisis y la interacción con el robot.[5, 17]

Estos nodos, junto con sus dependencias y archivos de configuración, se agrupan lógicamente en paquetes (*packages*). Su desarrollo se realiza mediante librerías cliente, entre las que destacan `rclcpp` para C++ y `rclpy` para Python, que proporcionan la API necesaria para interactuar con ROS 2. [18, 19]

A diferencia de ROS 1, que usaba protocolos propios, ROS 2 se apoya en el estándar Data Distribution Service (DDS), que es utilizado como base para la interfaz de middleware de ROS 2 (RMW), la cual permite la comunicación entre nodos. Como existen varias implementaciones de DDS, ROS 2 utiliza una interfaz abstracta que permite a la biblioteca cliente comunicarse con cualquier implementación, mientras que adaptadores específicos traducen esa interfaz a la API concreta de cada middleware. [20]

Además, ROS 2 utiliza variables de entorno para configurar distintos aspectos de la comunicación. Entre ellas destacan `RMW_IMPLEMENTATION`, que permite seleccionar la implementación de DDS, y `ROS_DOMAIN_ID`, que establece la red lógica en la que trabaja cada robot. Esta última permite separar robots que comparten una misma red física, evitando que sus mensajes interfieran entre sí. [21, 22, 23]

En la figura 2.1 se puede encontrar una imagen que representa la arquitectura descrita.

2.2.2. Estándar DDS

El *Data Distribution Service* (DDS) es un estándar de *middleware* y una API desarrollada por el *Object Management Group* (OMG) para el intercambio de información en sistemas distribuidos. Se basa en un modelo de publicación/suscripción que proporciona comunicación fiable, baja latencia y escalabilidad, por lo que resulta adecuado para sistemas con requisitos de tiempo real. [25, 26, 27]

Para la comunicación en red, DDS utiliza el protocolo RTPS (*Real-Time Publish-Subscribe*), que define el formato y transmisión de los mensajes y permite la interoperabilidad entre distintas implementaciones de DDS. [28, 25]

Como se introdujo en la sección 2.2.1, ROS 2 utiliza la capa RMW (*ROS Middleware*) para abstraer la implementación concreta de DDS. Entre las opciones disponibles destacan *Fast DDS*, de eProsima, y *Cyclone DDS*, de Eclipse, ambas utilizadas posteriormente para analizar el comportamiento de la red. [29, 30, 31]

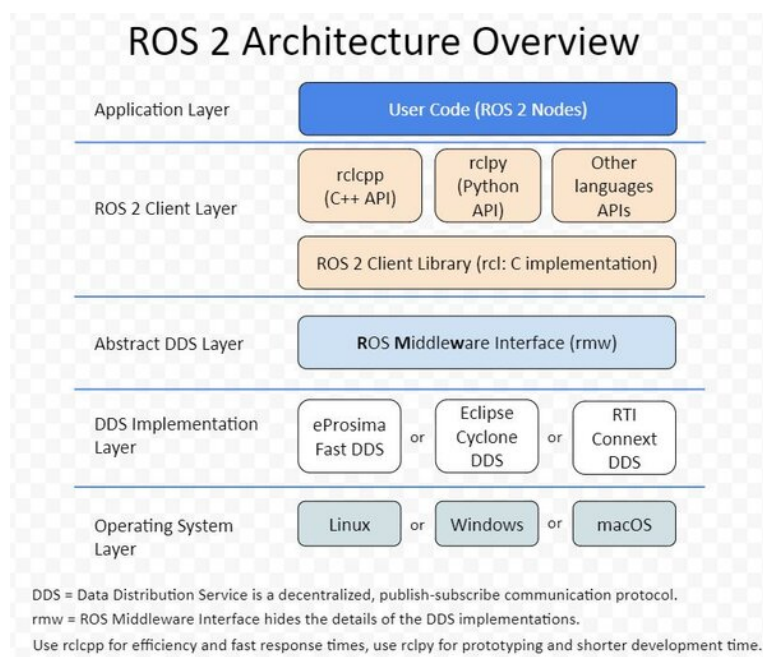


Figura 2.1: Arquitectura de ROS 2.

Fuente: Imagen obtenida de la fuente [24].

Una característica fundamental de DDS es la Calidad de Servicio (QoS), que permite establecer requisitos sobre la distribución de los datos. Entre sus políticas destaca *Reliability*, con las opciones *Best Effort*, que reduce la sobrecarga sin garantizar la entrega de todos los datos, y *Reliable*, que garantiza su recepción. Para establecer la comunicación, las configuraciones de QoS del publicador y el suscriptor deben ser compatibles, de lo contrario, los nodos no podrán intercambiar información.[32, 33]

DDS también incorpora un mecanismo de descubrimiento dinámico que permite a los nodos encontrarse y establecer comunicación sin un servidor central. Por defecto, este proceso utiliza *UDP multicast* para anunciar los nodos e intercambiar información sobre sus publicadores y suscriptores. En redes donde el *multicast* puede generar una sobrecarga elevada, como Wi-Fi, algunas implementaciones ofrecen alternativas. En particular, *Fast DDS* proporciona *Discovery Server*, que centraliza el descubrimiento mediante comunicación *unicast*, reduciendo el tráfico de la red. [34]

2.2.3. Modelo de comunicación de ROS 2

Como se ha explicado en la sección 2.2.1, ROS 2 utiliza una arquitectura distribuida en la que diferentes nodos trabajan de forma independiente.

Para comunicarse e intercambiar información, ROS 2 proporciona tres mecanismos principales:

Tópicos (*Topics*): constituyen uno de los principales mecanismos de comunicación de ROS 2 y utilizan un modelo de publicación/suscripción (*publish/subscribe*). Los nodos publicadores envían datos a través de un *topic*, mientras que los suscriptores reciben la información correspondiente. Este modelo es anónimo y desacoplado, permitiendo que múltiples nodos publiquen o reciban datos sin conocerse directamente, siempre que utilicen el mismo nombre y tipo de mensaje (Figura 2.2) [35, 36].

Además, el uso de nombres estandarizados, como `cmd_vel` para órdenes de movimiento o `scan` para datos de láser, facilita la interoperabilidad entre aplicaciones y robots. Estas convenciones se recogen en las ROS Enhancement Proposals (REPs) [37].

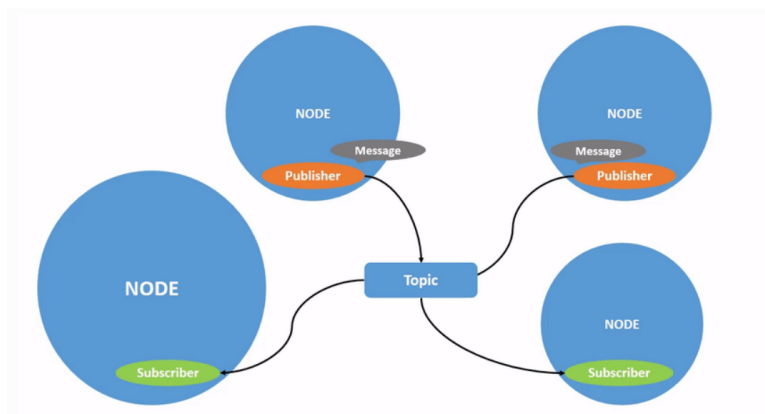
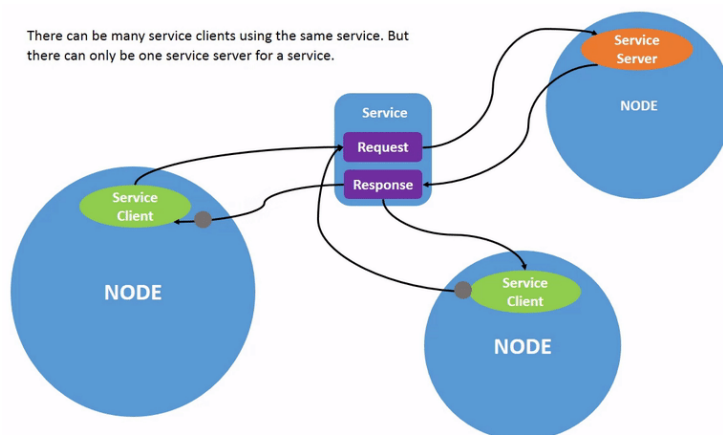


Figura 2.2: Nodos publicadores enviando el mensaje a través del *topic*

Fuente: Imagen obtenida de la fuente [36].

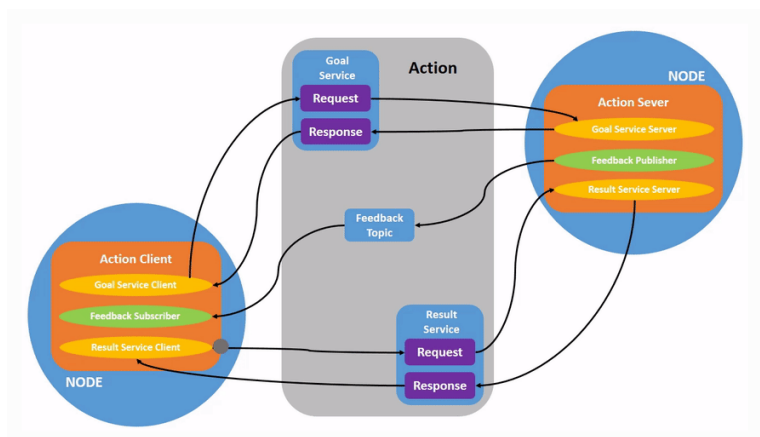
Servicios (*Services*): utilizan un modelo de petición–respuesta, en el que un nodo (*service client*) solicita una operación y otro (*service server*) la procesa y devuelve el resultado mediante dos tópicos, uno para la petición y otro para la respuesta. Un servicio puede tener varios clientes, pero un único servidor. Puesto que el cliente suele permanecer bloqueado hasta recibir la respuesta, este mecanismo está orientado a operaciones rápidas y no resulta adecuado para procesos de larga duración, para los que se emplean las acciones (Figura 2.3) [38, 39].

Acciones (*Actions*): están diseñadas para operaciones de mayor duración que requieren seguimiento del progreso y posibilidad de cancelación. Se basan en un modelo cliente–servidor y proporcionan tres elementos: un

Figura 2.3: Nodos cliente y servidor de un *service*

Fuente: Imagen obtenida de la fuente [39].

goal, que define el objetivo de la acción, un *feedback*, que informa sobre su progreso, y un *result*, que contiene el resultado final. Internamente, estos elementos se implementan mediante servicios y tópicos (Figura 2.4) [40, 41].

Figura 2.4: Nodos cliente y servidor de un *action*

Fuente: Imagen obtenida de la fuente [41].

Además, para garantizar una comunicación estructurada, ROS 2 utiliza **interfaces** fuertemente tipadas. Los datos intercambiados mediante tópicos, servicios y acciones se definen en archivos `.msg`, `.srv` y `.action`, a partir de los cuales ROS 2 genera el código necesario para permitir la comunicación entre nodos desarrollados en distintos lenguajes, como C++ y Python. La

definición de todas estas interfaces, su propósito y su estructura interna se encuentran recopiladas en la API oficial de ROS 2 [42, 43].

2.3. Aplicaciones móviles

En las últimas décadas, los teléfonos inteligentes (*smartphones*) se han extendido rápidamente, convirtiéndose en dispositivos habituales para acceder a información, servicios y aplicaciones móviles. Concretamente, en abril de 2026, se contabilizaban 5.830 millones de usuarios móviles únicos en todo el mundo, equivalentes al 70,4 % de la población mundial [44].

Esta evolución también se ha reflejado en las capacidades de los propios dispositivos. Los *smartphones* actuales integran una amplia variedad de sensores y tecnologías de comunicación que permiten a las aplicaciones obtener información del dispositivo y de su entorno, así como comunicarse con otros dispositivos y servicios externos. Gracias a estas capacidades, las aplicaciones móviles pueden desempeñar funciones cada vez más diversas y extender su uso a distintos ámbitos, aprovechando características como la portabilidad y la disponibilidad de estos dispositivos. [45, 46]

Dentro del mercado de sistemas operativos móviles, Android es el más usado a nivel mundial. Según StatCounter, en julio de 2026 Android representaba el 68,36 % del uso de sistemas operativos móviles en todo el mundo, frente al 31,60 % de iOS. [47] Esta presencia también se refleja en las ventas de *smartphones*, donde según Counterpoint Research, los dispositivos Android representaron el 73 % de las ventas mundiales durante el primer trimestre de 2026. [48] Estas cifras muestran la amplia presencia de Android en el mercado, donde la plataforma está disponible en dispositivos de numerosos fabricantes y en una gran variedad de modelos y características.

Desde el punto de vista del desarrollo, Android dispone de un amplio ecosistema de herramientas y bibliotecas para la creación de aplicaciones. Android Studio es el entorno de desarrollo integrado oficial e incorpora herramientas para la edición, compilación, depuración y pruebas de aplicaciones [49]. Kotlin es el lenguaje recomendado por Google para Android [50], mientras que Jetpack Compose proporciona un *toolkit* moderno para la creación de interfaces mediante un modelo declarativo basado en Kotlin [51].

2.4. Proyectos similares

Existen diferentes proyectos destinados a facilitar el control y la monitorización de robots mediante dispositivos móviles. El análisis de estas soluciones permite identificar las funcionalidades que ya han sido desarrolla-

das y las principales limitaciones existentes, sirviendo como referencia para definir el alcance de la aplicación desarrollada en este trabajo.

Uno de los primeros fue **ROS-Mobile** [52], una aplicación Android para el control y monitorización remota de robots, que incluye funciones de teleoperación y visualización de datos. Sin embargo, está desarrollada para ROS 1, por lo que no es directamente compatible con ROS 2, pero resulta una referencia interesante por su enfoque y por las funcionalidades que ofrece.

Como evolución de este proyecto se encuentra **ROS2-Mobile-Android** [53], que adapta parte de sus funcionalidades a ROS 2 mediante `rc1java`. Aunque permite teleoperación y visualización, continúa en desarrollo y presenta limitaciones en determinadas funcionalidades, como la visualización del vídeo de la cámara. Este proyecto evidencia las dificultades de integrar directamente ROS 2 en Android.

También existen soluciones basadas en *WebSocket*, como **ROS2 Connect Mobile** [54], centrada en la monitorización para iOS, y **ROS2 Ackermann Teleop** [55], orientada a la teleoperación. Estas últimas emplean una capa intermedia para comunicarse con ROS 2, evitando su integración directa en el dispositivo móvil.

Finalmente, durante el desarrollo de este TFG apareció **ROS2 Visualizer** [56], orientada a la visualización e interacción con sistemas ROS 2. Al publicarse posteriormente al inicio del proyecto, no se pudo utilizar como referencia para definir los requisitos ni las decisiones de diseño, aunque su desarrollo muestra el interés por este tipo de herramientas.

En conjunto, las soluciones analizadas presentan diferentes enfoques, pero ninguna reúne todas las características planteadas en este trabajo. Algunas están limitadas a ROS 1, otras se centran en funcionalidades concretas o en plataformas distintas de Android, mientras que la integración directa de ROS 2 en dispositivos móviles presenta dificultades. Por ello, este TFG propone una aplicación Android de propósito general que centraliza la configuración, el control y la monitorización de robots ROS 2. Además, se observa que el uso de una arquitectura cliente-servidor permitiría reducir parte de las dificultades de la integración directa de ROS 2 en Android, ofreciendo una solución más flexible y adaptable a distintos robots y escenarios.

Capítulo 3

Planificación, metodología y presupuesto

En este capítulo se explica la metodología seguida para desarrollar el proyecto, así como la planificación de las distintas tareas y fases que lo forman. También se incluye una estimación de los recursos necesarios y de los costes asociados a su realización.

3.1. Metodología de trabajo

El proyecto se ha desarrollado de forma individual siguiendo un enfoque iterativo e incremental [57]. Este enfoque permitió implementar y validar progresivamente las distintas funcionalidades, manteniendo una versión funcional de la aplicación durante el desarrollo.

El proceso de desarrollo se estructuró en las siguientes fases:

Fase de investigación inicial. En primer lugar, se investigaron las tecnologías y herramientas disponibles para desarrollar una aplicación Android capaz de comunicarse con un sistema basado en ROS 2. También se analizaron distintas alternativas de comunicación y proyectos similares para conocer las opciones existentes.

Fase de aprendizaje. Una vez identificadas las tecnologías necesarias, se dedicó una etapa a adquirir los conocimientos que no se dominaban previamente, estudiando conceptos de ROS 2 y DDS, además de Kotlin y los fundamentos del desarrollo de aplicaciones Android.

Fase de diseño de la comunicación. Una vez adquiridos los conocimientos necesarios y definida una arquitectura cliente-servidor, se analizaron distintas alternativas de comunicación entre el *frontend* y el

backend. Finalmente, se optó por desarrollar un protocolo de comunicación propio, cuyo diseño se realizó durante esta fase.

Fase de diseño de la aplicación e implementación inicial. Una vez definido el protocolo, se diseñó la aplicación, incluyendo el análisis de requisitos, la estructura del *frontend* y *backend*, y el diseño inicial de la interfaz. Posteriormente, se desarrolló una primera versión básica para comprobar la comunicación entre ambos componentes y validar el funcionamiento del protocolo antes de incorporar las distintas funcionalidades.

Fase de desarrollo de la aplicación. A partir de la arquitectura inicial, se desarrollaron progresivamente las funcionalidades de la aplicación, utilizando inicialmente una interfaz sencilla para facilitar su implementación y validación. Una vez comprobada la lógica principal, se diseñó e implementó la interfaz definitiva, mejorando la claridad y experiencia de uso.

Fase de validación y pruebas. Las pruebas se realizaron progresivamente durante el desarrollo, comprobando cada funcionalidad primero en simulación y posteriormente en el robot real. Esto permitió detectar y corregir errores antes de continuar con el resto de la aplicación. Finalmente, se realizó una última validación para comprobar el funcionamiento conjunto y evaluar las comunicaciones, el rendimiento, el tráfico de red y la tolerancia ante posibles fallos.

Fase de elaboración de la memoria. La elaboración de la memoria se llevó a cabo de forma paralela al desarrollo del proyecto. A lo largo de las distintas fases se fueron recopilando y documentando las decisiones adoptadas, los problemas encontrados, las soluciones implementadas y los resultados obtenidos.

Para organizar las tareas y la información del proyecto se utilizó **Notion** [58]. El código se gestionó mediante **GitHub** [59], manteniendo repositorios independientes para el *frontend* y el *backend*. Además, GitHub facilitó el trabajo entre los distintos equipos utilizados.

3.2. Planificación temporal

La planificación temporal se ha estructurado en base a las siete fases definidas en la metodología (Sección 3.1). La Figura 3.1 muestra el diagrama de Gantt con el cronograma final y la distribución real del trabajo a lo largo del proyecto.

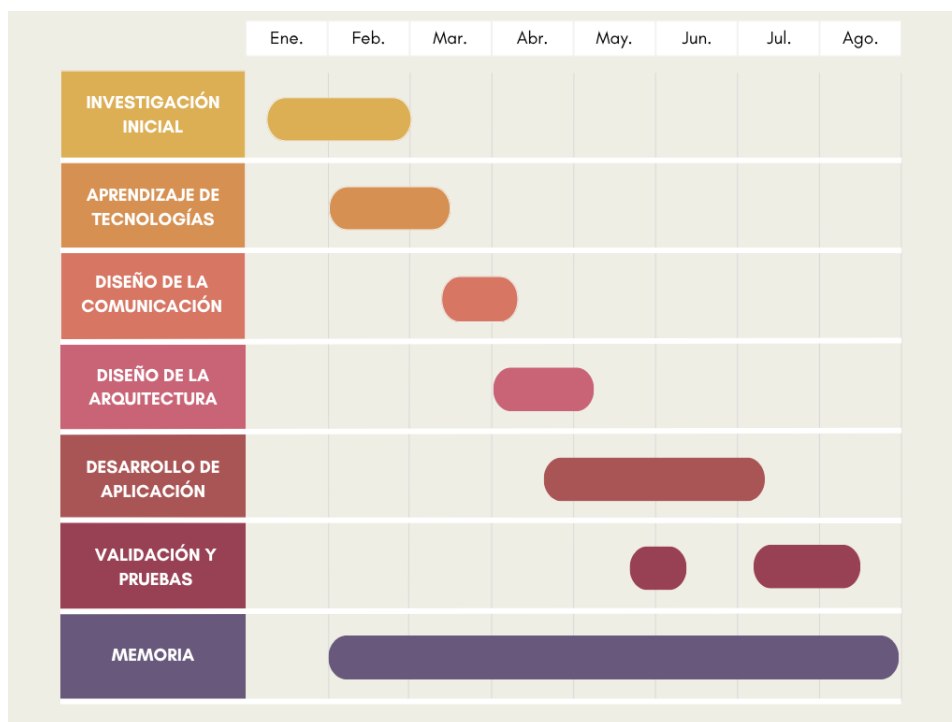


Figura 3.1: Diagrama de Gantt con la planificación temporal final del proyecto.

Análisis de desviaciones

La planificación inicial establecía junio como fecha de finalización. Sin embargo, durante el desarrollo surgieron dificultades que obligaron a modificar los plazos y trasladar la finalización a septiembre. La principal desviación estuvo relacionada con la arquitectura de comunicación.

Inicialmente, se planteó integrar ROS 2 directamente en la aplicación Android para permitir la comunicación entre el dispositivo móvil y el robot. Sin embargo, durante la implementación surgieron problemas con la biblioteca utilizada, lo que llevó a replantear la arquitectura y adoptar finalmente una solución cliente-servidor, con un servidor alojado en el robot y un protocolo propio de comunicación.

Este cambio implicó tareas no contempladas inicialmente, como el diseño e implementación del protocolo y la definición de la arquitectura de ambos extremos. Su desarrollo supuso un retraso de varias semanas y fue la principal causa de la ampliación del proyecto. Como consecuencia, fue necesario reorganizar las etapas restantes, dando lugar a la planificación final mostrada en el diagrama de Gantt.

También se produjeron desviaciones menores durante las fases de aprendizaje, diseño y pruebas. La complejidad de ROS 2, DDS y de la comunicación entre ambos extremos prolongó las primeras etapas, mientras que las pruebas sobre el robot real requirieron ajustes adicionales debido a diferencias respecto al entorno simulado.

3.3. Plan de gestión de riesgos y contingencias

Para facilitar el desarrollo del proyecto y reducir el impacto de posibles imprevistos, se elaboró un plan de gestión de riesgos. En la siguiente tabla se recogen los principales riesgos identificados, junto con su nivel de impacto y las medidas adoptadas para prevenirlos o reducir sus efectos (Tabla 3.1):

Tabla 3.1: Principales riesgos identificados durante el desarrollo.

Riesgo	Probabilidad	Impacto	Prevención	Mitigación
R1 - Diferencias entre simulación y robot real	Media	Alto	Validación progresiva en simulador y robot.	Ajustes tras las pruebas en el robot real.
R2 - Curva de aprendizaje de nuevas tecnologías (Kotlin, rclpy...)	Alta	Medio	Formación inicial en las tecnologías necesarias.	Aprendizaje progresivo y consulta de documentación.
R3 - Retraso en la finalización del proyecto por un alcance excesivo	Media	Alto	Priorizar funcionalidades principales y desarrollo una a una.	Reducir o posponer funcionalidades secundarias.
R4 - Problemas de integración y comunicación de ROS 2 y Android	Alta	Alto	Evaluación previa de arquitectura y alternativas de comunicación.	Separación frontend-backend y protocolo de comunicación propio.

3.4. Estimación de costes

En esta sección se presenta una estimación del presupuesto necesario para desarrollar este Trabajo de Fin de Grado, teniendo en cuenta los costes

de personal, equipamiento (*hardware* y *software*) y otros costes indirectos.

3.4.1. Costes de personal

La mayor parte del esfuerzo del proyecto corresponde al tiempo dedicado al desarrollo, la investigación y la redacción. Se estima una dedicación total de 450 horas por parte del alumno, tomando como referencia el coste de un perfil de Ingeniero Junior, con un sueldo estimado de 13,84 €/h [60]. Además, se contemplan 20 horas de supervisión y asesoramiento técnico por parte del tutor, con un coste estimado de 26,21 €/h. [61]

Concepto	Horas	Coste unitario (€/h)	Coste total (€)
Ingeniero Junior	450	13,84	6.228,00
Tutor del proyecto	20	26,21	524,20
Total costes de personal			6.752,20

Tabla 3.2: Estimación de costes de personal.

3.4.2. Costes de hardware y software

Para calcular el coste del *hardware* utilizado, se aplica una amortización lineal con una vida útil estimada de 4 años (48 meses) para los equipos informáticos y dispositivos móviles. De este modo, se divide el precio de cada equipo entre su vida útil en meses y se multiplica por los meses de uso para obtener el coste amortizado.

En el caso del robot TIAGo utilizado para las pruebas, no se considera un coste de adquisición, ya que es un equipo proporcionado por la universidad. Sin embargo, se incluye como recurso *hardware* necesario para realizar las pruebas sobre el sistema físico.

Concepto	Precio (€)	Vida útil	Uso	Amortización (€)
Portátil LG Gram	1.159,00	48	8	193,17
PC sobremesa (simulación)	1.630,17	48	8	271,70
Móvil Samsung Galaxy S25	685,80	48	5	71,44
Robot TIAGo	—	—	—	0,00
Total costes de hardware				536,31

Tabla 3.3: Estimación de amortización de hardware.

3.4.3. Costes indirectos

Se contemplan los gastos derivados del consumo eléctrico, conexión a Internet y uso del espacio de trabajo. Se ha estimado un coste fijo mensual de 45 € durante los 8 meses de duración del proyecto.

Concepto	Coste/mes (€)	Meses	Coste total (€)
Internet	5,00	8	40,00
Luz	40,00	8	320,00
Total costes indirectos			360,00

Tabla 3.4: Estimación de costes indirectos.

3.4.4. Presupuesto total

A partir de las estimaciones anteriores, se obtiene el coste total del proyecto sumando las distintas partidas de costes consideradas.

Partida	Coste total (€)
Costes de personal	6.752,20
Costes de hardware y software	536,31
Costes indirectos	360,00
PRESUPUESTO TOTAL	7.648,51

Tabla 3.5: Presupuesto total estimado del proyecto.

Capítulo 4

Diseño y desarrollo del protocolo

Como se estudia en el capítulo 5, el diseño de la aplicación está basado en dos bloques diferenciados: el *backend*, que contiene la lógica de la aplicación y es el encargado de comunicarse con el robot mediante el estándar DDS, y el *frontend*, encargado de la interacción del usuario final desde el dispositivo móvil con el servidor.

Por ello, para garantizar el correcto funcionamiento del sistema, es necesario establecer un protocolo, al que en adelante denominaremos **protocolo R2Pilot**, que permita la comunicación entre ambos extremos. Tanto el robot como el dispositivo móvil deben ser capaces de comunicarse entre sí para transmitir las órdenes del usuario y compartir la información necesaria sobre el estado del robot. Además, este intercambio de información debe realizarse de forma estructurada, sincronizada y en tiempo real, de modo que tanto el *frontend* como el *backend* mantengan en todo momento una visión coherente y actualizada del estado del robot. De esta manera, se consigue una respuesta coordinada y fiable ante las distintas acciones o eventos que puedan producirse durante el funcionamiento del sistema.

A lo largo de este capítulo, veremos todos los pasos que se han llevado a cabo para lograr diseñar e implementar dicho protocolo de comunicación.

4.1. Diseño del protocolo

Esta sección aborda el diseño del protocolo de aplicación, definiendo las reglas, el comportamiento y la estructura de los mensajes necesarios para garantizar la correcta comunicación entre los dos módulos de la aplicación.

Para empezar, se analizan los requisitos y necesidades del protocolo, y se

estudian las soluciones existentes para determinar si es necesario desarrollar uno propio. A partir de este análisis, se selecciona la capa de transporte más adecuada para establecer el canal de comunicación. Para terminar, se diseña y valida el protocolo, definiendo su comportamiento mediante diagramas de secuencia y de estados, así como la sintaxis y estructura de los mensajes intercambiados.

4.1.1. Análisis de los requisitos del protocolo

En primer lugar, se ha realizado una reflexión sobre las características que debería presentar el protocolo para conseguir el funcionamiento esperado de la aplicación. Para ello, se han analizado los requisitos de la aplicación y se ha deducido lo siguiente:

- **Baja latencia y tiempo real:** puesto que el objetivo principal de la aplicación es el control y monitorización de un robot, este no tiene sentido si no es en tiempo real. Además, hay funcionalidades específicas, como la teleoperación y la lectura de sensores del robot, que necesitan un retardo mínimo para que su funcionamiento sea correcto y libre de riesgos. Para ello, ante la alta frecuencia de los mensajes, es necesaria una estructura ligera de los mismos, para evitar retardos y un uso del ancho de banda excesivos.
- **Comunicación bidireccional (*Full-Duplex*) y concurrencia:** según el diseño de la aplicación, tanto el cliente como el servidor deben ser capaces de recibir y enviar información a la vez, con el objetivo de que el usuario pueda hacer uso de distintas funcionalidades simultáneamente, como controlar el robot y ver la información de los sensores al mismo tiempo.
- **Robustez y tolerancia a fallos:** en entornos inalámbricos, como es el caso, es muy común que hayan pequeñas desconexiones de red. Por ello, el protocolo debe detectar estas pérdidas instantáneamente y garantizar la seguridad del sistema y del robot. Además, se debe verificar la integridad de los datos antes de realizar cualquier acción para evitar posibles situaciones no esperadas y no seguras.
- **Interoperabilidad:** al comunicar dos entornos tan distintos como Android (Kotlin) y ROS 2 (Python), el protocolo debe basarse en un estándar de serialización de datos que sea independiente del lenguaje, ligero y fácil de procesar por ambas plataformas.
- **Seguridad y control de acceso:** es necesario garantizar que, al abrir el puerto de comunicación en el servidor, el cliente móvil sea el único que pueda comunicarse con el robot. Para ello, el protocolo debe ser

capaz de gestionar sesiones únicas y validar la identidad del emisor en cada paquete de datos transmitido.

- **Escalabilidad y fácil manipulación:** el diseño debe ser sencillo y flexible para que cada usuario pueda manipularlo y adecuarlo a las funcionalidades de su robot, sin necesidad de redefinir ni rediseñar ninguna parte del protocolo.

4.1.2. Alternativas existentes

Tras haber comprendido que se necesita esa conexión entre servidor y cliente, y los requisitos que esta debe cumplir, se ha investigado si actualmente ya existía algún método que nos permitiese conseguirla. Finalmente, se encontraron dos soluciones consolidadas en el ecosistema de ROS 2 para la comunicación con clientes externos, *rosbridge_suite* y *Foxglove Bridge*.

***rosbridge_suite*:** es un conjunto de paquetes que proporciona una API en formato JSON para que clientes externos a ROS puedan interactuar con sus nodos. A través de un servidor de WebSockets, permite que aplicaciones que no ejecutan ROS de forma nativa puedan publicar mensajes, suscribirse a tópicos y realizar llamadas a servicios mediante un protocolo basado en texto que traduce objetos JSON en comandos de ROS2 [62, 63, 64].

***Foxglove Bridge*:** es una solución parecida a *rosbridge_suite*, ya que usa un protocolo parecido, pero está mucho más orientada a ofrecer un alto rendimiento para la transmisión de grandes volúmenes de datos en tiempo real, como vídeo o nubes de puntos. Además, está estrechamente integrado con el ecosistema de Foxglove Studio [65, 66].

Sin embargo, ninguna de estas dos soluciones resultan realmente útiles para este proyecto, ya que el objetivo de la aplicación no es solo enviar y recibir peticiones de suscripción/publicación al robot, sino que se ofrecen también otro tipo de funcionalidades que necesitan de otras comunicaciones entre cliente y servidor. Además, ninguno de los dos puentes cuenta con un cliente sólido para Android, y en el caso de Foxglove Bridge no se podría desarrollar ya que no existe documentación actualizada de su protocolo de comunicación.

Otros motivos que empujan a crear un protocolo propio para la aplicación es la eficiencia, ya que muchas de las funcionalidades que ofrecen estas soluciones no se van a utilizar. De esta manera, podemos crear un protocolo que incluya justo los elementos necesarios para el correcto funcionamiento de la aplicación, consiguiendo una estructura de mensajes ligera que favorecerá al buen aprovechamiento del ancho de banda, el retardo y la mínima

carga computacional. Es más, esto nos permitirá tener un control completo sobre la serialización de los mensajes, pudiendo elegir la mejor opción para los requerimientos de la aplicación.

Por otro lado, al evitar el uso de herramientas externas, podemos asegurar que nuestra aplicación no se quedará obsoleta en ningún momento, teniendo la oportunidad de actualizar el diseño o ampliar funcionalidades cuando se desee.

4.1.3. Canal de comunicación

Una vez está clara la necesidad de diseñar un protocolo de aplicación, este debe presentar unas características que nos permita cumplir todos los requisitos indicados en la sección Análisis de los requisitos del protocolo.

Sin embargo, antes se decidirá cuál debe ser el canal de comunicación por el que viajarán los mensajes del protocolo, cuya elección será clave para definir requisitos como la bidireccionalidad de los datos, la latencia y la eficiencia.

Para ello, se han estudiado varios de los protocolos de comunicación más utilizados actualmente por la comunidad, con el fin de recopilar las características de todos ellos que estén relacionadas con los requisitos de comunicación que hemos impuesto en los objetivos del protocolo. Se puede observar en la Tabla 4.1 las principales características que se han tenido en cuenta para tomar la decisión final.

Tabla 4.1: Evaluación de protocolos de comunicación [67].

Protocolo	Latencia	Comunicación Full-Duplex	Robustez e Integridad	Soporte e Interoperabilidad
HTTP / REST	Alta	No	Alta	Alta
MQTT	Media	Sí	Alta	Alta
WebRTC	Baja	Sí	Alta	Media
WebTransport	Baja	Sí	Alta	Baja
SSE	Media	No	Alta	Alta
WebSockets	Baja	Sí	Alta	Alta

En definitiva, se ha elegido *WebSocket* como protocolo de comunicación entre el cliente y el servidor por su baja latencia y su capacidad para mantener una conexión bidireccional persistente. Esto es gracias a que, tras un intercambio inicial mediante *HTTP*, la conexión se actualiza para permitir el envío continuo de comandos sin necesidad de abrir nuevas conexiones ni añadir la sobrecarga de las peticiones *HTTP* tradicionales. Además, su amplia adopción hace que existan numerosas herramientas y librerías robustas, simplificando el desarrollo y reduciendo posibles problemas durante la

implementación. [68]

Aunque alternativas como *WebTransport* y *WebRTC* también permiten comunicaciones en tiempo real, presentan inconvenientes para este proyecto. *WebTransport* todavía no cuenta con el nivel de adopción y soporte de *WebSocket*, mientras que *WebRTC* añade una complejidad innecesaria al requerir infraestructura adicional, como servidores *STUN/TURN* y de señalización. [69, 70]

4.1.3.1. ¿WS o WSS?

Al establecer WebSockets como canal de comunicación, es necesario determinar si se utilizará la variante estándar en texto plano (`ws://`) o su variante segura mediante cifrado TLS (`wss://`). En este proyecto se ha optado por utilizar WebSockets sin cifrado, ya que aunque el uso de texto plano puede exponer la comunicación a riesgos como la interceptación de datos o ataques, el sistema está diseñado para funcionar únicamente en redes locales privadas y de confianza, como laboratorios, donde el acceso ya se encuentra protegido mediante mecanismos de seguridad de red como WPA2 o WPA3.

Además, el uso de `wss://` aumentaría la complejidad de implementación y dificultaría la adopción y modificación del código por parte de otros usuarios, debido a la necesidad de gestionar certificados TLS. Asimismo, la comunicación en texto plano facilita las tareas de desarrollo y depuración al permitir inspeccionar fácilmente los mensajes con herramientas como *Wireshark*. Finalmente, al evitar el cifrado y el intercambio inicial de claves, se reduce la sobrecarga y se minimiza la latencia.

4.1.4. Secuencia de Mensajes

Una vez seleccionado el protocolo de transporte que establecerá el canal de comunicación, el siguiente paso es diseñar el protocolo de aplicación. Este protocolo define qué información se intercambia entre el cliente y el servidor, así como el formato y las reglas que deben seguir los mensajes. Para empezar, en esta sección se especifican los distintos tipos de mensajes que forman parte del protocolo y se presenta un diagrama de secuencia que muestra el intercambio de los principales mensajes dentro de una arquitectura cliente-servidor.

Para identificar qué mensajes son necesarios, primero se ha analizado los mecanismos de comunicación de ROS 2 y las funcionalidades de la aplicación. Finalmente, el diseño se ha realizado de manera que las acciones de la aplicación se representan mediante mensajes que definen modos de comunicación específicos, mientras que la operación concreta a realizar se determina a través de parámetros incluidos en los propios mensajes, definidos en una

cabecera común.

A partir de esta abstracción, se definieron los siguientes mensajes de petición-respuesta, organizados en ocho bloques diferenciados recogidos en la tabla 4.2 (Los diagramas de secuencia visuales detallados para cada uno de estos bloques pueden consultarse en el Anexo C).

Finalmente, el protocolo define internamente dos eventos críticos que no son mensajes transmitidos por la red, sino disparadores de seguridad ejecutados localmente en ambos extremos: **SessionResetEvent** (reinicia los parámetros de la sesión actual al recibir una orden de desconexión del robot) y **ProtocolAbortEvent** (detiene cualquier ejecución y cierra el protocolo de forma segura si se detecta una pérdida de conexión física entre cliente y servidor).

4.1.5. Máquina de Estados

Una vez definidos los distintos tipos de mensajes que componen el protocolo, es necesario definir el comportamiento global del sistema. Para ello, se ha diseñado un diagrama de estados, tanto del cliente como del servidor, que representan los distintos estados en los que pueden encontrarse, así como las transiciones entre dichos estados que se producen en función de los mensajes intercambiados.

Este diagrama resulta fundamental en el diseño, ya que aporta al sistema un alto grado de seguridad, robustez y sincronización entre cliente y servidor. En concreto, define en qué estados pueden enviarse o recibirse los distintos mensajes, evitando posibles conflictos durante la ejecución. Además, garantiza que ambas partes compartan una misma visión del estado del sistema, realizando las transiciones de forma coordinada en función de los mensajes procesados.

El protocolo se basa en un modelo de Máquina de Estados Finitos Jerárquica (HFSM), en el que se cambia de estado según las acciones del cliente y las respuestas del servidor. A diferencia de las FSM tradicionales, este enfoque permite definir macroestados dentro de los estados secuenciales, que a su vez contienen subestados concurrentes. Estos subestados representan pequeñas máquinas de estados que se ejecutan en paralelo, lo que permite gestionar flujos de información independientes sin comprometer la estabilidad del sistema, resolviendo así uno de los requisitos clave del sistema, la concurrencia. [71, 72]

Dado que el protocolo sigue un modelo cliente-servidor, se definen dos diagramas de estados diferenciados, uno para cada entidad, aunque estrechamente relacionados entre sí para mantener la sincronización del sistema.

Antes de comenzar con la explicación de ambos diagramas, cabe re-

Tabla 4.2: Resumen de los mensajes del protocolo R2Pilot según su bloque funcional.

Bloque	Mensajes	Propósito
Envío de órdenes	CommandReq / Resp	Enviar órdenes puntuales al servidor, que devuelve éxito o error.
Consulta de información	QueryReq / Resp	Obtener datos estáticos en el cliente.
Acciones largas	ActionReq / Feedback	Gestión y monitorización del cliente de rutinas prolongadas (<i>actions</i> de ROS 2).
Flujo de órdenes	ControlModeReq / Resp ControlReq / Resp	Envío continuo de comandos de control desde el cliente.
Flujo de información	StreamReq / Resp StopStreamReq / Resp	Recepción continua de datos en el cliente (sensores).
Gestión interna	PingReq, AsyncNotify, ProtocolError	<i>Heartbeat</i> , alertas asíncronas y reporte de errores.

calcar que para una mayor claridad del diagrama, se han representado los mensajes de respuesta erróneos, es decir, aquellos con el parámetros `[success=false]`, como mensajes con la palabra “Error” en su nombre. Por ejemplo, el mensaje de respuesta `CommandResp[success=false]` está representado como el mensaje `CommandErrorResp`, y así con el resto de mensajes de respuesta.

4.1.5.1. Máquina de Estados del Cliente

El diagrama de estados del cliente modela el flujo de la aplicación móvil, desde la pantalla inicial hasta la conexión con el robot y el uso de las distintas funcionalidades. Su objetivo principal es habilitar o deshabilitar funcionalidades en la interfaz según el estado de la conexión y la respuesta del servidor.

El diagrama se puede encontrar en el anexo B en la figura B.1. En él, en la parte superior, coloreado en naranja, podemos observar los estados correspondientes a la gestión de la sesión. Se puede observar una ampliación de la parte superior del diagrama en el anexo B en la figura B.2. Dichos estados son:

- **Estado inicial:** es el punto negro de la parte superior izquierda. Representa el estado inicial del protocolo, donde aún no se ha establecido la conexión con el servidor.
- **Esperando conexión Backend:** estado intermedio antes de la conexión con el servidor, el cual existe cuando el cliente envía el primer mensaje `PingReq` para indicar al servidor que se encuentra disponible para la conexión.
- **Conexión Backend:** estado de conexión con el servidor, al cual se llega tras recibir el mensaje de recibo (ACK) del servidor. En este estado aún no existe una conexión del servidor con el robot, esta únicamente es entre cliente y servidor.
- **Esperando Inicio Sesión:** estado intermedio antes de iniciar sesión, que corresponde a establecer la conexión con el robot. Se entra en él tras enviar la petición al servidor de conectarse al robot.
- **Sesión iniciada:** estado en el que el cliente ya se encuentra conectado al robot tras la respuesta de confirmación del servidor. Este se trata de un macroestado en el que se pueden realizar todas las funcionalidades de la aplicación en relación al control y monitorización del robot.

- **Esperando Cierre Sesión:** estado intermedio antes de realizar el cierre de sesión, que se alcanza tras enviar el mensaje de petición al servidor. Este corresponde a la desconexión del robot, por lo que tras la confirmación del servidor, se vuelve al estado de **Conexión Backend**.
- **Esperando desconexión Backend:** estado intermedio antes de conseguir la desconexión con el servidor, y por tanto, alcanzar el estado final del protocolo.

Por otro lado, en la parte inferior, encontramos el macroestado de **Sesión Iniciada**, el cual incluye dos subestados concurrentes en su interior, que a su vez contienen dos pequeñas máquinas de estado finitos secuenciales, cuyos estados están coloreados de amarillo. Se puede encontrar una ampliación de la parte inferior del diagrama en el anexo B en la figura B.3. Estos subestados son:

- **Subestado Movimiento:** Incluye los mensajes y estados correspondientes al envío de órdenes al servidor para producir algún movimiento en el robot. Esta máquina tiene dos caminos, el de ejecución de acciones del robot o el de control remoto, por lo que estas dos funcionalidades no estarán disponibles simultáneamente.
- **Subestado Monitorización:** Incluye los estados y mensajes correspondientes a las funcionalidades de monitorización del robot, que en este caso es la visualización de los distintos sensores.

En conclusión, estos dos subestados concurrentes permiten que, cuando el protocolo se encuentre en el estado **Sesión iniciada**, este sea capaz de gestionar simultáneamente el envío de información de monitorización con la recepción de datos de control.

Además, también se puede observar el estado **Esperando Recibir Información Única** coloreado de azul, el cual se alcanza tras preguntar al servidor sobre cierta información, y que tras la respuesta vuelve al macroestado. Esta funcionalidad no pertenece a los subestados concurrentes, por lo que su ejecución será secuencial.

Por último, cabe destacar que, aunque no esté representado en el diagrama de esta forma por simplicidad, los mensajes **PingReq** y **AsyncNotify** se pueden enviar desde cualquier estado del protocolo. De la misma forma, los eventos **SessionResetEvent** y **ProtocolAbortEvent** podrían producirse en cualquiera de los subestados concurrentes del macroestado **Sesión iniciada**, alcanzando el final de los mismos.

4.1.5.2. Máquina de Estados del Servidor

El diagrama de estados del servidor define el flujo de la lógica de control en el servidor, desde la apertura del canal de comunicación hasta el intercambio de información con el robot. Su objetivo principal es validar la ejecución de comandos, asegurando que el robot solo responda a peticiones lícitas en el momento de la sesión adecuado para ello.

Se puede encontrar el diagrama en el anexo B en la figura B.4

En este caso, el diagrama de estados es bastante similar al del cliente, pero esta vez mostrando la recepción de las peticiones del cliente y las respuestas ofrecidas según los eventos producidos a la hora de la ejecución de las mismas. Son estas respuestas las que producen la transición de estados. Este diagrama se puede consultar en el Anexo B en la Figura B.4.

La forma de seguir el diagrama es exactamente la misma que en el caso anterior, encontrando en la parte superior el estado de **Conexion Backend** y **Sesión Iniciada**, junto a los mensajes que gestionan la sesión, y dentro del macroestado, los dos mismos subestados concurrentes explicados.

4.1.5.3. Validación del protocolo: Alcanzabilidad

La validación del protocolo garantiza que el comportamiento del sistema sea correcto, estable y consistente bajo cualquier circunstancia. Para ello, se ha realizado un análisis de alcanzabilidad, explorando todos los posibles comportamientos e interacciones entre cliente y servidor.

Dado que el protocolo implementa concurrencia, se han generado tres grafos de alcanzabilidad independientes: el flujo principal, el subestado de movimiento y el subestado de monitorización. Las leyendas detalladas y los grafos resultantes se encuentran documentados en el anexo D.

Tras el análisis de estos grafos, se ha verificado el cumplimiento de las siguientes propiedades fundamentales para la robustez del sistema:

- **Ausencia de puntos muertos (deadlocks):** todos los caminos posibles conducen a los estados finales esperados, evitando situaciones donde ambos sistemas queden bloqueados esperando una respuesta.
- **Ausencia de ciclos no productivos:** los únicos ciclos detectados corresponden a comportamientos esperados (como el flujo continuo de telemetría) y no impiden el avance ni la finalización del protocolo.
- **Cobertura total (acciones alcanzables):** todos los estados y mensajes diseñados son alcanzables y utilizados al menos una vez, confirmando que no hay transiciones inaccesibles (estados huérfanos).

- **Terminación adecuada:** ante eventos de desconexión o fin de sesión, el protocolo siempre cuenta con una ruta segura hacia un estado final estable.

En conclusión, la verificación formal confirma que el protocolo R2Pilot es sólido, garantizando una comunicación fiable y tolerante a fallos para la interacción con el robot.

4.1.6. Formato y estructura de los mensajes

Para terminar con el diseño del protocolo, es necesario estandarizar el formato que tendrán los mensajes que viajarán por el canal. Por tanto, esta sección tratará de resolver dos problemas fundamentales: la forma y el contenido. Por un lado, se debe seleccionar un mecanismo de serialización, cuya función es empaquetar los objetos y variables en una secuencia de datos para transmitirse a través de una red. Por otro lado, se requiere establecer el contenido de dichos paquetes, es decir, un esquema de datos que refleje qué campos son obligatorios, qué estructura jerárquica tienen y qué tipos de valores se esperan. [73]

4.1.6.1. Formato de los mensajes

La elección del formato de serialización es fundamental para garantizar que se cumplen los requisitos de retardo, flexibilidad, mantenibilidad y facilidad de modificación impuestos en los objetivos del proyecto.

Para ello, se han evaluado varios de los formatos de serialización de datos más utilizados, recopilando aquellas características que impactan directamente con los objetivos del proyecto. Estas se pueden observar en la Tabla 4.3.

Tabla 4.3: Evaluación de formatos de serialización de datos. [74, 75]

Formato	Legibilidad Humana	Flexibilidad del Esquema	Soporte (Python / Kotlin)	Eficiencia de Red
XML	Alta	Alta	Alta	Baja
Protocol Buffers	Nula	Media	Alta	Muy alta
FlatBuffers	Nula	Media	Alta	Muy alta
JSON	Alta	Alta	Muy alta	Media

En definitiva, se ha seleccionado *JSON* (*JavaScript Object Notation*) como formato de serialización para los mensajes del protocolo. Aunque inicialmente las alternativas binarias, como *Protocol Buffers* o *FlatBuffers*, podrían parecer la opción idónea para minimizar la latencia en comunicaciones en

tiempo real, como es el caso, su uso no se ha considerado es el más indicado para este escenario. Dado que la transmisión de vídeo (el flujo de datos más pesado) se ha delegado a un canal independiente vía HTTP, el volumen de datos que asume el protocolo es muy reducido. Por tanto, el impacto del tamaño del paquete y del tiempo de serialización de *JSON* no resulta significativo operando sobre una red Wi-Fi local estándar.

Además, el uso de serializadores binarios presentan inconvenientes que chocan directamente con uno de los requisitos fundamentales del proyecto: la extensibilidad. El sistema debe permitir que cualquier usuario pueda modificar la aplicación para adecuarla a su propio robot sin necesidad de enfrentarse a un rediseño complejo. Estos inconvenientes notables son tales como la nula legibilidad humana de los paquetes, una mayor complejidad de implementación y una menor flexibilidad del esquema ante cambios.

Por el contrario, los formatos basados en texto plano facilitan enormemente las fases de desarrollo y validación, ya que permiten inspeccionar y depurar el tráfico de red en tiempo real de forma directa. Dentro de estas opciones, se ha priorizado *JSON* frente a *XML* debido a que es un formato más ligero, lo que se traduce en un menor tamaño de paquete y una serialización más rápida. Asimismo, *JSON* es una alternativa más moderna y se ha consolidado como el estándar en la industria actual, garantizando una gran integración en la inmensa mayoría de sistemas y plataformas actuales.

4.1.6.2. Estructura de los mensajes

Ya que se ha elegido JSON como el formato en el que viajará la información de los mensajes del protocolo, solo queda definir como se estructura dicha información. Esta debe ser clara y escalable, y para ello que se ha definido una estructura formada por una cabecera (*header*), que incluya la información básica para identificar las propiedades del mensaje, y un cuerpo (*payload*), que contiene la información enviada en el mensaje.

Estructura de la cabecera La cabecera contiene los metadatos esenciales para el procesamiento del mensaje. Su estructura es estática, lo que significa que estos campos deben estar presentes obligatoriamente en todos los mensajes que se envíen del protocolo. Los campos diseñados para esta sección se detallan en la Tabla 4.4.

Estructura del cuerpo A diferencia de la cabecera, el cuerpo del mensaje (*payload*) es completamente dinámico. Su contenido y estructura dependen directamente del valor especificado en el campo **type** de la cabecera.

Para garantizar la robustez, el protocolo establece un esquema estricto

Tabla 4.4: Definición de los campos de la cabecera del protocolo.

Campo	Tipo de Dato	Descripción y Justificación
<code>msg_id</code>	String	Identificador único de secuencia. Su función principal es correlacionar peticiones con sus respuestas y permitir la detección de paquetes asíncronos o perdidos.
<code>type</code>	String (Enum)	Define el tipo de mensaje y determina cómo debe procesarse e interpretarse el <i>Payload</i> . Los tipos de mensaje aparecen en la sección 4.1.4
<code>session_id</code>	String (UUIDv4)	Token de identificación que verifica que el mensaje pertenece a la sesión lógica actual del usuario. Funciona como mecanismo de seguridad.
<code>timestamp</code>	Número (Double)	Marca temporal de emisión del mensaje. Resulta fundamental para el cálculo de latencias en la red y el descarte de mensajes desfasados.

de campos obligatorios para cada tipo de mensaje, permitiendo así una validación rápida y estandarizada. En la tabla 4.5 se resume la estructura del *payload* exigida para cada mensaje, la cual se puede ver de forma clara y extensa en el *JSON Schema* alojado en el servidor.

Cabe destacar que, para conseguir un protocolo robusto, se ha contemplado el diseño de respuestas ante comportamientos no esperados. Toda petición (salvo las asíncronas) recibe una respuesta cuyo *payload* base siempre incluye: `success` (booleano), `resp_type`, y un `code` numérico similar a los estados HTTP (200 OK, 400 Bad Request, 403 Forbidden, 404 Not Found, 500 Internal Error). Si la petición falla, se exige además un campo `details` con la descripción del problema.

4.2. Desarrollo del protocolo

Esta sección del capítulo describe cómo se ha llevado a cabo la implementación del protocolo diseñado, tanto en el servidor (*backend*) como en el cliente móvil (*frontend*). El objetivo es que ambos extremos sean capaces de enviar y recibir los mensajes definidos, interpretar correctamente la información que contienen y mantenerse sincronizados mediante el seguimiento de la máquina de estados finitos.

Durante la implementación se ha optado por una organización de los archivos que establece una clara separación de responsabilidades. Esta estructura facilita la comprensión del código y mejora su mantenibilidad. Además, permite que, si en el futuro se desea sustituir tecnologías como WebSocket o JSON por otras alternativas, el cambio pueda realizarse únicamente en los

Tabla 4.5: Estructura de datos del Payload según el tipo de mensaje.

Tipo de Mensaje	Campos requeridos ([Opcionales])	Descripción del contenido
COMMAND_REQ	<code>action</code> , [<code>param1</code> , <code>param2...</code>]	Orden a ejecutar y la información necesaria.
PING_REQ, ACK	(<i>Payload vacío</i>)	Minimiza el tamaño de la trama en la red.
CONTROL_MODE_REQ	<code>event</code> , <code>type</code> , <code>topic</code>	Inicio/fin de un modo de control específico y el tópico donde publicar.
CONTROL_REQ	<code>data</code>	Vector de velocidades o posiciones articulares.
ACTION_REQ / STOP...	<code>type</code> , <code>target</code>	Acción prolongada a iniciar/detener.
QUERY_REQ	<code>resource_type</code>	Información estática a consultar.
STREAM_REQ / STOP....	<code>recurso</code> , <code>topic</code> , [<code>quality_level</code>]	Configuración de la suscripción a sensores o cámara.
ASYNC_NOTIFY	<code>type</code> , <code>details</code> , <code>severity</code>	Tipo de evento, gravedad y detalles.
PROTOCOL_ERROR	<code>error_code</code> , <code>description</code>	Tipo de error y descripción.
<i>Cualquier</i> *RESP	<code>success</code> , <code>code</code> , <code>resp_type</code>	Éxito, código y tipo de respuesta.
↪ Fallos	<code>details</code>	Detalles (se adjunta si <code>success=false</code>).
↪ Campos adicionales requeridos en función del valor de <i>resp_type</i> :		
QUERY_RESP	<code>data</code>	Información solicitada.
ACTION_FEEDBACK	<code>progress</code> , <code>done_exec</code> , <code>status</code>	Progreso, estado y finalización de una acción en curso.
STREAM_RESP	<code>stream_data</code> <code>stream_url</code>	Datos de los sensores o URL de vídeo (HTTP).

componentes encargados de gestionar el canal de comunicación o la serialización de los datos. Del mismo modo, cualquier modificación en la máquina de estados o en la estructura de los mensajes quedaría aislada en los archivos correspondientes, minimizando el impacto sobre el resto del sistema.

4.2.1. Despliegue y uso del canal WebSocket

Gracias a la amplia estandarización del protocolo WebSocket, el despliegue del canal de comunicación se ha llevado a cabo de forma sencilla y fiable mediante el uso de librerías consolidadas para Python y Kotlin. A continuación, se describe la implementación de este canal en ambos nodos de la arquitectura y se justifica la elección de las librerías empleadas.

4.2.1.1. Servidor

En el servidor, desarrollado en Python, todo el código relacionado con el establecimiento y la gestión del canal de comunicación se ha centralizado en el archivo `websocket_server.py`, mediante la clase `WebSocketServer`. De este modo, el resto de módulos del servidor utilizan las funciones definidas en este archivo para comunicarse por la red, sin depender directamente del canal utilizado. Esta separación de responsabilidades mejora la organización del código y facilita posibles cambios futuros.

Durante la fase de diseño, se evaluaron alternativas del ecosistema Python para implementar un servidor *WebSocket*, como *Tornado* [76], *FastAPI* [77] y *Socket.IO* [78]. Sin embargo, *FastAPI* y *Tornado* están principalmente orientados al desarrollo de aplicaciones web, mientras que *Socket.IO* utiliza un protocolo propio para la comunicación en tiempo real, diferente del estándar *WebSocket*. Por ello, se seleccionó la librería `websockets`, una solución ligera y especializada en este protocolo, fiel al estándar RFC 6455 y con integración nativa con `asyncio`, el marco estándar de Python para operaciones asíncronas y concurrentes. Además, la librería abstrae la complejidad de bajo nivel del protocolo, gestionando aspectos como el establecimiento y cierre de las conexiones [79, 80].

A nivel arquitectónico, la clase `WebSocketServer` asume tres responsabilidades:

- **Inicialización y despliegue:** mediante inyección de dependencias, se vincula al resto de módulos del servidor y arranca el servicio en la IP y puerto configurados.
- **Recepción continua:** implementa un bucle infinito asíncrono (*handler*) que procesa los paquetes entrantes por cada cliente conectado.

- **Abstracción del envío:** proporciona *callbacks* inyectables para que otras capas puedan emitir mensajes o cerrar el canal de forma segura.

4.2.1.2. Cliente

En el cliente, desarrollado en Kotlin, los archivos de código se han organizado de la misma forma para preservar las mismas ventajas de separación de responsabilidades que se ha comentado en el servidor. En este caso, el establecimiento y la gestión del canal de comunicación se ha centralizado en el archivo `WebSocketClient.kt`, a través de la clase `WebSocketClient`.

Para la selección de la librería de red se evaluó inicialmente *OkHttp*, muy popular en Android. No obstante, su gestión de *WebSockets* se basa en un modelo clásico de *callbacks* (`WebSocketListener`), lo que obliga a desarrollar código intermedio para adaptarlo a las arquitecturas modernas [81]. En su lugar, se optó por *Ktor*, un *framework* asíncrono desarrollado por JetBrains que ofrece las siguientes ventajas: [82, 83]:

- **Soporte nativo para corrutinas:** permite realizar operaciones de red asíncronas, utilizando el motor CIO propio de Ktor, sin bloquear los hilos de la interfaz de usuario.
- **Gestión de flujos reactivos:** el canal expone los mensajes entrantes directamente a la lógica de la aplicación, integrándose perfectamente con herramientas como `Flow` y permitiendo que la interfaz responda a los eventos en tiempo real de manera sencilla.
- **Control automático de la sesión:** permite configurar parámetros de alto nivel, como `pingInterval`, automatizando el envío de tramas de mantenimiento.

A nivel interno, la clase `WebSocketClient` encapsula el ciclo de vida de la conexión (`connect`, `send`, `disconnect`). Para su implementación, se emplea el motor de red asíncrono CIO (*Coroutine-based I/O*) integrado en Ktor, elegido por estar diseñado íntegramente para aprovechar las corrutinas de Kotlin y soportar *WebSockets* de forma nativa [84]. Además de exponer la función de envío de mensajes para que las capas lógicas superiores puedan transmitir peticiones hacia la red, la clase expone la información entrante utilizando flujos reactivos propios de Kotlin, un `SharedFlow` que actúa como búfer para canalizar los mensajes JSON recibidos y un `StateFlow` que funciona como *watchdog* interno, notificando instantáneamente a la interfaz cualquier cambio en el estado físico de la conexión.

4.2.2. Procesamiento y validación de los mensajes

El sistema incluye una capa encargada de la serialización a formato JSON, deserialización y validación de la integridad de los mensajes. Siguiendo el mismo principio de diseño tanto en el cliente como en el servidor, estas tareas se han centralizado en módulos específicos, que se resumen en la Tabla 4.6.

Tabla 4.6: Módulos de software implementados para el procesamiento del protocolo.

Responsabilidad	Backend (Python)	Frontend (Kotlin)
Modelado de Datos	<code>models.py</code>	<code>Models.kt</code>
<i>Implementación</i>	<code>@dataclass</code> y <code>Union</code>	<code>data class</code> y <code>sealed interface</code>
Validación de Esquema	<code>validator.py</code>	<i>Validación implícita</i>
<i>Herramienta/Técnica</i>	JSON Schema	Sistema de tipos estático
Codificación (JSON)	<code>message_codec.py</code>	<code>MessageCodec.kt</code>
<i>Librería utilizada</i>	<code>json</code> (nativa)	<code>kotlinx.serialization</code>

4.2.2.1. Modelado de datos y validación

El primer paso para el procesamiento es representar los mensajes JSON como objetos manejables en memoria, de modo que el código sea más claro, fácil de mantener y menos propenso a errores. En el servidor (Python), esto se realiza en el archivo `models.py` mediante `dataclasses`, agrupando todas las estructuras en la clase principal `RobotMessage`. Dado que Python es de tipado dinámico, el `payload` se define mediante la directiva `Union` para albergar cualquier cuerpo de mensaje. Para asegurar la robustez, se implementó el archivo `validator.py` utilizando JSON Schema [85]. Esta herramienta actúa como un filtro estricto previo, si un mensaje entrante carece de campos obligatorios o contiene tipos de datos incorrectos, es bloqueado.

En el cliente (Kotlin), la estructura se replica en `Models.kt` utilizando `data classes`. A diferencia de Python, los payloads dinámicos se gestionan mediante una `sealed interface` [86] denominada `Payload`, lo que permite al compilador conocer todos los tipos posibles y asegurar su correcta manipulación. Gracias al fuerte tipado estático de Kotlin, no fue necesario implementar un motor de validación externo como JSON Schema, ya que si el servidor envía un mensaje malformado, la propia deserialización falla de forma segura mediante una excepción controlada.

4.2.2.2. Serialización y deserialización

La serialización y deserialización de los datos se gestiona a través de las clases `MessageCodec`, en cliente y servidor.

Para el servidor Python, se evaluaron librerías de alto rendimiento como `orjson` [87] o `ujson` [88]. Sin embargo, se priorizó el uso de la librería nativa `json` [89] de la biblioteca estándar de Python. Esta decisión se basa en que se priorizó la portabilidad y la facilidad de despliegue en distintos robots, especialmente en aquellos con recursos más limitados, como los que utilizan una *Raspberry Pi*. Al formar parte de la biblioteca estándar de Python, `json` no requiere dependencias adicionales, y ofrece un rendimiento suficiente para gestionar los datos de la aplicación.

En el frontend Android, se descartaron librerías heredadas del ecosistema Java, como `Gson` [90] o `Jackson` [91], en favor de `kotlinx.serialization` [92]. Esta librería oficial de JetBrains utiliza un plugin que genera el código de conversión durante la fase de compilación, consiguiendo minimizar el retardo de la comunicación y aumentar la eficiencia de la aplicación [93].

4.2.3. Implementación de la Máquina de Estados

Una vez establecido el canal de comunicación y la codificación de los mensajes, el último elemento del protocolo consiste en implementar la máquina de estados. Este componente actúa como un filtro estricto sobre los mensajes entrantes y salientes, garantizando que ambas partes mantengan una visión sincronizada del sistema en todo momento.

4.2.3.1. Servidor

En el servidor, la lógica se implementa en el archivo `state_machine.py` a través de la clase `ProtocolStateMachine`. Para posibilitar la concurrencia definida en el diseño, la clase almacena simultáneamente el estado de las tres máquinas de estados jerárquicas: el estado global (`global_state`), el subestado de movimiento (`movement_state`) y el subestado de monitorización (`monitor_state`).

La evaluación de las transiciones se divide en dos fases principales:

- **Validación previa (`can_transition`):** actúa como barrera de entrada. Comprueba si la petición del cliente se permite en función del estado actual del servidor. Si el mensaje llega en un momento no permitido, la función lo bloquea y devuelve un código de error al instante, evitando que la orden alcance al robot.

- **Consolidación** (`commit_transition`): se invoca únicamente cuando el servidor ya ha procesado la petición y está a punto de enviar la respuesta. Evalúa si la operación fue exitosa o fallida y actualiza las variables de estado.

4.2.3.2. Cliente

En la aplicación móvil, el diagrama de estados se implementa en el archivo `StateMachine.kt` mediante la clase `ProtocolStateManager`.

La principal diferencia es cómo se almacena la información, ya que en lugar de variables estándar, el cliente utiliza flujos de datos observables (`StateFlow`) de Kotlin. Este enfoque reactivo permite que las distintas pantallas de la capa de interfaz gráfica (*View*) se suscriban a estos flujos y se reconfiguren de forma automática según el estado del sistema, sin necesidad de consultarlo continuamente. Además, se expone un flujo adicional (`systemAlert`) encargado de propagar errores y advertencias directamente a la interfaz.

Dado que el cliente es el emisor de las peticiones, la gestión de sus estados es ligeramente más compleja e involucra tres etapas:

- **Filtro de salida** (`canSendMessage`): evalúa si el usuario tiene permitido enviar un comando específico en el estado actual. Esto previene fallos lógicos y evita el envío repetido de peticiones si el usuario pulsa varias veces un botón.
- **Transición de espera** (`commitRequestSent`): se ejecuta en el instante en que el mensaje sale hacia la red. Mueve la máquina a un estado transitorio (por ejemplo, "Esperando inicio de sesión") que permite a la interfaz reaccionar mostrando un indicador de carga.
- **Resolución** (`commitResponseReceived`): analiza la respuesta del servidor. Si la petición tuvo éxito, el sistema avanza al nuevo estado, y si fue rechazada o falló, retrocede de forma segura al estado anterior y emite una notificación al flujo de alertas para informar al usuario.

4.2.4. Implementación del *Heartbeat* y eventos

Para garantizar la estabilidad del sistema y proteger la integridad del robot físico ante cortes en la red inalámbrica, el protocolo cuenta con una gestión de eventos y un mecanismo de supervisión continua de la conexión (*Heartbeat*).

Gestión de Eventos Internos El sistema reacciona ante situaciones de desconexión mediante dos posibles eventos:

- **SessionResetEvent:** se dispara, en cliente y servidor, al finalizar la sesión lógica con el robot. Ejecuta funciones (`trigger_session_reset`) que devuelven la máquina de estados principal al estado "Conexión Backend", reinician las máquinas concurrentes (Movimiento y Monitorización) y eliminan cualquier rastro de la sesión anterior.
- **ProtocolAbortEvent:** se activa ante una pérdida de la conexión física entre el cliente móvil y el servidor. Provoca un reinicio completo del protocolo (mediante `client_disconnected` en el servidor y `triggerFullReset` en el cliente), forzando al sistema a regresar a su estado inicial de forma segura.

Mecanismo de *Heartbeat* Dado que el protocolo WebSocket puede tardar en notificar las pérdidas de conexión por caídas de cobertura Wi-Fi, se ha implementado un mecanismo de detección activa que cumple una doble función: mantener la interfaz del cliente actualizada y, sobre todo, evitar que el robot continúe ejecutando órdenes si se pierde el control.

- **Emisión (Cliente):** Una vez establecida la conexión, la clase `SessionManager` despliega una corrutina (`StartHeartbeat`) encargada de emitir un mensaje `PingReq` a una frecuencia de 1 Hz.
- **Supervisión (Servidor):** La clase `ConnectionManager` implementa un temporizador de vigilancia (`watchdog_loop`) que verifica continuamente el tiempo transcurrido desde el último `PingReq`. Si este intervalo supera el umbral límite de 3 segundos, el servidor asume que el cliente se ha desconectado. Automáticamente, bloquea los comandos de movimiento, dispara el evento `ProtocolAbortEvent` y cierra el canal físico.

Capítulo 5

Diseño y desarrollo de la aplicación

Este capítulo aborda el objetivo principal del proyecto, el diseño y desarrollo de una aplicación móvil para el control y la monitorización de robots que operan con ROS2. El capítulo se divide en diferentes secciones, desde la identificación y análisis del problema hasta el diseño e implementación de la solución, mostrando todo el desarrollo seguido para la creación de la aplicación R2Pilot.

A lo largo del capítulo se detallan y justifican los criterios seguidos y las principales decisiones tomadas durante el diseño y desarrollo de la aplicación, así como las herramientas y tecnologías empleadas. El objetivo es proporcionar una visión clara de la estructura y el funcionamiento de la aplicación, de manera que este contenido facilite al usuario su posterior modificación y adaptación a las características específicas de su robot. Aunque esta adaptación no es necesaria para utilizar la aplicación, puede resultar útil si se quieren aprovechar al máximo las funcionalidades particulares que ofrece cada robot.

5.1. Descripción del problema y objetivos

Como se ha comentado a lo largo de esta memoria, el proyecto surge de la necesidad de desarrollar una aplicación móvil para la supervisión y el control de robots basados en ROS 2. El principal desafío consiste en trasladar la complejidad de este ecosistema, descrita en el capítulo 2, a una herramienta gráfica sencilla, accesible y capaz de funcionar en tiempo real, teniendo además en cuenta las limitaciones propias de los dispositivos móviles, como el reducido espacio de pantalla y unos recursos más limitados que los de un ordenador.

A esta dificultad se suma la diversidad de robots que pueden utilizar ROS 2, ya que cada uno puede disponer de hardware, sensores y sistemas de control diferentes. Por ello, una aplicación diseñada específicamente para un único robot tendría una utilidad limitada. En este proyecto se ha tomado como referencia el robot TIAGo de PAL Robotics, disponible para las pruebas y suficientemente completo para validar las funcionalidades desarrolladas, pero se ha planteado la arquitectura de forma que pueda adaptarse y extenderse a otros robots sin requerir modificaciones significativas en el conjunto del sistema.

5.1.1. Objetivos

El objetivo principal de la aplicación es proporcionar a investigadores, personal técnico y cualquier usuario interesado en el control y la supervisión de robots basados en ROS 2 una herramienta que facilite su interacción con estos desde un dispositivo móvil, ofreciendo un acceso rápido, sencillo y cómodo. Para ello, se establecen los siguientes objetivos:

- OBJ-1. Abstracción y accesibilidad:** ofrecer una interfaz gráfica móvil sencilla e intuitiva que elimine la dependencia de terminales o *software* de escritorio, facilitando el uso a perfiles menos técnicos.
- OBJ-2. Centralización del control:** reunir en una única aplicación las principales formas de interacción con el robot, incluyendo tanto el control de sus motores como la visualización de sus sensores.
- OBJ-3. Monitorización en tiempo real:** proporcionar al usuario una visión clara del estado del robot mediante la muestra de datos en tiempo real, facilitando así su supervisión y control a distancia.
- OBJ-4. Seguridad y fiabilidad:** implementar mecanismos que supervisen continuamente el estado de la red y detengan al robot automáticamente ante pérdidas de conexión.
- OBJ-5. Adaptabilidad:** diseñar una aplicación que, aunque se valide principalmente sobre TIAGo, pueda adaptarse y extenderse a otros robots con características diferentes, facilitando su reutilización en diferentes entornos.

5.2. Análisis de requisitos

Para definir las funcionalidades y características de la aplicación, es necesario realizar previamente un análisis de requisitos [94]. Esta fase permite

identificar las necesidades del sistema y asegurar que la solución desarrollada cumpla con los objetivos planteados.

Los requisitos se dividen en Funcionales (RF), que especifican las acciones que puede realizar el usuario, y No Funcionales (RNF), que establecen las características y condiciones que debe cumplir la aplicación durante su funcionamiento. En las tablas 5.1 y 5.2 se recogen todos los requisitos identificados, agrupados por su ámbito de aplicación.

Tabla 5.1: Requisitos Funcionales (RF) de la aplicación.

ID	Requisito	Descripción
Gestión de la Conexión y Sesión		
RF-1	Configurar IP y puerto del servidor	Introducir y validar IP y puerto del servidor en el cliente.
RF-2	Establecer conexión <i>WebSocket</i>	Establecer y verificar la conexión con el servidor.
RF-3	Configurar DDS	Seleccionar el <i>Domain ID</i> y tipo de middleware.
RF-4	Iniciar sesión	Comprobar disponibilidad del robot e iniciar sesión de control.
RF-5	Cerrar sesión	Detener robot, eliminar datos y cerrar sesión.
RF-6	Cerrar conexión <i>WebSoc- ket</i>	Cerrar el canal WebSocket y eliminar rastro de conexión.
RF-7	Apagar robot	Apagado remoto del SO del robot (doble confirmación).
RF-8	Reiniciar robot	Reinicio remoto del SO del robot (doble confirmación).
Monitorización del Robot		
RF-9	Consultar información <i>hardware/software</i>	Consultar datos del robot como capacidades o estado físico.
RF-10	Consultar acciones pregrabadas	Consultar las rutinas disponibles.
RF-11	Consultar progreso de acción	Visualizar el estado de una acción en curso.
RF-12	Configurar y visualizar cámara	Seleccionar tópico, calidad y visualizar vídeo en directo.
RF-13	Detener cámara	Cancelar suscripción y transmisión de vídeo.
RF-14	Consultar elementos de comunicación ROS 2	Consultar tópicos, servicios y acciones de la red ROS 2.

Continúa en la página siguiente

Tabla 5.1 – Continuación de la página anterior

ID	Requisito	Descripción
RF-15	Visualizar datos de sensores	Monitorización gráfica en tiempo real de los sensores.
RF-16	Detener visualización de sensores	Cancelar suscripción y envío de los sensores activos.
Control del Robot		
RF-17	Teleoperar base	Control de la base del robot mediante un <i>joystick</i> virtual.
RF-18	Ejecutar acciones pregrabadas	Solicitar la ejecución de rutinas predefinidas.
RF-19	Detener acciones pregrabadas	Cancelar la ejecución de una rutina en curso.
RF-20	Controlar articulaciones del robot	Envío de posición objetivo mediante barras deslizantes.
RF-21	Control y monitorización concurrente	Permitir control y monitorización de forma simultánea.
Seguridad y Alertas		
RF-22	Comprobar conexión al servidor	<i>Ping</i> continuo; aviso y cierre de sesión si falla.
RF-23	Comprobar conexión al robot	Comprobación continua de la presencia en la red ROS 2.
RF-24	Restricción de funcionalidades	Ocultar menus de control según el <i>hardware</i> no detectado en el robot.
RF-25	Control de latencia	Detener teleoperación si la latencia de red es excesiva.
RF-26	Límite de velocidad	Cancelar comando si el movimiento de las articulaciones es muy brusco.
RF-27	Alertas críticas	Notificar problemas de conexión o protocolo en la interfaz.

Tabla 5.2: Requisitos No Funcionales (RNF) de la aplicación.

ID	Requisito	Descripción
Entorno y Compatibilidad		
RNF-1	Plataforma móvil	Desarrollo en Kotlin para dispositivos Android 8.0+.

Continúa en la página siguiente

Tabla 5.2 – *Continuación de la página anterior*

ID	Requisito	Descripción
RNF-2	Plataforma servidor	Desarrollo en Python (con <code>rclpy</code>) sobre Ubuntu/ROS 2.
RNF-3	Protocolo de red	Comunicación bidireccional basada en WebSocket.
RNF-4	Protocolo R2Pilot	Cumplimiento estricto del formato y estado de mensajes.
Rendimiento y Eficiencia		
RNF-5	Tiempo real	Operación fluida sin retardos perceptibles para el usuario.
RNF-6	Baja latencia	Respuesta inmediata en teleoperación para permitir control.
RNF-7	Optimización recursos	Procesado y submuestreo en servidor para ahorrar recursos en móvil.
Usabilidad		
RNF-8	Interfaz intuitiva	Diseño claro y fácilmente interpretable.
RNF-9	Distribución visual	Organización sin saturación para control/monitorización a la vez.
RNF-10	Joystick táctil	Área de control adecuada para evitar errores.
RNF-11	Controles suaves	Modificación de articulaciones precisa y sin saltos.
RNF-12	Claridad visual	Representación visual clara y tipografía legible de los sensores.
Seguridad e Integridad		
RNF-13	Integridad física	Respetar límites del hardware y frenar ante inestabilidad.
RNF-14	Exclusividad de control	Garantizar un único operador conectado simultáneamente.
Arquitectura y Escalabilidad		
RNF-15	Diseño modular	Código desacoplado para facilitar la ampliación o modificación.
RNF-16	Interfaz dinámica	Interfaz dinámica adaptable a las capacidades descubiertas.

5.3. Entorno de desarrollo y herramientas

En esta sección se presentan los entornos de desarrollo y las principales herramientas empleadas durante la realización de la aplicación.

Para la organización y mantenimiento del código se ha utilizado GitHub, donde se han creado repositorios independientes para el *backend* y el *frontend*. El código ha sido documentado con Doxygen para facilitar su mantenimiento y futuras ampliaciones, y la documentación está disponible públicamente en los repositorios mediante *GitHub Pages*. Esta permite consultar la arquitectura, la jerarquía de clases y las especificaciones de los distintos módulos y funciones del sistema [95].

5.3.1. Entorno y herramientas del *Backend*

El desarrollo del servidor se ha realizado sobre **Ubuntu 22.04 LTS** [96], utilizando un contenedor de **Docker** proporcionado oficialmente por PAL Robotics para el robot TIAGo. Este contenedor, basado también en Ubuntu 22.04, incluye ROS 2 Humble Hawksbill y los paquetes necesarios para trabajar con el robot y su simulación. La elección de Ubuntu 22.04 se debe a su compatibilidad oficial con ROS 2 Humble, donde está catalogado como sistema operativo de nivel *Tier 1* [97, 98]. Además, su condición de versión LTS proporciona un entorno estable durante el desarrollo.

Inicialmente se utilizó WSL2 [99] sobre Windows 11, pero los problemas relacionados con la configuración de red y el acceso a la GPU desde Docker dificultaron la ejecución del simulador. Por este motivo, finalmente se optó por una instalación nativa de Ubuntu. El uso de Docker permite, además, disponer de un entorno previamente configurado con las dependencias necesarias, facilitando la ejecución del simulador y la reproducción del entorno del robot en diferentes equipos [100].

El servidor se ha desarrollado en **Python** [101], utilizando **Visual Studio Code** [102] como entorno de desarrollo. Python permite trabajar con `rc1py`, la biblioteca cliente de ROS 2, y ofrece un desarrollo sencillo y flexible para la implementación de las funcionalidades del servidor, como la gestión de conexiones *WebSocket* y el procesamiento de mensajes JSON [19, 103]. Por su parte, VS Code proporciona integración con Docker y ROS 2 mediante sus extensiones correspondientes [104, 105].

5.3.2. Entorno y herramientas del *Frontend*

El cliente móvil, dirigido a la plataforma Android por los motivos recogidos en la sección 2.3, se ha desarrollado utilizando el mismo sistema operativo Linux empleado para el servidor, pero sin necesidad de utilizar

Docker ni instalar ROS 2. Esto permite trabajar simultáneamente en el desarrollo del servidor y del cliente.

La aplicación se ha implementado utilizando **Kotlin** [106], **Jetpack Compose** [107] y **Android Studio** [49]. Kotlin es el lenguaje recomendado oficialmente por Google para el desarrollo de aplicaciones Android y ofrece un código conciso y seguro, además de proporcionar soporte para corrutinas, fundamentales para gestionar las operaciones asíncronas de la aplicación [108, 109, 110].

Por otro lado, Jetpack Compose es el conjunto de herramientas recomendado por Android para la creación de interfaces, mientras que Android Studio es el entorno de desarrollo oficial para la creación de aplicaciones Android, que proporciona las herramientas necesarias para compilar, ejecutar y probar la aplicación.

5.3.3. Entorno de pruebas

Las pruebas de la aplicación se han realizado mediante dos entornos complementarios: la simulación del robot en **Gazebo** [111] y el robot físico TIAGo de PAL Robotics [112]. Durante las primeras fases se ha utilizado Gazebo, integrado en el contenedor oficial de TIAGo, ya que permite simular su comportamiento, sensores y comunicaciones ROS 2 en un entorno seguro, evitando riesgos para el *hardware*. Esta integración también evita configuraciones adicionales y proporciona una simulación próxima al entorno real. En las fases finales, las pruebas sobre el robot físico permiten verificar el funcionamiento de la aplicación en condiciones reales.

Por último, el cliente móvil se ha probado sobre un teléfono y una tableta Samsung, permitiendo comprobar la adaptación de la interfaz a diferentes tamaños de pantalla. El uso de dispositivos físicos también permite evaluar aspectos que no pueden comprobarse completamente mediante los emuladores, como el rendimiento, la fluidez de la interfaz y el consumo de batería.

5.4. Diseño de la arquitectura del sistema

El diseño de la arquitectura del sistema es fundamental para definir su estructura y comportamiento global, y un buen diseño de la misma permitirá cumplir los requisitos establecidos. Para este proyecto se ha adoptado una arquitectura distribuida de tipo **Cliente-Servidor**, compuesta por un cliente (*frontend*), encargado de proporcionar la interfaz al usuario y realizar las peticiones, y un servidor (*backend*), responsable de procesarlas, comunicarse con el robot y responderlas [113].

La elección de esta arquitectura para el proyecto se debe principalmente

a dos razones, ambas condicionadas por el hecho de que la aplicación está destinada a dispositivos móviles Android:

- **Limitación de recursos en móviles:** el gran volumen de datos generado por ROS 2 hace conveniente delegar en el servidor su procesamiento, filtrado y compresión, reduciendo así el consumo de recursos y batería del dispositivo.
- **Limitaciones del ecosistema ROS 2 en Android:** se descartó la posibilidad de ejecutar ROS 2 directamente en Android mediante `rcljava` [114], debido a que se trata de una librería no oficial, desactualizada y con escaso soporte, cuya integración con Android presenta además una elevada complejidad.

Para gestionar la comunicación en tiempo real y sincronizar el comportamiento de ambos componentes se ha diseñado e implementado el protocolo **R2Pilot**, cuya justificación, diseño y desarrollo se detallan en el Capítulo 4.

Además, la arquitectura del sistema se ha diseñado para que el servidor (*backend*) se ejecute directamente en el ordenador del robot, evitando la dependencia de un servidor externo. Esta decisión permite acceder directamente al sistema y simplificar tareas de gestión, como la configuración o el reinicio del robot, además de reducir la latencia y el consumo de ancho de banda al mantener la comunicación DDS entre el servidor y el robot en local mediante la interfaz *localhost*.

Finalmente, la arquitectura debe gestionar de forma concurrente la comunicación, ROS 2 y la interfaz gráfica. Para ello, el servidor emplea `asyncio` para la comunicación *WebSocket* y `threading` para los nodos ROS 2, mientras que el cliente utiliza corrutinas de Kotlin con `Dispatchers.IO` para las tareas de red y procesamiento, manteniendo libre el hilo principal y garantizando una interfaz fluida.

A continuación, se describe la estructura interna de ambos componentes, comenzando por el servidor y continuando con el cliente.

5.4.1. Estructura del *backend*

La estructura del servidor se ha diseñado dividiendo sus responsabilidades en capas lógicas bien definidas, garantizando un código escalable y fácil de mantener. Esta separación abarca desde la gestión de la red hasta la interacción con el middleware ROS 2. En la Tabla 5.3 se detallan los módulos desarrollados para cada capa y su responsabilidad principal, los cuales se encuentran representados visualmente en el diagrama de bloques de la Figura 5.1.

Tabla 5.3: Estructura de módulos y archivos del servidor (Backend).

Capa Lógica	Archivo (.py)	Responsabilidad
Lógica Central	main.py	Punto de entrada e inicialización del sistema.
	director.py	Orquestador: procesa peticiones y delega tareas.
	state_machine.py	Implementa la máquina de estados del protocolo.
	host_utils.py	Interacción con el sistema operativo.
Comunicaciones	websocket_server.py	Establecimiento y gestión del canal físico WebSocket.
	connection_manager.py	Gestión de sesión lógica del cliente.
Protocolo	validator.py	Validación estructural de mensajes entrantes (JSON Schema).
	message_codec.py	Serialización y deserialización de la información.
	models.py	Clases de datos (<i>dataclasses</i>) de los mensajes.
ROS 2	ros2_node_gateway.py	Pasarela principal de comunicación con el ecosistema, ofrece una API interna y delega tareas al resto de módulos de esta capa.
	ros2_discovery.py	Descubrimiento dinámico de la red ROS 2 y <i>hardware</i> del robot.
	ros2_sensor_manager.py	Procesamiento y filtrado de flujos de los sensores.
	ros2_control_manager.py	Gestión de teleoperación y control articular.
	ros2_action_manager.py	Interacción con acciones prolongadas (<i>PlayMotion</i>).
	ros2_safety_node.py	Nodo independiente de supervisión de seguridad.
Utilidades	constants.py	Centralización de valores estáticos y configuraciones.

Para facilitar la integración con ROS 2, el backend se ha organizado dentro de un paquete propio de ROS 2 que reúne todos los archivos necesarios para el funcionamiento del proyecto. Su ejecución se controla mediante un archivo de lanzamiento específico (`backend.launch.py`), encargado de iniciar de forma conjunta tanto el servidor principal de la aplicación como los nodos auxiliares de ROS 2 que necesita el sistema. De esta manera, todo el entorno puede ponerse en marcha de forma sencilla y centralizada utilizando un único comando.

Para una comprensión más clara de esta estructura y la interacción entre los distintos bloques, se presenta la Figura 5.1, que constituye el diagrama de bloques del servidor.

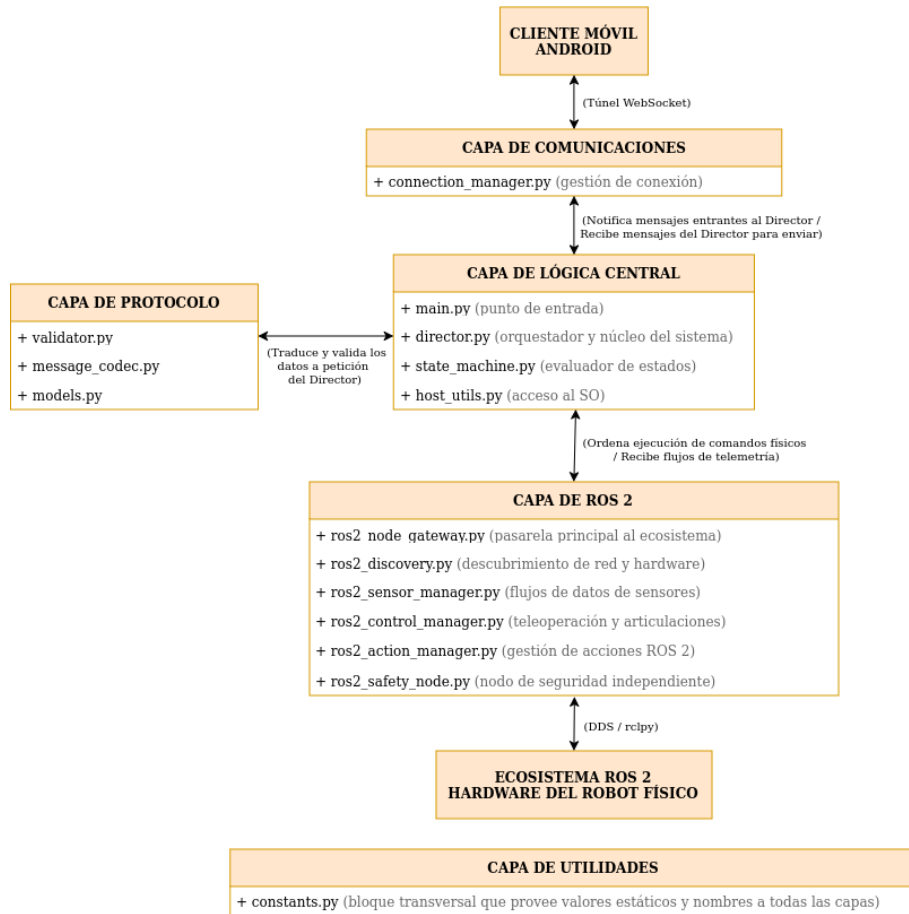


Figura 5.1: Diagrama de bloques de la arquitectura del servidor.

5.4.2. Estructura del *frontend*

Para el desarrollo del cliente móvil se ha implementado el patrón arquitectónico *Model-View-ViewModel* (MVVM), recomendado oficialmente por Google para Android [115, 116]. Esta organización permite mantener separada la interfaz de usuario de la lógica de la aplicación, manteniendo una clara separación de responsabilidades y facilitando así el mantenimiento, las pruebas y la incorporación de nuevas funcionalidades. En la Tabla 5.4 se describen los componentes agrupados por su capa correspondiente, cuyo flujo de interacción se ilustra en la Figura 5.2.

Tabla 5.4: Estructura de componentes del cliente móvil (Frontend - MVVM).

Capa MVVM	Componente (.kt)	Responsabilidad
Model	WebSocketClient, SessionManager	Gestión del túnel de red y ciclo de vida de la sesión lógica.
	Director, StateMachine	Lógica central: peticiones, validación y gestión de estados.
	MessageCodec, Models	Serialización y definición de estructuras de datos.
	Constants	Valores estáticos de la aplicación.
ViewModel	MainViewModel	Gestor del estado global y navegación principal de la app.
	*ViewModel (ej. Joystick...)	Intermediarios lógicos: mantienen el estado reactivo (<i>StateFlow</i>) de cada pantalla específica.
View	MainActivity, MainScreen	Punto de entrada, enrutador de navegación y diseño base.
	*Screen (ej. LobbyScreen)	Funciones <i>Composable</i> que dibujan la interfaz gráfica observando los estados del <i>ViewModel</i> .
	*Component (ej. Joystick...)	Elementos visuales reutilizables e interactivos.

Al igual que para el servidor, se presenta en la figura 5.2 el diagrama de bloques de la arquitectura del cliente para una mejor comprensión:

5.5. Diseño gráfico

En esta sección se describe el diseño de la interfaz gráfica y el resultado final tras su implementación, siguiendo los requisitos no funcionales RNF-8, RNF-9, RNF-10, RNF-11 y RNF-12, descritos en la Sección 6.2.

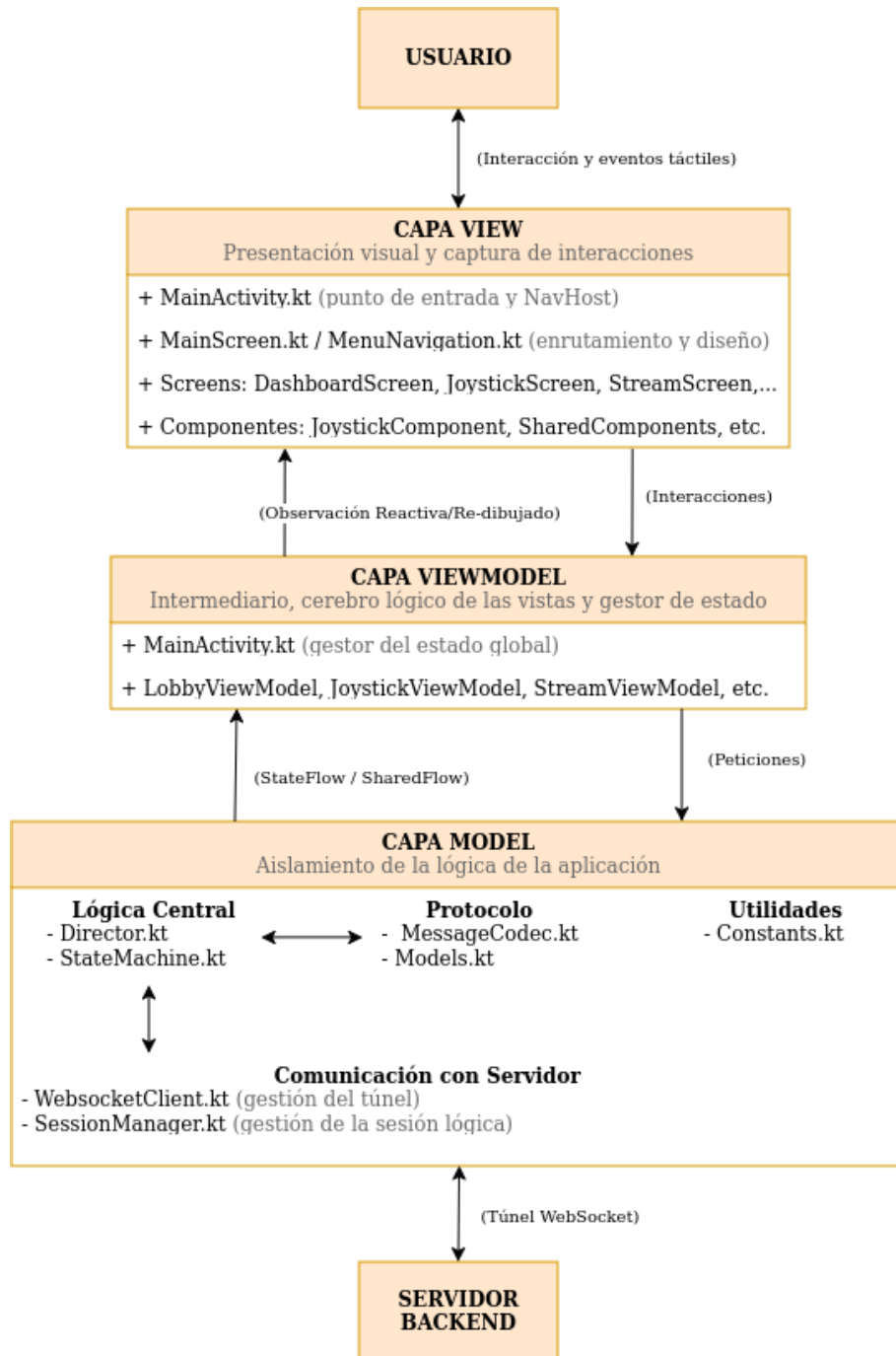


Figura 5.2: Diagrama de bloques de la arquitectura del cliente.

Debido a las limitaciones de espacio propias de los dispositivos móviles, el diseño busca aprovechar la superficie disponible manteniendo una interfaz sencilla, intuitiva y cómoda, que permita combinar las funciones de control y monitorización y presentar la información de los sensores de forma clara y ordenada. Para ello, se presentan los *wireframes* utilizados para definir la distribución de los elementos, el sistema de diseño empleado y, finalmente, la interfaz resultante.

5.5.1. *Wireframes* de la interfaz

Para definir la disposición de los elementos antes de la implementación, se desarrollaron *wireframes* iterativos utilizando la herramienta Draw.io. El diseño se centró en dos aspectos fundamentales de usabilidad:

- **Accesibilidad táctil:** se ha optado por una barra de navegación inferior e interactiva, facilitando su alcance con los pulgares mientras se sostiene el dispositivo.
- **Prevención de errores:** las acciones más críticas se han separado visual y espacialmente del resto de elementos, situándolas en zonas inferiores para reducir la posibilidad de accionarlas de forma accidental.

Los bocetos detallados de todas las interfaces se recogen en el **Anexo E** para su consulta.

5.5.2. Logotipo, tipografía, iconografía y paleta de colores

La identidad de la aplicación, bajo el nombre **R2Pilot** y el lema "*CONTROL. SIMPLIFIED.*", se diseñó para transmitir abstracción y sencillez en el control de robots basados en ROS 2. El sistema de diseño contiene los siguientes elementos:

- **Logotipo:** se han creado dos variantes (Figura 5.3): una extendida con animación de carga para la pantalla de bienvenida, y una versión compacta (R2PT) adaptada a las proporciones del icono de la aplicación Android.
- **Tipografía:** se emplea *Manrope* (sans-serif geométrica) para los elementos generales de la interfaz por su buena legibilidad en pantallas de diferentes tamaños y resoluciones, y *Monospace* para la telemetría, asegurando la alineación constante de los datos numéricos en tiempo real (práctica recomendada por Material Design y Apple) [117, 118].



(a) Logotipo de inicio.



(b) Icono de la aplicación.

Figura 5.3: Logotipo e icono de la aplicación.

- **Iconografía:** basada íntegramente en *Material Icons* de Google (código abierto), garantizando escalabilidad nativa en Jetpack Compose y coherencia con el ecosistema Android [119].
- **Paleta de colores:** (Figura 5.4) utiliza fondos de alto contraste (blancos/grises) y cian (el presente en el logo) como color principal, junto con textos oscuros, facilitando la lectura. Además, aplica un código de colores para informar del estado del robot de un vistazo: rojo (crítico/-parada), amarillo (advertencias/batería) y verde (seguro/conexión).



(a) Paleta principal.

(b) Paleta secundaria.

Figura 5.4: Paletas de colores de la aplicación.

Fuente: Elaboración propia por medio de Coolors.

5.5.3. Implementación de la interfaz

Una vez definida la estructura y el sistema visual, la interfaz se implementó utilizando Jetpack Compose. Para asegurar el mantenimiento y la coherencia gráfica, los parámetros se centralizaron en tres archivos principa-

les (`Color.kt`, `Type.kt` y `Theme.kt`). Algunos ejemplos del resultado final, aplicando estos principios sobre la arquitectura *ViewModel*, se muestran en la Figura 5.5. Para ver el resto de la interfaz en funcionamiento, se pueden consultar los vídeos de demostración referenciados en la sección 6.1 de este documento.

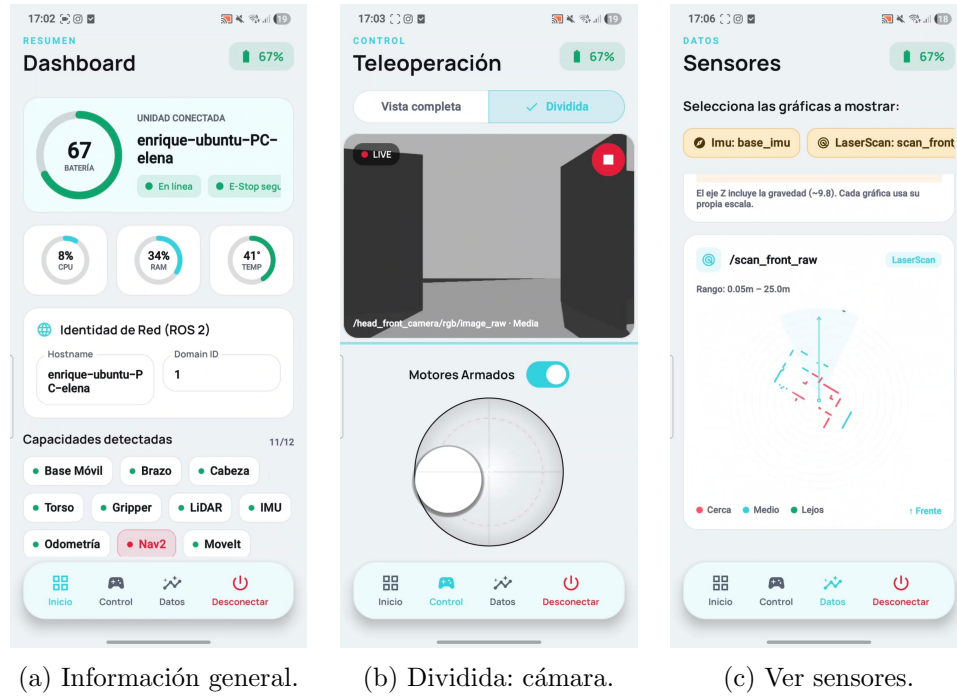


Figura 5.5: Conjunto de pantallas e interfaces de la aplicación.

5.6. Diseño y desarrollo de las funcionalidades de la aplicación

Para cerrar este capítulo, esta sección aborda el diseño y la implementación de las principales funcionalidades del sistema, definidas en los requisitos funcionales de la Sección 5.2. Para mantener una explicación concisa, el análisis se centra en el flujo de ejecución, los principales criterios de diseño y los componentes clave necesarios para su funcionamiento, evitando entrar en una descripción detallada del código fuente. En primer lugar, se presenta el flujo general de ejecución, común a todas las funcionalidades, y posteriormente se detallan sus particularidades, agrupándolas según sus características.

5.6.1. Funcionamiento general del sistema

Para la ejecución de cualquiera de las funcionalidades del sistema, tal como se mostró anteriormente en los diagramas de bloques de la aplicación (figuras 5.1 y 5.2), se sigue un flujo de ejecución común que puede resumirse en las siguientes etapas:

1. **Interacción:** el usuario inicia una acción desde la interfaz gráfica de la aplicación móvil (capa *View*).
2. **Gestión en el cliente:** la capa *ViewModel* notifica la intención al orquestador principal (*Director*) del *Model*, quien construye la petición formal tras validar que el estado actual lo permite.
3. **Transmisión:** los componentes de comunicación y protocolo preparan y envían la petición mediante el canal *WebSocket* al servidor.
4. **Procesamiento en el servidor:** el servidor recibe y decodifica la petición, y su *Director* interno enruta la operación hacia los nodos de ROS 2 o el sistema operativo.
5. **Respuesta y actualización:** tras ejecutar la tarea, el servidor emite el resultado. El cliente lo decodifica, lo correlaciona con la petición pendiente y actualiza de forma reactiva la interfaz.

Dado que la gestión de WebSockets, la traducción de mensajes y la máquina de estados ya se han explicado en detalle en la sección 4.2, el análisis se centra a continuación en los archivos *Director* de ambos extremos, encargados de coordinar las distintas funcionalidades del sistema.

- En el **servidor** (*director.py*), actúa como la principal barrera de control. Al recibir un paquete, valida el *token* de sesión y consulta a la máquina de estados. Si la operación es lícita, enruta el mensaje hacia el gestor específico de la capa de ROS 2 (sensores, acciones, control) o del sistema *host*. Finalizada la operación, prepara la respuesta y la emite de forma asíncrona, garantizando que no se bloquee el hilo principal del servidor web.
- En el **cliente** (*Director.kt*), su responsabilidad es exponer una API unificada para que los *ViewModels* interactúen con la red. Antes de emitir un paquete, consulta la máquina de estados local. Las peticiones aprobadas se transmiten y se almacenan temporalmente en un registro de espera (*ConcurrentHashMap*) junto con un temporizador de caducidad (*timeout*) de 5 segundos como medida de seguridad. Al recibir los paquetes entrantes, el orquestador los relaciona con las peticiones pendientes y actualiza los flujos reactivos (*StateFlow* y *SharedFlow*), los cuales notifican automáticamente los cambios a la interfaz gráfica.

5.6.2. Configuración de red y gestión remota del robot

Estas funcionalidades, la reconfiguración de los parámetros de red de ROS 2 y la gestión del apagado o reinicio remoto, están disponibles en la fase previa al inicio de sesión y, a diferencia del resto, no interactúan con el ecosistema de ROS 2, sino directamente con el sistema operativo anfitrión (Linux) del robot.

Desde el cliente (`LobbyScreen.kt`), el usuario puede desencadenar estas operaciones, las cuales se empaquetan en mensajes `CommandReq` indicando la acción deseada. En el servidor, el *Director* intercepta estas peticiones y delega su ejecución en el módulo `HostSystemManager`, el cual gestiona dos escenarios principales:

- **Configuración de red (`action=change_vars`):** el servidor aplica los nuevos parámetros de red modificando el archivo `.bashrc` del sistema para garantizar que la configuración de ROS 2 persista tras reiniciar. Adicionalmente, si el usuario habilita el modo *Discovery Server*, el gestor crea y habilita automáticamente un servicio nativo de Linux (`systemd`) para asegurar que dicho servidor de descubrimiento se ejecute en el arranque.
- **Gestión de energía (`action=reboot / shutdown`):** por seguridad, el servidor detiene cualquier movimiento en curso y desconecta ordenadamente los nodos de ROS 2. Una vez finalizada la desconexión segura, se envía el comando de apagado o reinicio al sistema operativo.

5.6.3. Obtención de información del robot

Estas funcionalidades permiten obtener información del propio robot, como su estado, identidad y capacidades, así como descubrir los tópicos, servicios y acciones disponibles en la red de ROS 2. Todas las solicitudes utilizan mensajes del tipo `QueryReq`, cuyo campo `resourceType` determina la información solicitada.

Gestión desde el cliente y adaptación de la interfaz Para optimizar el ancho de banda, la aplicación cliente emplea una estrategia de multiplexación temporal en la pantalla principal: alterna cada 2,5 segundos entre la consulta del estado del robot (`HOST_INFO`) y de sus características y capacidades (`ROBOT_INFO`), logrando un refresco de información completo cada 5 segundos. Las consultas de la red ROS 2 (`TOPICS`, `SERVICES`, `ACTIONS`) se solicitan bajo demanda del usuario.

La obtención del `ROBOT_INFO` resulta fundamental, ya que la interfaz evalúa las capacidades descubiertas (como `hasCameras`, `hasBase` o `hasPlayM-`

otion) y construye la barra de navegación dinámicamente, habilitando únicamente los módulos compatibles con el hardware del robot conectado.

Descubrimiento dinámico en el servidor (*DiscoveryManager*)

Para evitar depender de configuraciones estáticas y lograr que la aplicación sirva para distintos robots, el módulo *DiscoveryManager* explora la anatomía del robot analizando el grafo de ROS 2 a través de la API de `rclpy`. Este proceso utiliza dos estrategias complementarias, cuya teoría se explica en el capítulo 2:

- **Identificación del tipo de mensaje:** se buscan tipos de mensajes que permitan identificar directamente un componente. Por ejemplo, un tópico de tipo `sensor_msgs/msg/Imu` confirma la presencia de un sensor IMU.
- **Filtrado por nombre:** cuando el tipo de mensaje es más genérico, como `trajectory_msgs/msg/JointTrajectory`, se utilizan palabras clave en el nombre del tópico para determinar el componente asociado.

Además, el mecanismo de descubrimiento incorpora una medida de seguridad para evitar confusiones cuando hay varios robots en el mismo *Domain ID*. Para ello, comprueba que no existan tópicos críticos duplicados y, si detecta más de un robot activo, bloquea la conexión y notifica al cliente, evitando así posibles colisiones o el cruce de órdenes. Finalmente, parámetros ajenos a ROS 2, como el uso de memoria o CPU (`HOST_INFO`), se extraen interrogando directamente al sistema Linux mediante utilidades como `psutil`.

Exploración de interfaces de ROS 2 Por último, para la obtención bajo demanda de los elementos de comunicación (tópicos, servicios y acciones), el *DiscoveryManager* aprovecha las herramientas de introspección nativas de la API de `rclpy` (como `get_topic_names_and_types`). Al procesar la solicitud, el servidor escanea el grafo de ROS 2 en tiempo real y devuelve al cliente una lista estructurada con los elementos demandados y sus tipos.

5.6.4. Control del robot

Las funcionalidades de esta sección forman una parte fundamental de la aplicación, ya que permiten al usuario controlar de forma remota el robot. Se incluyen la teleoperación de la base móvil y el control de articulaciones, que se analizan conjuntamente por su funcionamiento similar, y la ejecución de movimientos predefinidos mediante el módulo *PlayMotion*.

Teleoperación y control de articulaciones

Para el control en tiempo real se utiliza el flujo continuo de mensajes `ControlReq`. En la teleoperación (`JoystickScreen.kt`), las coordenadas táctiles del *joystick* se traducen a vectores de velocidad lineal y angular, transmitiéndose a una frecuencia optimizada de 10 Hz, o a 2,5 Hz si no se utiliza el joystick, consiguiendo ahorrar ancho de banda. Para el control articular (`JointControlScreen.kt`), las barras deslizantes envían la posición objetivo a 20 Hz durante la interacción del usuario. En el propio cliente móvil, un mecanismo evalúa la velocidad de arrastre de estas barras, bloqueando comandos que pudieran resultar en movimientos excesivamente rápidos que pudieran poner en riesgo la integridad del robot.

En el servidor, el `DiscoveryManager` provee la información necesaria para estas funcionalidades al cliente analizando la red y el archivo URDF (*Unified Robot Description Format*), publicado en el tópico `/robot_description` [120], identificando los tópicos de teleoperación, los controladores articulares activos y sus límites de movimiento.

Durante la ejecución del movimiento, el servidor implementa un doble mecanismo de seguridad:

- **Filtros de red:** en la teleoperación, se evalúa la latencia de llegada de cada paquete. Si un comando de velocidad llega con un retraso superior a 0,6 segundos, se descarta por seguridad. Adicionalmente, un *watchdog* a 10 Hz comprueba si en el envío transcurre 1 segundo sin datos, y en ese caso, asume una desconexión y fuerza un frenado de emergencia. Este filtro no se aplica al control de articulaciones porque estas mantienen su última posición ante una pérdida de conexión. De esta forma, también se evita el envío continuo de posiciones estáticas y se reduce el consumo de ancho de banda.
- **Nodo de seguridad anticolidión (`R2PilotSafetyNode`):** en lugar de publicar las velocidades directamente en el controlador del robot, el servidor las inyecta en un tópico intermedio. Este nodo de seguridad intercepta dichas velocidades, las contrasta en tiempo real con las lecturas de proximidad del sensor LiDAR y, si detecta riesgo de colisión o exceso de velocidad, ajusta o descarta el comando antes de reenviarlo al robot. Para las articulaciones, asegura que la posición solicitada jamás exceda los límites angulares definidos en el URDF.

Ejecución de movimientos predefinidos

La ejecución de rutinas complejas se gestiona mediante el estándar *Actions* de ROS 2, integrándose con el paquete oficial `play_motion2` [121]. Previamente, la interfaz pregunta al servidor mediante un `QueryReq` para listar los movimientos disponibles, para poder mostrárselos al usuario.

Al iniciar un movimiento (`ActionReq`), el `ActionManager` del servidor actúa como *Action Client*. Tras consultar al servicio interno del robot la estimación temporal de la rutina, lanza la operación de forma asíncrona. Para mantener informado al usuario, un temporizador a 4 Hz interpola el tiempo transcurrido con la duración estimada, emitiendo mensajes `ActionFeedback` que la aplicación muestra como una barra de progreso. En cualquier momento, el operador puede enviar un `StopActionReq`, el cual invoca la función que cancela la acción de ROS 2.

5.6.5. Monitorización de la cámara y sensores en tiempo real

Estas funcionalidades permiten monitorizar el robot en tiempo real. El cliente solicita estas transmisiones enviando un `STREAM_REQ` (indicando el recurso, tópico y calidad) y las finaliza mediante un `STOP_STREAM_REQ` explícito o al navegar a otra pantalla.

Transmisión de vídeo (Cámaras)

Para evitar la saturación del canal WebSocket, la transmisión de vídeo se delega al protocolo HTTP utilizando el paquete `web_video_server` de ROS 2, el cual expone los flujos de imágenes en formato MJPEG [122]. Tras descubrir los tópicos `sensor_msgs/msg/Image`, el servidor genera y envía la URL donde se publica el vídeo, que el cliente reproduce mediante un componente `WebView` (`AndroidView`).

Durante las pruebas en el entorno real surgió un conflicto relacionado con la Calidad de Servicio (QoS), ya que el robot publicaba el flujo de cámara bajo un perfil *Best Effort*, mientras que `web_video_server` requería suscripciones *Reliable*, impidiendo la recepción. En lugar de modificar paquetes de terceros o alterar la configuración base del robot prestado, se diseñó una solución transparente, donde el `R2PilotBridgeNode` actúa como un puente de retransmisión (*relay*) local. Se suscribe al vídeo original en *Best Effort* y lo republica internamente en `/camera/rgb/relay` con un perfil compatible, garantizando la estabilidad y portabilidad del sistema.

Procesamiento de sensores

Al solicitar un sensor, el `SensorManager` inicializa su suscripción en ROS 2 y un *callback* dedicado intercepta y estandariza los datos del sensor al formato del protocolo.

Además, se regula la frecuencia de envío en función del sensor solicitado y, para estructuras de alta densidad como el LiDAR, se realiza un submuestreo espacial, descartando puntos intermedios para transmitir únicamente 1 de cada N muestras. Estos valores están justificados en el capítulo 6. Los datos filtrados se envían al cliente, donde se renderizan mediante la API `Canvas` de Android, de baja sobrecarga gráfica.

Capítulo 6

Evaluación y resultados

Este capítulo evalúa el sistema desarrollado para comprobar el cumplimiento de los requisitos iniciales y analizar sus principales limitaciones. Para ello, se presentan las pruebas realizadas sobre la aplicación móvil bajo diferentes condiciones, incluyendo el análisis de las comunicaciones y los *middlewares* empleados, el tráfico de red generado y su variación según el DDS utilizado, así como el rendimiento de la aplicación. Finalmente, se estudia su comportamiento ante situaciones adversas, como cortes o degradaciones de la conexión, con el fin de evaluar su robustez y tolerancia a fallos.

6.1. Resultados finales y evaluación técnica

El resultado final del proyecto es una plataforma completa y funcional, formada por un servidor intermedio integrado en el robot y una aplicación cliente desarrollada para dispositivos móviles Android. El sistema permite controlar y supervisar el robot de una forma sencilla y fluida, cumpliendo con los objetivos planteados al inicio del proyecto y con los requisitos diseñados para la aplicación.

La validación del sistema se realizó principalmente mediante pruebas manuales y continuas, comprobando el correcto funcionamiento del sistema y de cada funcionalidad integrada. Para garantizar la seguridad, las primeras pruebas se llevaron a cabo en un entorno simulado con Gazebo. Posteriormente, se realizaron pruebas sobre el robot físico TIAGo de Pal Robotics para verificar el funcionamiento de la aplicación en condiciones reales [112]. Finalmente, para comprobar la compatibilidad con otros robots basados en ROS 2, se realizaron pruebas en simulación con el robot Leo Rover de ROS Components, un robot más sencillo que TIAGo. [123, 124]

Finalmente, una vez desarrollada la aplicación, se han identificado algunas limitaciones adicionales a las previstas inicialmente, derivadas tanto de

las decisiones de diseño adoptadas como de ciertas dificultades encontradas durante la implementación de las diferentes funcionalidades:

- **Limitaciones en la universalidad de la aplicación:** el sistema de autodescubrimiento, en algunos casos, se basa en nombres de tópicos habituales. Por ello, algunos robots con configuraciones diferentes podrían necesitar alguna configuración adicional para su adecuación.
- **Dependencia de Linux:** el servidor debe ejecutarse en un sistema Linux, ya que necesita utilizar herramientas del sistema para gestionar determinadas funciones.
- **Dependencia de paquetes externos:** algunas funcionalidades requieren software adicional instalado en el robot. En particular, la ejecución de animaciones depende de `PlayMotion`, mientras que la transmisión de vídeo depende de `web_video_server`.

Para mostrar el funcionamiento final de la aplicación en entornos reales y simulados, se incluyen vídeos con sus principales funcionalidades. Estos están disponibles en listas de reproducción para las pruebas realizadas con el robot TIAGo y las pruebas en simulación con Leo Rover. Además, el Anexo A incluye un manual de instalación y ejecución de la aplicación.

6.1.1. Entorno y condiciones de experimentación

Todas las pruebas presentadas en las siguientes secciones se realizan utilizando los mismos equipos y la misma configuración de red. A continuación, se detalla esta configuración, que se mantiene como base común para todas las evaluaciones de rendimiento, latencia y tráfico de red:

- **Equipo de simulación (PC):** la simulación del robot mediante Gazebo se ha ejecutado en un equipo de sobremesa con procesador Intel Core i7-14700K, 32 GB de memoria RAM DDR5 y tarjeta gráfica NVIDIA GeForce RTX 4060 Ti (16 GB), bajo el sistema operativo Ubuntu 22.04 LTS nativo.
- **Equipo servidor (portátil):** el *backend* y el procesamiento de la comunicación se han alojado en un ordenador portátil LG Gram (modelo 15Z90Q), equipado con un procesador Intel Core i7-1260P y 16 GB de memoria RAM, ejecutando igualmente Ubuntu 22.04 LTS.
- **Equipo cliente (móvil):** el *frontend* se han realizado utilizando un teléfono inteligente Samsung Galaxy S25.

- **Red Inalámbrica (Wi-Fi):** la comunicación inalámbrica entre los dispositivos se ha realizado sobre una red Wi-Fi local y privada bajo el estándar Wi-Fi 5 (802.11ac), operando en la banda de 5 GHz. La infraestructura de red estuvo gestionada por un router Arcadyan PRV3397B-E-LT proporcionado por el ISP, con una buena calidad de la señal.

6.2. Evaluación del Middleware de ROS 2 (DDS)

En esta sección se analiza el comportamiento de la red local durante la ejecución de la aplicación utilizando distintas implementaciones de DDS, prestando especial atención a las comunicaciones entre el servidor y el robot. Los resultados permiten comparar el comportamiento de cada implementación y determinar cuál se adapta mejor a las necesidades del proyecto.

6.2.1. Metodología y diseño del experimento

Para aislar y caracterizar el tráfico, el experimento se definió bajo las siguientes condiciones:

- **Escenarios evaluados:** Cyclone DDS, Fast DDS (*Multicast*) y Fast DDS mediante *Discovery Server*. La implementación de DDS se seleccionó mediante la variable `RMW_IMPLEMENTATION`, mientras que `ROS_DISCOVERY_SERVER` permitió configurar el servidor de descubrimiento [125].
- **Fases de análisis:** fase de descubrimiento, correspondiente a los primeros 30 s, y estado de reposo, durante 60 s.
- **Despliegue:** dos equipos conectados a la misma red Wi-Fi, uno ejecutando la simulación del robot en Gazebo y otro alojando el servidor.
- **Captura de red:** mediante `tshark`, con posterior procesamiento en Python. Se aplicaron filtros sobre los paquetes UDP correspondientes al rango de puertos asociado al `ROS_DOMAIN_ID` utilizado y, cuando procedía, al puerto específico del *Discovery Server* [22].

6.2.2. Análisis de la fase de descubrimiento

En la red inalámbrica, el descubrimiento genera un tráfico significativo. La Figura 6.1 compara los tres escenarios: *Fast DDS* con *Multicast* alcanza los mayores picos, superiores a 4000 KB/s, mientras que *Cyclone DDS* los reduce aproximadamente a la mitad (2000 KB/s). Por su parte, *Fast DDS*

con *Discovery Server* minimiza aún más el tráfico al centralizar el descubrimiento y reducir el intercambio directo entre nodos.

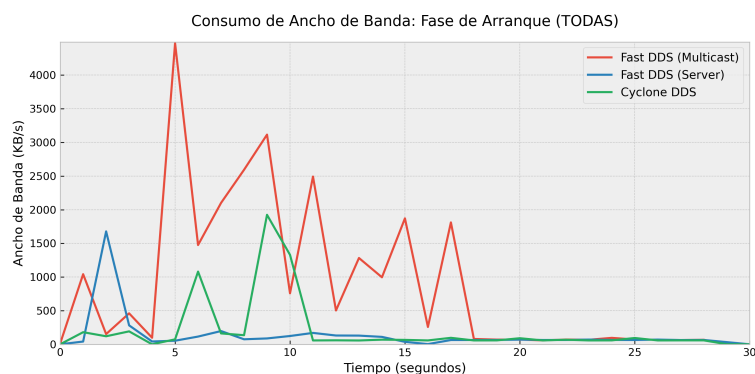


Figura 6.1: Consumo de ancho de banda del tráfico DDS (primeros 30 s).

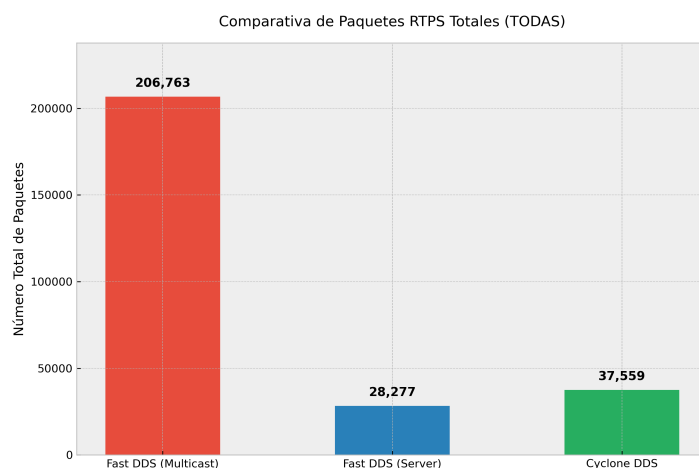


Figura 6.2: Comparativa del número total de paquetes RTPS durante el descubrimiento.

Esta congestión no se debe al tamaño de los datos, sino a la gran cantidad de mensajes generados durante el descubrimiento. Como muestra la Figura 6.2, *Fast DDS* con *Multicast* generó 206,763 paquetes, frente a los 37,559 de *Cyclone DDS* y los 28,277 del *Discovery Server*.

6.2.3. Análisis del estado de reposo

Una vez finalizado el descubrimiento, el consumo de ancho de banda depende principalmente del intercambio de datos. Como muestra la Figura 6.3, los tres escenarios presentan un comportamiento muy similar, con

un consumo de entre 60 y 70 KB/s. Destacan los pequeños picos de *Cyclone DDS*, alcanzando unos 95 KB/s, que corresponden a los mensajes de mantenimiento de la conexión (*heartbeats*) enviados por los nodos.

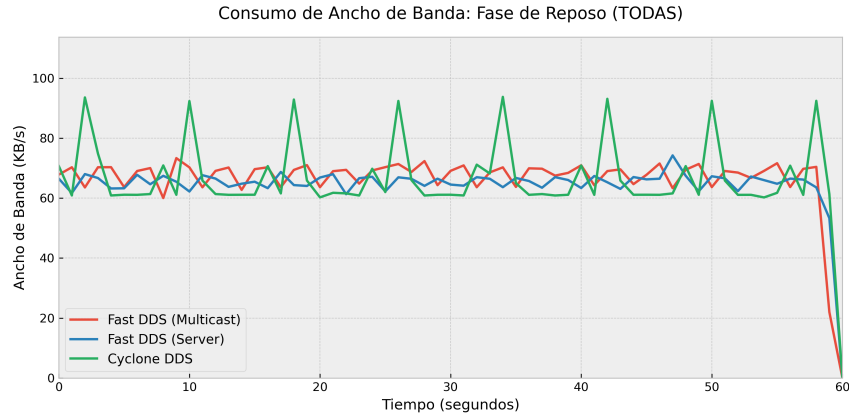


Figura 6.3: Consumo de ancho de banda durante el estado de reposo (60 segundos).

Estos resultados muestran que, una vez establecida la comunicación, la implementación de DDS apenas afecta al consumo de red, el cual es mucho menor que en la fase de descubrimiento.

6.2.4. Conclusiones

Los resultados muestran que la fase de descubrimiento genera un tráfico de red considerablemente mayor que el estado de reposo, especialmente con *Cyclone DDS* y, más aún, con *Fast DDS* mediante *Multicast*, debido a la elevada cantidad de paquetes intercambiados durante el descubrimiento.

En este contexto, *Cyclone DDS* ofrece un funcionamiento eficiente sin configuración adicional, mientras que *Fast DDS* con *Discovery Server* resulta la opción más adecuada para minimizar el tráfico, al centralizar el descubrimiento mediante *Unicast* y reducir considerablemente el intercambio *Multicast* entre nodos.

6.3. Análisis y optimización del tráfico de red generado

Al ubicar el servidor en el propio robot, el tráfico DDS queda limitado al entorno local (*localhost*), reduciendo la carga sobre la red Wi-Fi. Esta sección analiza el tráfico global generado durante la ejecución de la aplicación

para evaluar el impacto de esta decisión y optimizar la comunicación con el dispositivo móvil. Para ello, se estudia tanto el tráfico DDS que se evita transmitir por la red inalámbrica como el generado por la propia aplicación, sobre el que se aplican estrategias de optimización como el ajuste de las frecuencias de transmisión y el submuestreo de los datos del LiDAR.

6.3.1. Tráfico generado por la aplicación

6.3.1.1. Metodología y diseño del experimento

En este apartado se analiza el tráfico generado por la aplicación en tres escenarios: reposo, teleoperación de la base móvil y visualización del LiDAR. Para medir ambos flujos simultáneamente, el servidor y el simulador se ejecutaron en equipos diferentes, haciendo que el tráfico DDS y Web-Socket atravesara la misma interfaz Wi-Fi. Las capturas se realizaron con *Wireshark* y se analizaron mediante filtros específicos:

- **Tráfico DDS:** filtrado mediante `rtps` y el identificador `wrEntityId` para aislar los tópicos de cada funcionalidad.
- **Tráfico R2Pilot:** filtrado mediante el puerto TCP del servidor Web-Socket y las cabeceras JSON del *payload*, que identifican los mensajes `ControlReq/Resp` y `StreamReq/Resp`.

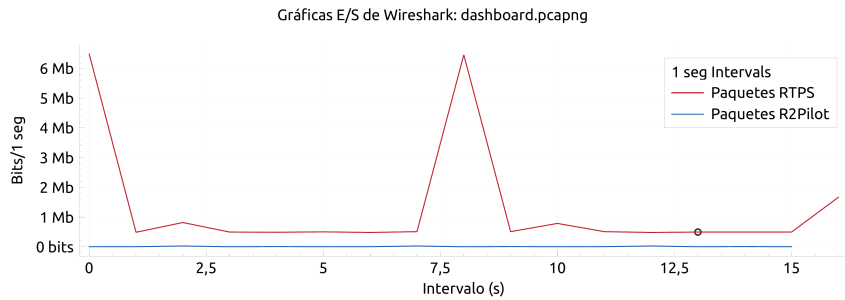
6.3.1.2. Aplicación en reposo

En este primer escenario, el usuario mantiene la aplicación abierta en la pantalla principal, ya conectada al robot, pero sin utilizar ninguna funcionalidad, durante 15 segundos.

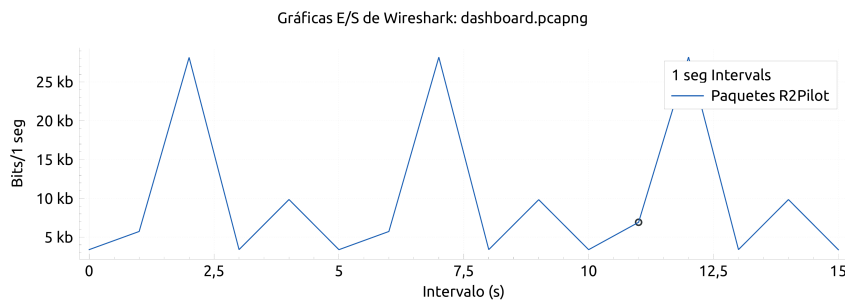
En la Figura 6.4 se observa una diferencia significativa entre el tráfico RTPS y R2Pilot. El tráfico DDS mantiene una carga de aproximadamente 1 Mbps, con picos de hasta 6,5 Mbps, debidos principalmente a la recepción continua en el servidor del LiDAR y del estado de las articulaciones, además del intercambio de mensajes RTPS entre nodos. En cambio, R2Pilot genera únicamente pequeños picos inferiores a 30 kbps, asociados principalmente a las actualizaciones periódicas del menú y a los mensajes *PingReq/PingResp*.

6.3.1.3. Uso de funcionalidades

En este caso, se captura únicamente el tráfico de la funcionalidad analizada, también durante 15 segundos, considerando los paquetes correspondientes a los comandos de control o datos del LiDAR y sus respectivas consultas y respuestas.



(a) Tráfico DDS y R2Pilot.



(b) Tráfico exclusivo de R2Pilot.

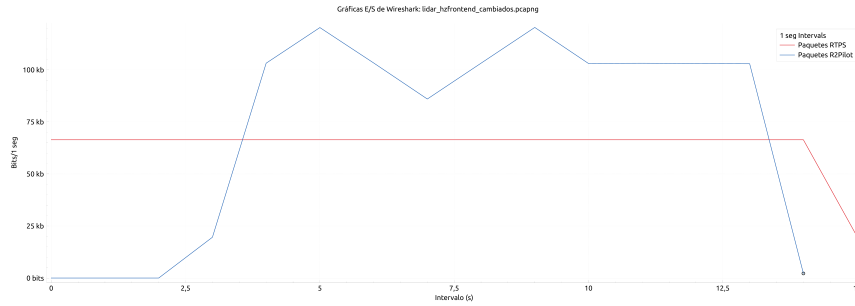
Figura 6.4: Tráfico generado con la aplicación en reposo.

En la teleoperación (Figura 6.5b), ambos protocolos presentan poco consumo de ancho de banda, aunque R2Pilot genera ligeramente más tráfico (unos 25 kbps frente a los 23 kbps de DDS). Esta diferencia se debe principalmente al uso de JSON, cuyo formato de texto introduce una pequeña sobrecarga respecto a la transmisión binaria de DDS.

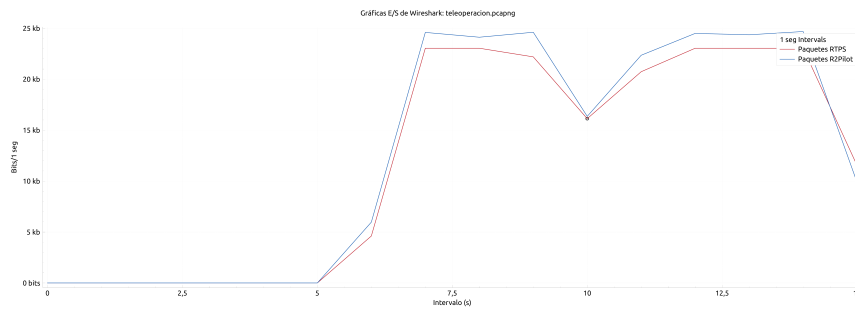
La diferencia es más notable con los datos del LiDAR (Figura 6.5a). A pesar del submuestreo aplicado, la conversión de los datos a JSON eleva el consumo de R2Pilot hasta unos 120 kbps, mientras que DDS mantiene un flujo de aproximadamente 65 kbps. Sin embargo, este incremento sigue siendo reducido para las redes Wi-Fi actuales y, cabe destacar, que R2Pilot solo transmite estos datos cuando son solicitados por la aplicación. De este modo, se evita mantener de forma continua el tráfico del LiDAR en la red cuando el usuario no necesita visualizarlo.

6.3.1.4. Conclusiones

El análisis del tráfico confirma que alojar el servidor en el propio robot y utilizar R2Pilot para la comunicación con el cliente son decisiones adecuadas. Aunque R2Pilot introduce un pequeño sobrecoste por el uso de JSON, este resulta despreciable frente al beneficio de mantener el tráfico continuo de



(a) Tráfico para datos del LIDAR.



(b) Tráfico para teleoperación.

Figura 6.5: Tráfico generado por las funcionalidades de la aplicación.

ROS 2 en la red local. De este modo, se reduce la carga de la red Wi-Fi y del dispositivo móvil, transmitiendo únicamente los datos necesarios y bajo demanda.

La visualización de la cámara no se incluye en la comparativa, ya que utiliza un canal independiente mediante HTTP y *web_video_server*. No obstante, este caso también respalda la arquitectura propuesta, puesto que transmitir directamente el flujo de vídeo de ROS 2 sin procesar por Wi-Fi generaría una carga elevada que podría afectar a la estabilidad y latencia del sistema.

6.3.2. Optimización del tráfico enviado al cliente

Para mantener una comunicación fluida y en tiempo real mediante R2Pilot, se aplican distintas técnicas de optimización del tráfico. Estas consisten principalmente en regular la frecuencia de envío de los mensajes y reducir la cantidad de datos transmitidos cuando no afecta a su visualización, destacando el submuestreo aplicado al LiDAR por su elevado volumen de datos.

6.3.2.1. Optimización de la frecuencia de envío

Para evaluar el efecto de la frecuencia de transmisión sobre el consumo de red, se capturó el tráfico con *Wireshark* para distintas frecuencias y se procesó mediante un script de Python para generar gráficas comparativas. El objetivo es determinar una frecuencia que permita mantener la recepción en tiempo real sin generar tráfico innecesario en la red inalámbrica.

Este ajuste se aplica a los comandos de control y datos de los sensores, adaptándolo a las necesidades de cada uno. Para evitar extender la memoria y repetir el análisis, se presentan dos casos representativos: la teleoperación de la base móvil y la visualización de los datos del sensor IMU.

Envío de comandos de control En el caso de la teleoperación de la base móvil (ver Figura 6.6), se evaluaron frecuencias de envío entre 2 Hz y 50 Hz. Como muestra la gráfica, a 50 Hz el consumo de red se acerca a los 100 kbps, aunque esta frecuencia resulta innecesaria para enviar comandos de movimiento al robot. Por otro lado, 2 Hz reduce considerablemente el tráfico, pero provoca un movimiento poco fluido e inseguro. Por este motivo, se seleccionó una frecuencia de **10 Hz**, que ofrece un buen equilibrio entre fluidez y consumo de red.

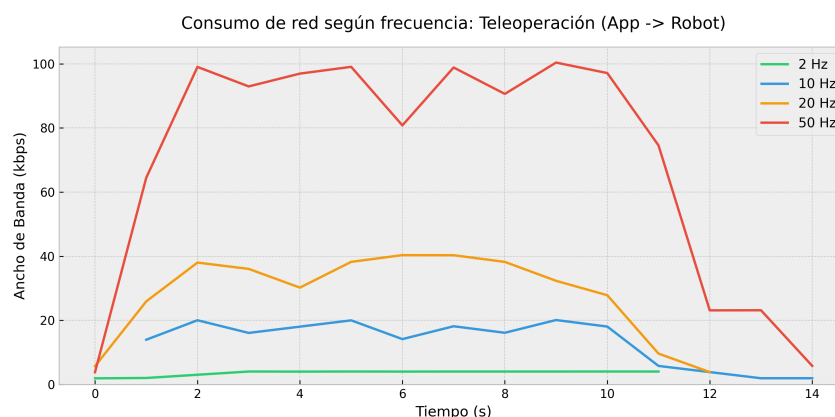


Figura 6.6: Análisis de frecuencias de teleoperación.

Envío de telemetría y sensores En el caso del sensor IMU (Figura 6.7), el comportamiento es diferente. El robot publica sus lecturas a 100 Hz, pero reenviarlas al cliente móvil a esta misma frecuencia supondría un consumo superior a 250 kbps para un solo sensor, además de generar actualizaciones más rápidas de lo que el usuario puede percibir. Para evitarlo, el servidor limita el envío a **20 Hz**. De esta forma, la interfaz mantiene una respuesta

fluida y en tiempo real, mientras que el consumo de red se reduce en más de un 60 % respecto a la frecuencia de publicación del robot.

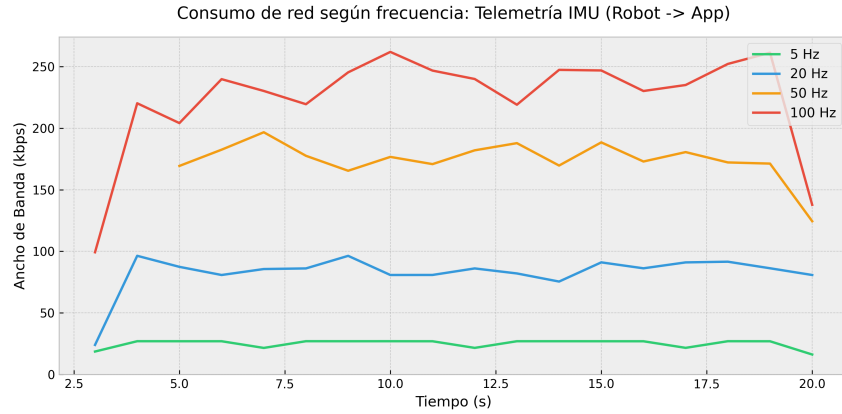


Figura 6.7: Análisis de frecuencias del sensor IMU.

6.3.2.2. Optimización de los datos del LiDAR

Además del ajuste de frecuencias, las estructuras de datos densas requieren reducir la cantidad de información transmitida. El caso más relevante es el LiDAR, cuyos escaneos contienen cientos de valores de distancia y pueden superar los 300 kbps al enviarse completos en JSON (Figura 6.8).

Para reducir este consumo, el servidor aplica un submuestreo enviando únicamente 1 de cada N puntos. Los resultados muestran que los factores $1/3$ y $1/10$ reducen el consumo hasta aproximadamente 100–120 kbps y 50 kbps, respectivamente. Sin embargo, la comparación visual (Figura 6.9) muestra que, debido al tamaño y resolución de la pantalla móvil, no se aprecian diferencias significativas entre los datos completos y los factores $1/2$ y $1/3$. Por ello, se seleccionó **$1/3$** , que reduce el consumo en más de un 60 % manteniendo una representación clara.

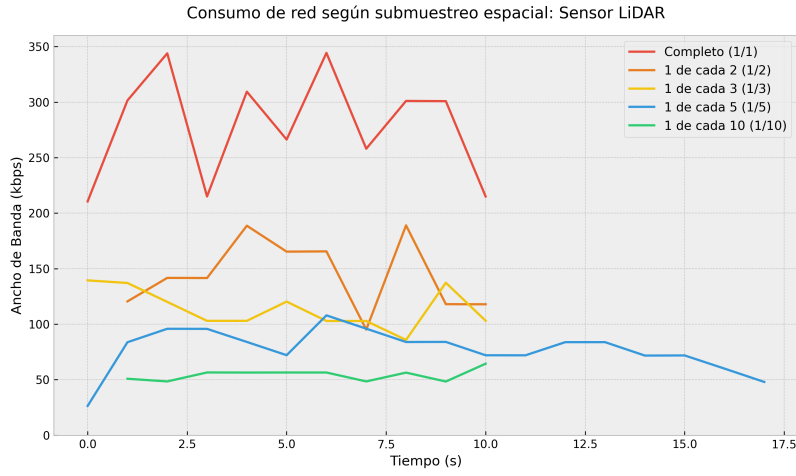
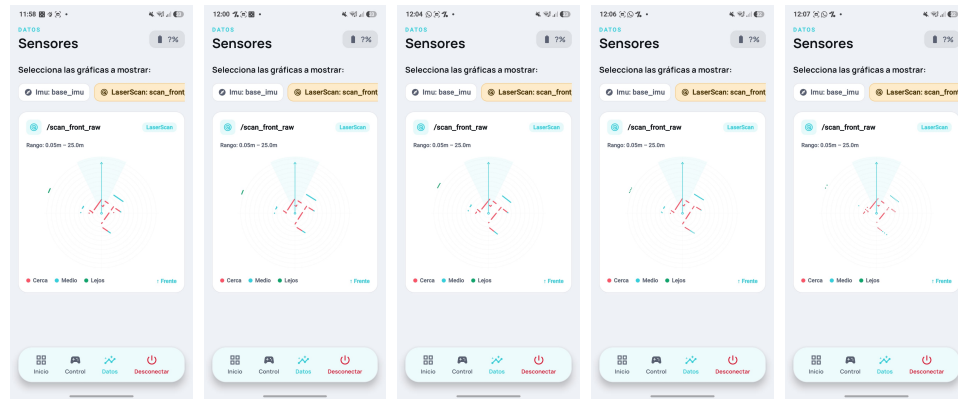


Figura 6.8: Gráfica comparativa del submuestreo del LiDAR.



(a) Envío completo. (b) Envío 1/2 puntos. (c) Envío 1/3 puntos. (d) Envío 1/5 puntos. (e) Envío 1/10 puntos.

Figura 6.9: Interfaz gráfica del LiDAR con diferentes niveles de muestreo.

6.4. Evaluación de requisitos de red

Una vez analizadas y optimizadas las distintas funcionalidades por separado, se evalúa el comportamiento del sistema cuando todas funcionan simultáneamente en condiciones reales de uso. El objetivo es estudiar el escenario más exigente, determinando el ancho de banda necesario y comprobando que la latencia permite mantener una comunicación en tiempo real.

Dado que el tráfico DDS se ha justificado que permanece dentro del

robot, el análisis considera únicamente el tráfico que circula por la red Wi-Fi: los mensajes del protocolo *R2Pilot* y el vídeo transmitido mediante HTTP por *web_video_server*.

6.4.1. Evaluación del ancho de banda

Para evaluar el ancho de banda consumido por la aplicación, se ha analizado el tráfico de red durante 30 segundos mediante la captura de paquetes con *Wireshark* en dos escenarios críticos:

- **Escenario A (Teleoperación y todos los sensores activos):** el usuario envía continuamente comandos de velocidad mientras recibe simultáneamente los datos de todos los sensores del robot. Como se muestra en la Figura 6.10, el consumo alcanza picos de ~ 600 kbps, dominados por los datos sensoriales (~ 500 kbps), mientras que los comandos de control apenas representan ~ 20 kbps. El resto corresponde a mensajes de control y consulta de información.
- **Escenario B (Teleoperación y flujo de vídeo activo):** Se combina la conducción de la base con el flujo de vídeo HTTP (Figura 6.11), representando el mayor consumo del sistema, con picos de hasta 1,6 Mbps. El vídeo de *web_video_server* supone más del 97 % del tráfico, con valores de 1–1,5 Mbps, mientras que el tráfico WebSocket de *R2Pilot* se mantiene por debajo de 45 kbps.

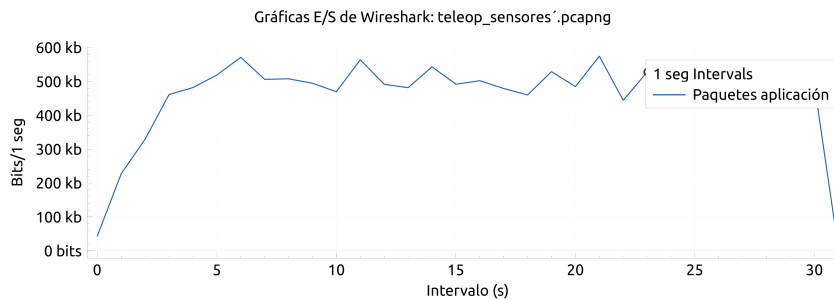


Figura 6.10: Gráfica escenario A.

Estos resultados muestran que la aplicación presenta un consumo compatible con una red Wi-Fi convencional. El tráfico máximo alcanza 1,6 Mbps, debido principalmente al vídeo, mientras que sin este flujo se mantiene por debajo de 600 kbps incluso con todos los sensores activos. Este comportamiento es consecuencia de las decisiones de diseño adoptadas, tanto a nivel arquitectónico, manteniendo el tráfico DDS en el robot, como de protocolo, mediante el uso de *R2Pilot* y la transmisión independiente del vídeo.

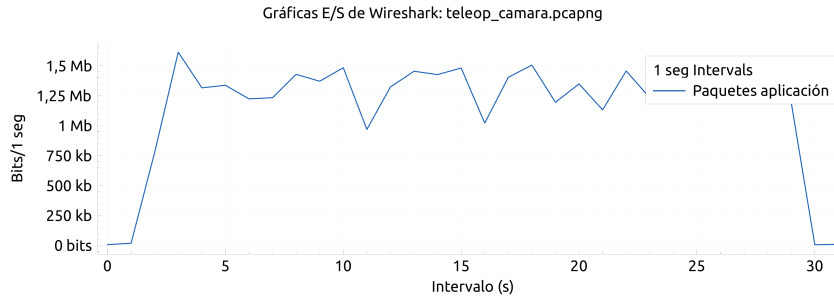


Figura 6.11: Gráfica escenario B.

6.4.2. Evaluación de la latencia

Dado que el consumo de ancho de banda no supone un problema, se analiza la latencia de la red para completar la evaluación. Como referencia para garantizar una comunicación en tiempo real, se establece una latencia máxima de 100 ms [126]. Para ello, se midió mediante *Wireshark* el tiempo de ida y vuelta (*Round Trip Time*, RTT) a nivel TCP durante la teleoperación y recepción de sensores (Figura 6.12). La mayoría de los paquetes presentan un RTT de entre 5 y 20 ms, con un máximo puntual de 75 ms durante los 30 segundos de captura.

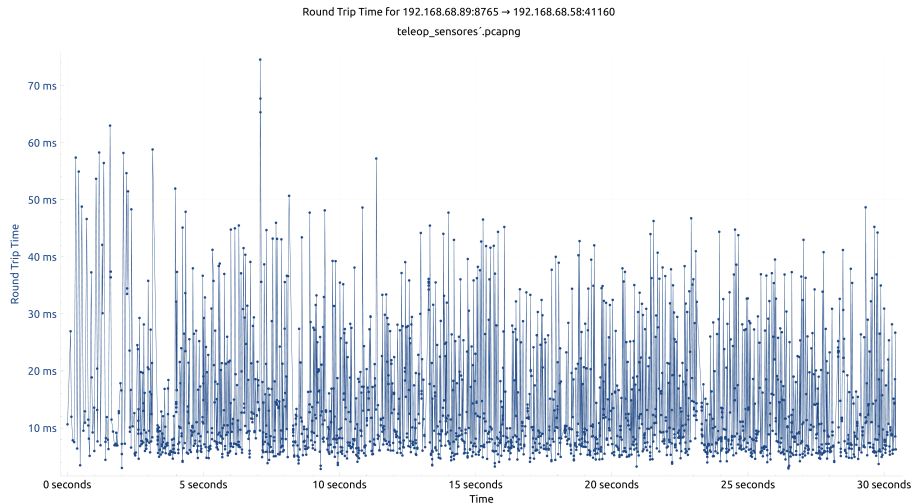


Figura 6.12: Gráfica RTT.

Suponiendo un canal simétrico, la latencia en un sentido puede aproximarse como $L \approx RTT/2$, obteniéndose una media inferior a 10 ms y un máximo inferior a 40 ms. Estos resultados confirman que la arquitectura permite una comunicación en tiempo real, manteniéndose ampliamente por debajo del límite de 100 ms.

6.5. Validación de seguridad y tolerancia a fallos

En entornos inalámbricos reales, la comunicación puede verse afectada por interferencias o microcortes, por lo que el sistema debe garantizar un funcionamiento seguro para evitar daños al robot o su entorno. Para ello, el protocolo R2Pilot implementa mecanismos de vigilancia y gestión de errores.

6.5.1. Gestión de pérdida o degradación de la conexión

Para detectar pérdidas de conexión se han implementado varios temporizadores de vigilancia (*watchdogs*), cuyo funcionamiento se describe en el capítulo 5. Su comportamiento se validó mediante pruebas de desconexión, registrando los tiempos de respuesta mediante los *logs* del servidor.

El servidor supervisa la conexión con el cliente mediante mensajes *PingReq/PingResp* a 1 Hz. Si no recibe respuesta durante más de 3 s, el *watchdog* principal libera los recursos de la sesión, detiene los motores y restablece la máquina de estados. En las pruebas, esta respuesta se completó en **0,2904 ms**. El cliente, por su parte, detecta la desconexión, elimina los datos temporales y vuelve a la pantalla de inicio mostrando un aviso.

La teleoperación dispone de un segundo *watchdog*, ya que requiere una respuesta especialmente rápida ante una pérdida de conexión. Los comandos se envían a 10 Hz y, si transcurre 1 s sin recibir uno nuevo, el robot se detiene automáticamente. El tiempo de reacción medido fue de **0,1172 ms**.

6.5.2. Gestión de errores

La gestión de errores se integra directamente en el protocolo R2Pilot mediante mensajes específicos, manteniendo sincronizados cliente y servidor. Ante situaciones como mensajes mal formados, comandos no permitidos o acciones que el robot no puede ejecutar, la operación se cancela y se envía un mensaje con un código y una descripción del error. Cuando el fallo no está asociado a una petición concreta, se utilizan mensajes asíncronos.

Además, tanto el cliente como el servidor disponen de un sistema de registro (*logging*) que almacena eventos relevantes, como cambios de estado, conexiones, ejecución de funcionalidades y envío de mensajes. Estos registros facilitan la detección y resolución de problemas y permiten supervisar el estado interno del sistema.

Capítulo 7

Conclusión y Trabajo futuro

Este capítulo recoge las principales conclusiones del Trabajo de Fin de Grado y las posibles líneas de trabajo futuro. En primer lugar, se analiza el grado de cumplimiento de los objetivos y se resumen los resultados obtenidos. Finalmente, se plantean mejoras y ampliaciones para continuar desarrollando la solución.

7.1. Conclusiones

El objetivo principal de este Trabajo de Fin de Grado ha sido desarrollar y evaluar una aplicación móvil para Android capaz de controlar y monitorizar robots basados en ROS 2, manteniendo una arquitectura modular y adaptable a distintas plataformas robóticas. Este objetivo se ha alcanzado mediante una arquitectura cliente-servidor, formada por una aplicación Android y un servidor integrado en el robot, junto con un protocolo de comunicación propio para el intercambio bidireccional de órdenes e información.

La solución permite gestionar y configurar el robot, controlar su base móvil y articulaciones, ejecutar movimientos predefinidos y monitorizar en tiempo real sus cámaras y sensores. Su funcionamiento se ha validado tanto sobre el robot físico TIAGo como mediante la simulación del Leo Rover, comprobando su adaptación a diferentes plataformas robóticas.

A continuación, se revisan los objetivos específicos planteados en el capítulo 1, indicando cómo se ha abordado cada uno y en qué medida se ha conseguido cumplirlos:

1. **OE1. Analizar las necesidades relacionadas con el control y la monitorización de robots basados en ROS 2, y establecer los requisitos funcionales y no funcionales que debe cumplir la aplicación (100 %).**

Se ha analizado el contexto de la robótica móvil, ROS 2, DDS, las aplicaciones móviles y distintas soluciones de control remoto de robots. Este estudio permitió definir los requisitos funcionales y no funcionales que guiaron el diseño de la solución, recogidos principalmente en el capítulo 2 y en las secciones 5.1 y 5.2.

2. **OE2. Definir una solución que permita la interacción remota entre la aplicación móvil y el robot, garantizando la comunicación bidireccional de órdenes e información (100 %).**

Se ha diseñado e implementado el protocolo R2Pilot, basado en una arquitectura cliente-servidor y *WebSocket*, que define la estructura y tipos de mensajes, las máquinas de estados y los mecanismos de validación y detección de desconexiones. Su diseño e implementación se detallan en el capítulo 4.

3. **OE3. Integrar las capacidades de control, monitorización y gestión proporcionadas por ROS 2 con la aplicación móvil (100 %).**

Se ha desarrollado un servidor que interactúa con ROS 2 mediante DDS y con la aplicación mediante R2Pilot, integrando funciones de consulta, control y monitorización de sensores y cámaras. Estas funcionalidades se describen en las secciones 5.4 y 5.6 y se validan en el capítulo 6.

4. **OE4. Desarrollar una aplicación Android que proporcione una interfaz eficiente y cómoda para interactuar con las diferentes capacidades del robot (100 %).**

Se ha desarrollado una aplicación nativa en Kotlin que integra las funcionalidades de control y monitorización en una única interfaz, permitiendo su uso simultáneo. La comunicación con el servidor se realiza mediante R2Pilot. El diseño y la implementación se detallan en el capítulo 5.

5. **OE5. Diseñar una solución modular que permita su uso con diferentes plataformas robóticas basadas en ROS 2 y facilite su adaptación a las características y necesidades específicas de cada robot (80 %).**

Se han implementado mecanismos de descubrimiento que analizan el robot e identifican sus capacidades y medios de interacción. Estos mecanismos se han validado con TIAGo y Leo Rover, demostrando la adaptación a diferentes plataformas. No obstante, algunas funcionalidades dependen de convenciones en los nombres de los *topics* y pueden requerir configuración adicional, por lo que existe margen de mejora en la generalización. Este objetivo se evidencia en la sección 5.6.3 y

el capítulo 6, mientras que la arquitectura modular se recoge en la sección 5.4.

6. OE6. Evaluar el funcionamiento, rendimiento y robustez de la solución mediante pruebas en diferentes configuraciones y plataformas basadas en ROS 2 (100 %).

Se han realizado pruebas funcionales y de rendimiento en simulación y sobre el robot real, incluyendo análisis de tráfico, ancho de banda, latencia y tolerancia a fallos. También se han validado los mecanismos de parada del robot ante pérdidas de comunicación. Los resultados se presentan en el capítulo 6.

Además de alcanzar los objetivos técnicos planteados, la realización de este TFG ha permitido aplicar conocimientos adquiridos durante el grado, especialmente en programación, redes y diseño de protocolos, y ampliarlos en áreas como Android, Kotlin, ROS 2, DDS, WebSocket, Docker y robótica.

Durante el desarrollo surgieron situaciones no previstas, destacando el cambio de la arquitectura de comunicación, que obligó a replantear parte de la solución y modificar la planificación inicial. Afrontar esta situación permitió mejorar la capacidad de adaptación y resolución de problemas, y puso de manifiesto la importancia de revisar las decisiones de diseño conforme evoluciona un proyecto.

A nivel personal, el proyecto ha sido una experiencia satisfactoria, tanto por los conocimientos adquiridos como por haber transformado una necesidad concreta en una aplicación funcional, validada tanto en simulación como sobre un robot real. En conjunto, el TFG me ha permitido mejorar la autonomía, la organización, el pensamiento crítico y la capacidad de toma de decisiones, además de ampliar los conocimientos técnicos en diferentes tecnologías y ámbitos.

7.2. Trabajo futuro

A partir del desarrollo realizado y de las limitaciones identificadas durante la evaluación, se plantean diferentes líneas de trabajo que permitirían continuar mejorando la solución. Una de las principales sería aumentar el grado de generalización de la aplicación. Aunque el mecanismo de descubrimiento desarrollado permite identificar automáticamente buena parte de las capacidades de un robot, algunas funcionalidades todavía dependen de convenciones en los nombres de los topics y de determinadas estructuras propias de cada plataforma. Como posible mejora, podría desarrollarse un mecanismo de descubrimiento y configuración más completo que permita

identificar de forma más robusta las capacidades de cada robot y asociarlas automáticamente con las funcionalidades disponibles en la aplicación, reduciendo así la necesidad de realizar adaptaciones específicas.

Para comprobar y mejorar esta capacidad de adaptación, sería interesante ampliar la validación a un mayor número de plataformas basadas en ROS 2. Durante el proyecto se ha comprobado el funcionamiento tanto con TIAGo como con Leo Rover, lo que ha permitido validar la arquitectura en robots con características diferentes. Probar la aplicación con otras plataformas permitiría detectar nuevas diferencias entre sus implementaciones y seguir mejorando las abstracciones desarrolladas.

También sería interesante continuar ampliando las funcionalidades de la solución, incorporando nuevas operaciones de ROS 2 y nuevas opciones de control, monitorización y diagnóstico del robot. Esto podría incluir, por ejemplo, el soporte para nuevos sensores, mecanismos de control o herramientas de supervisión, así como una gestión más completa de las sesiones y de las capacidades del robot. Estas ampliaciones afectarían tanto al protocolo R2Pilot como a la aplicación, manteniendo la comunicación entre ambos componentes. Además, la arquitectura modular desarrollada facilita la incorporación progresiva de nuevas capacidades sin modificar la estructura principal de la solución, permitiendo que futuras versiones puedan adaptarse a un mayor número de necesidades y plataformas.

Por último, sería conveniente abordar aspectos relacionados con la seguridad de las comunicaciones y la facilidad de despliegue de la solución. En aquellos escenarios en los que la aplicación se utilice sobre redes que no sean de confianza, podría estudiarse la incorporación de comunicaciones seguras mediante WebSocket Secure (WSS), proporcionando una mayor protección de la información intercambiada entre la aplicación y el robot. Asimismo, podría mejorarse el proceso de instalación y configuración del servidor para facilitar su despliegue en diferentes robots y entornos de trabajo. Estas mejoras permitirían avanzar hacia una solución más segura y sencilla de utilizar, manteniendo la arquitectura desarrollada como base.

Bibliografía

- [1] J. Wang and D. Herath, *What Makes Robots? Sensors, Actuators, and Algorithms*, pp. 177–203. Singapore: Springer Nature Singapore, 2022.
- [2] International Federation of Robotics, “Standardisation in robotics.” <https://ifr.org/standardisation>, 2026. Accedido: 29-agosto-2026.
- [3] S. Opiyo, J. Zhou, E. Mwangi, W. Kai, and I. Sunusi, “A review on teleoperation of mobile ground robots: Architecture and situation awareness,” *International Journal of Control, Automation and Systems*, vol. 19, no. 3, pp. 1384–1407, 2021.
- [4] Open Source Robotics Foundation, “Ros - robot operating system.” <https://www.ros.org/>, 2026. Accedido: 29-agosto-2026.
- [5] Open Robotics, “Nodes.” <https://docs.ros.org/en/humble/Concepts/Basic/About-Nodes.html>, 2023. Accedido: 25-agosto-2026.
- [6] Departamento de Teoría de la Señal, Telemática y Comunicaciones (TSTC) - UGR, “Puesta en marcha del laboratorio singular win-net-lab,” 2025.
- [7] Yavuz Ertuğrul - Acrome, “Essentials of building a mobile robot: All you need to know.” <https://acrome.net/post/essentials-of-building-a-mobile-robot-all-you-need-to-know>, 2024. Accedido: 25-agosto-2026.
- [8] Aileen - Fdatabot, “What is a mobile robot? definition, types, and applications explained.” <https://www.f databot.com/what-is-a-mobile-robot-definition-types-and-applications-explained/>, 2026. Accedido: 25-agosto-2026.
- [9] Keyland, “Robots móviles: qué son, cómo funcionan y cómo optimizan la intralogística.” <https://www.keyland.es/que-es-un-robot-movil/>, 2026. Accedido: 25-agosto-2026.

- [10] Robotnik, “Types of sensors used in mobile robots.” <https://robotnik.eu/types-of-sensors-used-in-mobile-robots/>, 2023. Accedido: 25-agosto-2026.
- [11] TE Connectivity, “Sensor technology for collaborative robots.” <https://www.te.com/en/industries/industrial-machinery/applications/robotics/sensors-for-cobots-and-robotics.html>, 2026. Accedido: 25-agosto-2026.
- [12] IBM, “What is lidar?.” <https://www.ibm.com/think/topics/lidar>, 2026. Accedido: 25-agosto-2026.
- [13] Robotics Center, “Robot teleoperation over the internet: Handling latency and packet loss.” <https://www.roboticscenter.ai/blog/teleoperation-internet-delay-handling>, 2026. Accedido: 25-agosto-2026.
- [14] Open Robotics, “Ros 2 documentation: Humble hawkbill.” <https://docs.ros.org/en/humble/index.html>, 2026. Accedido: 25-agosto-2026.
- [15] Esther Weon - Foxglove, “The building blocks of ros 2.” <https://foxglove.dev/blog/the-building-blocks-of-ros2>, 2022. Accedido: 25-agosto-2026.
- [16] Open Robotics, “Ros 2 distributions.” <https://docs.ros.org/en/foxy/Releases.html>, 2023. Accedido: 25-agosto-2026.
- [17] MathWorks, “Introduction to robot operating system 2 (ros 2).” <https://www.mathworks.com/help/ros/gs/robot-operating-system-ros2-basic-concepts.html>, 2026. Accedido: 25-agosto-2026.
- [18] Open Robotics, “Creating a package.” <https://docs.ros.org/en/foxy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>, 2023. Accedido: 25-agosto-2026.
- [19] Open Source Robotics Foundation, “About ros 2 client libraries.” <https://docs.ros.org/en/humble/Concepts/Basic/About-Client-Libraries.html>, 2025. Accedido: 12-agosto-2026.
- [20] Dirk Thomas, “Ros middleware interface.” https://design.ros2.org/articles/ros_middleware_interface.html, 2017. Accedido: 25-agosto-2026.
- [21] Open Robotics, “Configuring ros 2 environment.” <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/>

- Configuring-ROS2-Environment.html, 2023. Accedido: 25-agosto-2026.
- [22] Open Robotics, “The ros_domain_id.” <https://docs.ros.org/en/foxy/Concepts/About-Domain-ID.html>, 2021. Accedido: 25-agosto-2026.
- [23] Open Robotics, “Working with multiple ros 2 middleware implementations.” <https://docs.ros.org/en/foxy/How-To-Guides/Working-with-multiple-RMW-implementations.html>, 2023. Accedido: 25-agosto-2026.
- [24] Automatic Addison, “Ros 2 architecture overview.” <https://automaticaddison.com/ros-2-architecture-overview/>, 2021. Accedido: 25-agosto-2026.
- [25] eProsima, “What is dds?.” https://fast-dds.docs.eprosima.com/en/2.x/fastdds/getting_started/definitions.html, 2019. Accedido: 26-agosto-2026.
- [26] DDS Foundation, “What is dds?.” <https://www.dds-foundation.org/what-is-dds-3/>, 2025. Accedido: 26-agosto-2026.
- [27] Object Management Group (OMG), “Data distribution service (dds).” <https://www.omg.org/omg-dds-portal/>, 2025. Accedido: 26-agosto-2026.
- [28] eProsima, “Rtps introduction.” <https://www.eprosima.com/developer-resources/whitepapers/rtps>, 2026. Accedido: 26-agosto-2026.
- [29] Open Robotics, “Dds implementations.” <https://docs.ros.org/en/foxy/Installation/DDS-Implementations.html>, 2023. Accedido: 26-agosto-2026.
- [30] eProsima, “What is fast dds?.” <https://fast-dds.docs.eprosima.com/en/latest/>, 2019. Accedido: 26-agosto-2026.
- [31] Eclipse Foundation, “Eclipse cyclone dds.” <https://cyclonedds.io/>, 2022. Accedido: 26-agosto-2026.
- [32] Object Management Group (OMG), “Quality of service (qos) policies.” https://www.omgwiki.org/ddsf/doku.php?id=ddsf:public:guidebook:06_append:glossary:q:quality_of_service_qos_policies, 2021. Accedido: 26-agosto-2026.
- [33] eProsima, “Standard qos policies.” https://fast-dds.docs.eprosima.com/en/v3.4.3/fastdds/dds_layer/core/policy/standardQosPolicies.html, 2019. Accedido: 26-agosto-2026.

- [34] eProsima, “Discovery.” <https://fast-dds.docs.eprosima.com/en/latest/fastdds/discovery/discovery.html>, 2019. Accedido: 26-agosto-2026.
- [35] Open Robotics, “Topics.” <https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html>, 2025. Accedido: 25-agosto-2026.
- [36] Open Robotics, “Understanding topics.” <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>, 2025. Accedido: 25-agosto-2026.
- [37] Open Robotics, “Ros enhancement proposals (reps).” <https://reps.openrobotics.org/>, 2026. Accedido: 25-agosto-2026.
- [38] Open Robotics, “Services.” <https://docs.ros.org/en/humble/Concepts/Basic/About-Services.html>, 2023. Accedido: 25-agosto-2026.
- [39] Open Robotics, “Understanding services.” <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>, 2025. Accedido: 25-agosto-2026.
- [40] Open Robotics, “Actions.” <https://docs.ros.org/en/humble/Concepts/Basic/About-Actions.html>, 2023. Accedido: 25-agosto-2026.
- [41] Open Robotics, “Understanding actions.” <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>, 2025. Accedido: 25-agosto-2026.
- [42] Open Robotics, “Interfaces.” <https://docs.ros.org/en/humble/Concepts/Basic/About-Interfaces.html>, 2025. Accedido: 25-agosto-2026.
- [43] Open Robotics, “Ros 2 api documentation.” <https://docs.ros2.org/latest/api/>, 2026. Accedido: 25-agosto-2026.
- [44] DataReportal, “Digital around the world.” <https://datareportal.com/global-digital-overview>, 2026. Accedido: 26-agosto-2026.
- [45] Google, “Descripción general de accesorios y hosts usb.” <https://developer.android.com/develop/connectivity/usb>, 2026. Accedido: 26-agosto-2026.

- [46] Google, “Guías de conectividad.” <https://developer.android.com/develop/connectivity/overview>, 2026. Accedido: 26-agosto-2026.
- [47] Statcounter, “Mobile operating system market share worldwide.” <https://gs.statcounter.com/os-market-share/mobile/worldwide>, 2026. Accedido: 26-agosto-2026.
- [48] Counterpoint Research, “Global smartphone sales share by operating system.” <https://counterpointresearch.com/en/insights/global-smartphone-os-market-share>, 2026. Accedido: 26-agosto-2026.
- [49] Google, “Meet android studio.” <https://developer.android.com/studio/intro/index.html>, 2026. Accedido: 26-agosto-2026.
- [50] Google, “Android’s kotlin-first approach.” <https://developer.android.com/kotlin/first>, 2026. Accedido: 26-agosto-2026.
- [51] Google, “Develop ui.” <https://developer.android.com/develop/ui>, 2026. Accedido: 26-agosto-2026.
- [52] N. Rottmann, N. Studt, F. Ernst, and E. Rueckert, “Ros-mobile: An android application for the robot operating system,” *arXiv preprint arXiv:2011.02781*, 2020. Disponible en: <https://github.com/ROS-Mobile/ROS-Mobile-Android>.
- [53] N. Rottmann, N. Studt, F. Ernst, and E. Rueckert, “Ros-mobile: An android application for the robot operating system,” *arXiv preprint arXiv:2011.02781*, 2020. Repositorio disponible en: <https://github.com/YasuChiba/ROS2-Mobile-Android>.
- [54] Simon Bogh Holding ApS, “Ros2 connect mobile.” Aplicación para iOS disponible en Apple App Store, 2026. Disponible en: <https://apps.apple.com/ar/app/ros2-connect-mobile/id6744851782>.
- [55] Vighnesh D, “Ros ackermann teleop.” Aplicación para Android disponible en Google Play Store, 2025. Disponible en: <https://play.google.com/store/apps/details?id=com.vighnesh.rosackermannteleop>.
- [56] Burf Robotics, “Ros2 visualizer.” Aplicación para Android disponible en Google Play Store, 2026. Disponible en: <https://play.google.com/store/apps/details?id=co.burf.ros2visualizer>.
- [57] María Dueñas - Codurance, “Las claves del desarrollo iterativo e incremental.” <https://www.codurance.com/es/publications/desarrollo-iterativo-e-incremental>, 2022. Accedido: 27-agosto-2026.

- [58] Notion Labs, “Notion – el espacio de trabajo conectado.” <https://www.notion.com/es-es>, 2026. [Accedido: 27-ago-2026].
- [59] GitHub, Inc., “Github.” <https://github.com/>, 2026. Accedido: 12-agosto-2026.
- [60] Indeed, “Salarios de ingeniero/a junior en españa.” <https://es.indeed.com/career/ingeniero-a-junior/salaries?salaryType=HOURLY>, 2026. Accedido: 27-agosto-2026.
- [61] Indeed, “Salarios de director de proyecto en españa.” <https://es.indeed.com/career/director-de-proyecto/salaries?salaryType=HOURLY>, 2026. Accedido: 27-agosto-2026.
- [62] Robot Web Tools, “rosbridge_suite repository.” https://github.com/RobotWebTools/rosbridge_suite, 2026. [Accedido: 4-ago-2026].
- [63] Open Source Robotics Foundation, “rosbridge_suite – ROS wiki.” https://wiki.ros.org/rosbridge_suite, 2022. [Accedido: 4-ago-2026].
- [64] Open Source Robotics Foundation, “rosbridge_suite package documentation – ROS 2 documentation.” https://docs.ros.org/en/jazzy/p/rosbridge_suite/, 2024. [Accedido: 4-ago-2026].
- [65] Foxglove Technologies, “Foxglove bridge documentation.” <https://docs.foxglove.dev/docs/visualization/ros-foxglove-bridge>. [Accedido: 4-ago-2026].
- [66] E. Weon, “Announcing the new foxglove bridge for live ros data.” <https://foxglove.dev/blog/announcing-the-new-foxglove-bridge-for-live-ros-data>, Feb. 2023. [Accedido: 4-ago-2026].
- [67] Matthew O’Riordan, “WebSocket vs HTTP, SSE, MQTT, WebRTC more.” <https://websocket.org/comparisons/>, Jan. 2025. [Actualizado: 12-mar-2026; Accedido: 5-ago-2026].
- [68] Matthew O’Riordan, “WebSocket protocol: Rfc 6455 handshake, frames more.” <https://websocket.org/guides/websocket-protocol/>, Sept. 2024. [Actualizado: 10-mar-2026; Accedido: 5-ago-2026].
- [69] Matthew O’Riordan, “WebSockets vs. WebRTC: When to use each protocol.” <https://websocket.org/comparisons/webrtc/>, Jan. 2025. [Actualizado: 10-mar-2026; Accedido: 5-ago-2026].

- [70] Matthew O’Riordan, “WebSockets vs. WebTransport: When to use which.” <https://websocket.org/comparisons/webtransport/>, Jan. 2025. [Actualizado: 10-mar-2026; Accedido: 5-ago-2026].
- [71] M. C. O. Vidal, “Diagramas de estado.” Material docente del Dpto. de Lenguajes y Sistemas Informáticos, UPV/EHU. <http://www.vc.ehu.es/jiwotvim/IngenieriaSoftware/Teoria/BloqueII/UML-5.pdf>. [Accedido: 5-ago-2026].
- [72] K.-S. Chang and D. Zhu, “Hierarchical Finite State Machines (HFSM) and Behavior Trees (BT).” CS123 Lecture Notes, Stanford University. https://web.stanford.edu/class/cs123/lectures/CS123_lec08_HFSM_BT.pdf. [Accedido: 5-ago-2026].
- [73] IBM Corporation, “Serialization overview.” <https://www.ibm.com/docs/en/wxs/8.6.2?topic=overview-serialization>, June 2026. Accedido: 06-agosto-2026.
- [74] A. Sumaray and S. K. Makki, “A comparison of data serialization formats for optimal efficiency on a mobile platform,” in *Proceedings of the 6th International Conference on Ubiquitous Information Management and Communication*, pp. 1–6, 2012.
- [75] E. Minetto, “Json vs flatbuffers vs protocol buffers,” Aug. 2024. Accedido: 06-agosto-2026.
- [76] The Tornado Authors, “Tornado web server documentation.” <https://www.tornadoweb.org/en/stable/>, 2026. Accedido: 06-agosto-2026.
- [77] S. Ramírez, “Fastapi.” <https://fastapi.tiangolo.com/>, 2026. Accedido: 06-agosto-2026.
- [78] M. Grinberg, “python-socketio documentation.” <https://python-socketio.readthedocs.io/en/stable/>, 2018. Accedido: 06-agosto-2026.
- [79] A. Augustin, “websockets documentation.” <https://websockets.readthedocs.io/en/stable/>, 2026. Accedido: 06-agosto-2026.
- [80] A. Augustin, “websockets - github repository.” <https://github.com/python-websockets/websockets>, 2026. Accedido: 06-agosto-2026.
- [81] Block.Inc., “Okhttp documentation.” <https://lysine.dev/okhttp/>, 2022. Accedido: 07-agosto-2026.
- [82] JetBrains s.r.o, “Websockets in ktor client.” <https://ktor.io/docs/client-websockets.html>, Apr. 2026. Accedido: 07-agosto-2026.

- [83] JetBrains s.r.o, “Ktor: Asynchronous framework for web applications.” <https://ktor.io/>, 2026. Accedido: 07-agosto-2026.
- [84] JetBrains s.r.o., “Client engines - ktor documentation.” <https://ktor.io/docs/client-engines.html>, 2025. Accedido: 07-agosto-2026.
- [85] JSON Schema, “Json schema.” <https://json-schema.org/>, 2026. Accedido: 07-agosto-2026.
- [86] JetBrains s.r.o., “Sealed classes and interfaces - kotlin documentation.” <https://kotlinlang.org/docs/sealed-classes.html>, June 2026. Accedido: 08-agosto-2026.
- [87] P. S. Foundation, “orjson - python package index.” <https://pypi.org/project/orjson/>, 2026. Accedido: 09-agosto-2026.
- [88] Python Software Foundation, “ujson - python package index.” <https://pypi.org/project/ujson/>, 2026. Accedido: 09-agosto-2026.
- [89] Python Software Foundation, “json — json encoder and decoder.” <https://docs.python.org/3/library/json.html>, 2026. Accedido: 07-agosto-2026.
- [90] Google, “Gson - github repository.” <https://github.com/google/gson>, 2026. Accedido: 08-agosto-2026.
- [91] FasterXML, LLC, “jackson-module-kotlin - github repository.” <https://github.com/FasterXML/jackson-module-kotlin>, 2026. Accedido: 08-agosto-2026.
- [92] JetBrains s.r.o., “Kotlin serialization documentation.” <https://kotlinlang.org/docs/serialization.html>, June 2026. Accedido: 08-agosto-2026.
- [93] JetBrains s.r.o., “kotlinx.serialization - github repository.” <https://github.com/Kotlin/kotlinx.serialization>, 2026. Accedido: 08-agosto-2026.
- [94] IBM - Alexandra Jonker - Alice Gomstyn, “What is requirements management?.” <https://www.ibm.com/think/topics/what-is-requirements-management>, 2026. Accedido: 11-agosto-2026.
- [95] D. van Heesch, “Doxygen: Generate documentation from source code,” 2026. Accedido: 03-09-2026.

- [96] Canonical Ltd., “Ubuntu 22.04 lts release notes.” <https://documentation.ubuntu.com/release-notes/22.04/>, 2026. Accedido: 12-agosto-2026.
- [97] Open Source Robotics Foundation, “Ros 2 humble hawkbill.” <https://docs.ros.org/en/foxy/Releases/Release-Humble-Hawksbill.html>. Accedido: 12-agosto-2026.
- [98] Open Source Robotics Foundation, “Getting started with ros.” <https://www.ros.org/blog/getting-started/>. Accedido: 12-agosto-2026.
- [99] Microsoft, “¿qué es el subsistema de windows para linux?.” <https://learn.microsoft.com/es-es/windows/wsl/about>, 2026. Accedido: 12-agosto-2026.
- [100] Docker Inc., “Docker: Accelerated container application development.” <https://www.docker.com/>, 2026. Accedido: 12-agosto-2026.
- [101] Python Software Foundation, “Python.” <https://www.python.org/>, 2026. Accedido: 12-agosto-2026.
- [102] Microsoft, “Visual studio code - code editing. redefined.” <https://code.visualstudio.com/>, 2026. Accedido: 12-agosto-2026.
- [103] M. Makai, “Why use python?.” <https://www.fullstackpython.com/why-use-python.html>, 2022. Accedido: 12-agosto-2026.
- [104] J. Shim, “Ros 2 - visual studio marketplace.” <https://marketplace.visualstudio.com/items?itemName=JaehyunShim.vscode-ros2>. Accedido: 12-agosto-2026.
- [105] SUSE, “Vs code remote containers.” <https://docs.rancherdesktop.io/how-to-guides/vs-code-remote-containers/>, 2026. Accedido: 12-agosto-2026.
- [106] JetBrains s.r.o., “Kotlin programming language.” <https://kotlinlang.org/>, 2026. Accedido: 12-agosto-2026.
- [107] Google LLC, “Jetpack compose.” <https://developer.android.com/compose>, 2026. Accedido: 12-agosto-2026.
- [108] Google LLC, “Enfoque de prioridad de kotlin en android.” <https://developer.android.com/kotlin/first?hl=es-419>, 2026. Accedido: 12-agosto-2026.
- [109] Google LLC, “Desarrolla apps para android con kotlin.” <https://developer.android.com/kotlin?hl=es-419>, 2026. Accedido: 12-agosto-2026.

- [110] JetBrains s.r.o., “Coroutines.” <https://kotlinlang.org/docs/coroutines-overview.html>, 2026. Accedido: 12-agosto-2026.
- [111] Open Source Robotics Foundation, “Ros 2 integration overview - gazebo classic.” https://classic.gazebosim.org/tutorials?tut=ros2_overview, 2026. Accedido: 12-agosto-2026.
- [112] PAL Robotics, “Tiago: The mobile manipulator robot.” <https://pal-robotics.com/es/robot/tiago/>, 2026. Accedido: 12-agosto-2026.
- [113] Fernán García de Zúñiga, “Todo sobre la arquitectura cliente-servidor.” <https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor>, 2026. Accedido: 13-agosto-2026.
- [114] Jacob Perron Ivan Santiago Paunovic, “ros2_java - github repository.” https://github.com/ros2-java/ros2_java, 2022. Accedido: 13-agosto-2026.
- [115] Microsoft, “Model-view-viewmodel (mvvm).” <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>, 2024. Accedido: 13-agosto-2026.
- [116] Google LLC, “Viewmodel overview.” <https://developer.android.com/topic/libraries/architecture/viewmodel>, 2026. Accedido: 13-agosto-2026.
- [117] Google LLC, “Typography - material design 3.” <https://m3.material.io/styles/typography/fonts>, 2026. Accedido: 14-agosto-2026.
- [118] Apple Inc., “Typography - human interface guidelines.” <https://developer.apple.com/design/human-interface-guidelines/typography>, 2025. Accedido: 14-agosto-2026.
- [119] Google LLC, “Google fonts icons - material symbols and icons.” <https://fonts.google.com/icons>, 2026. Accedido: 14-agosto-2026.
- [120] Open Robotics, “Urdf.” <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html>, 2026. Accedido: 19-agosto-2026.
- [121] PAL Robotics, “/play_motion2.” https://docs.pal-robotics.com/25.01/actions/play_motion2.html, 2024. Accedido: 16-agosto-2026.

-
- [122] RobotWebTools, “web_video_server.” https://github.com/RobotWebTools/web_video_server, 2026. Accedido: 16-agosto-2026.
- [123] ROS Components, “Leo rover.” <https://www.roscomponents.com/es/producto/leo-rover/>, 2026. Accedido: 18-agosto-2026.
- [124] Leo Rover, “leo_simulator-ros2.” https://github.com/LeoRover/leo_simulator-ros2, 2025. Accedido: 18-agosto-2026.
- [125] Open Robotics, “Using fast dds discovery server as discovery protocol.” <https://docs.ros.org/en/humble/Tutorials/Advanced/Discovery-Server/Discovery-Server.html>, 2026. Accedido: 20-agosto-2026.
- [126] Agora, “What’s the difference between bandwidth and latency in real-time communication?.” <https://www.agora.io/en/blog/difference-between-bandwidth-and-latency/>, 2024. Accedido: 23-agosto-2026.

Anexo A

Manual de instalación y ejecución de la aplicación

Este anexo describe el procedimiento necesario para instalar y ejecutar la aplicación R2Pilot completa a partir de los repositorios que contienen el código fuente del *backend* y del *frontend*. El *backend* se ejecuta en el robot y debe iniciarse antes que la aplicación móvil.

A.1. Requisitos previos y de seguridad

Antes de instalar y ejecutar R2Pilot deben cumplirse los siguientes requisitos:

- **Hardware y sistema operativo:** El backend debe ejecutarse en un robot con Linux (desarrollado y probado sobre Ubuntu 22.04 LTS). El frontend requiere un dispositivo Android 8.0 (API 26) o posterior.
- **Red:** Robot y dispositivo móvil deben conectarse a la misma red Wi-Fi local y segura, debido al no cifrado de las comunicaciones.
- **Dependencias del robot:** Se requiere ROS 2 correctamente configurado (Humble Hawksbill recomendada), junto con los paquetes `web_video_server` (monitorización de cámaras) y `play_motion2` (acciones pregrabadas). También son necesarios `git`, `colcon` y `rosdep`, instalables mediante:

```
1 sudo apt update
2 sudo apt install git python3-colcon-common-extensions python3-rosdep
```

Permisos de apagado y reinicio: Para que la aplicación pueda gestionar estas operaciones, deben concederse al usuario del servidor permisos específicos mediante `sudo visudo`. Por ejemplo, para el usuario `pal`:

```
1 pal ALL=(ALL) NOPASSWD: /bin/systemctl poweroff, /bin/systemctl reboot
```

A.2. Instalación y ejecución del backend

El backend se encuentra disponible en el repositorio de GitHub del proyecto. Para instalarlo se crea un espacio de trabajo de ROS 2 y se descarga el repositorio dentro de su directorio `src`:

```
1 mkdir -p ~/r2pilot_ws/src
2 cd ~/r2pilot_ws/src
3 git clone https://github.com/enriquegmez/TFG_R2Pilot-Backend.git
```

A continuación, se instalan las dependencias declaradas por el paquete:

```
1 cd ~/r2pilot_ws
2 rosdep update
3 rosdep install --from-paths src --ignore-src -r -y
```

Una vez preparado el entorno, se compila y se carga el entorno del espacio de trabajo y se inicia el backend:

```
1 colcon build
2 source install/setup.bash
3 ros2 launch r2pilot_server backend.launch.py
```

El servidor WebSocket utilizado por la aplicación escucha por defecto en el puerto 8765. El lanzamiento también inicia el servidor de vídeo empleado por la aplicación, el paquete `web_video_server`.

A.3. Instalación y ejecución del frontend

El código fuente de la aplicación Android se encuentra en un repositorio independiente:

```
1 git clone https://github.com/enriquegmez/TFG_R2Pilot-Frontend.git
```

El directorio descargado se abre directamente como proyecto en Android Studio. Una vez abierto, se espera a que Android Studio sincronice el proyecto y descargue las dependencias de Gradle.

Para ejecutarla se conecta el dispositivo Android al ordenador, se selecciona como dispositivo de ejecución en Android Studio y se pulsa *Run*.

A.3.1. Configuración de la dirección del backend

Antes de establecer la conexión, la aplicación solicita la dirección IP del servidor backend. El valor mostrado inicialmente se encuentra definido en la constante `DEFAULT_SERVER_IP` del archivo `Constants.kt` y puede modificarse directamente desde la aplicación.

Si se utiliza habitualmente el mismo robot, puede modificarse esta constante para establecer su dirección IP como valor predeterminado y evitar introducirla manualmente en cada ejecución.

A.4. Puesta en funcionamiento

Para utilizar el sistema completo deben seguirse los siguientes pasos:

1. Encender e iniciar el sistema ROS 2 del robot.
2. Ejecutar el backend siguiendo el procedimiento descrito en la Sección A.2.
3. Conectar el dispositivo Android y el robot a una red que permita su comunicación.
4. Ejecutar la aplicación móvil desde Android Studio, siguiendo el procedimiento descrito en la Sección A.2 si es la primera vez, o desde el propio dispositivo móvil si ya se encuentra instalada.
5. Comprobar o introducir la dirección IP del robot y establecer la conexión con el backend.

El backend debe estar ejecutándose antes de iniciar la conexión desde la aplicación móvil.

Anexo B

Diagramas de estados



Figura B.1: Diagrama de estados del cliente.



Figura B.2: Ampliación del diagrama de estados del cliente para visualizar la parte superior.



Figura B.3: Ampliación del diagrama de estados del cliente para visualizar la parte inferior.

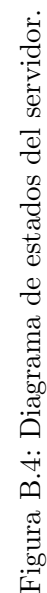


Figura B.4: Diagrama de estados del servidor.

Anexo C

Diagramas de secuencia de mensajes

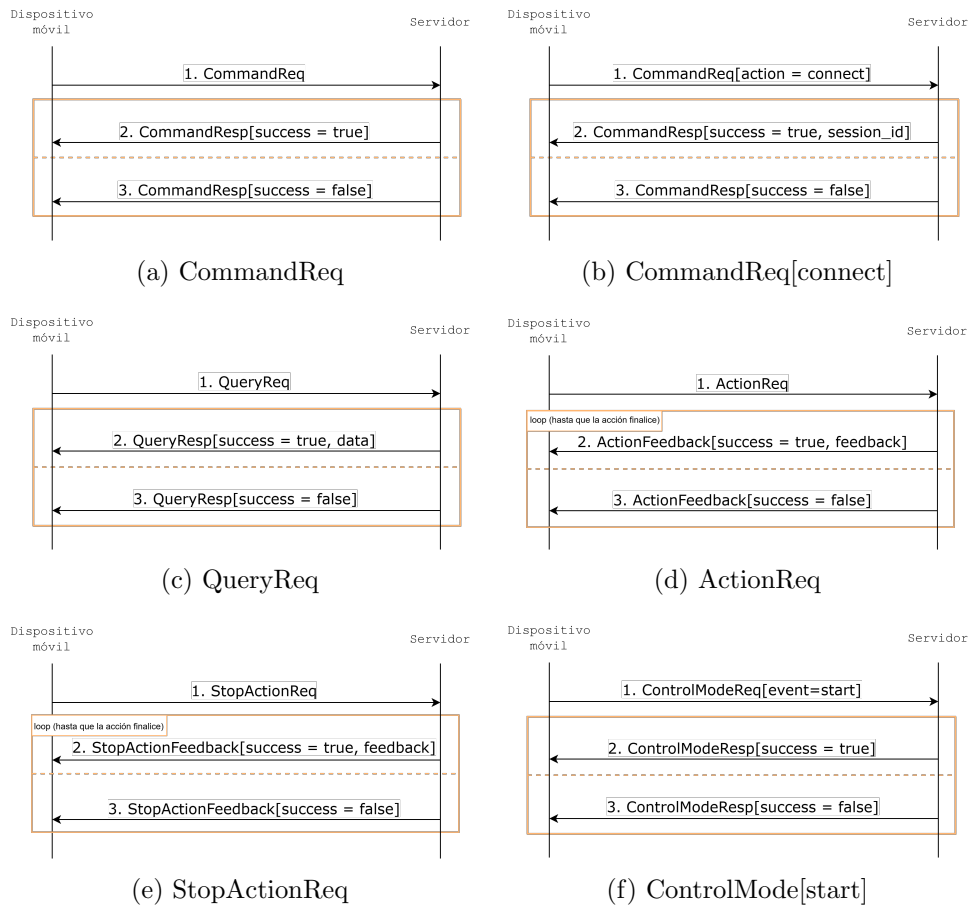
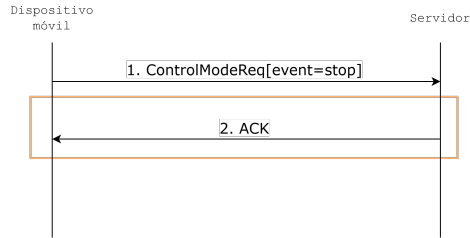
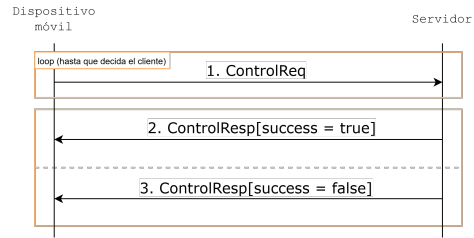


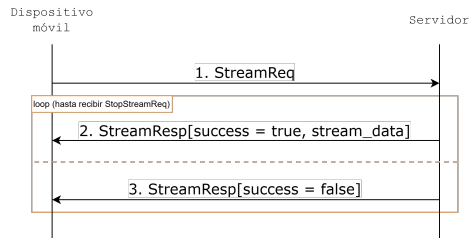
Figura C.1: Resumen de los diagramas de secuencia de los mensajes del protocolo R2Pilot (Parte 1).



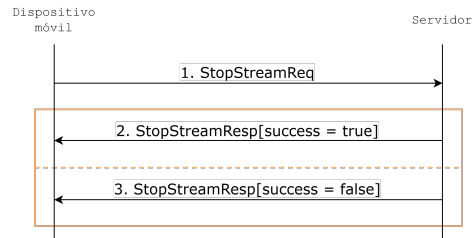
(g) ControlMode[stop]



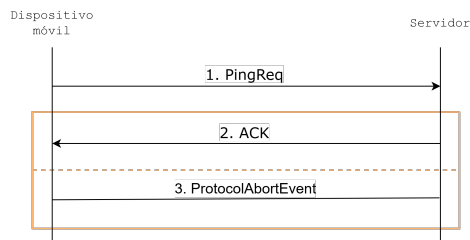
(h) ControlReq



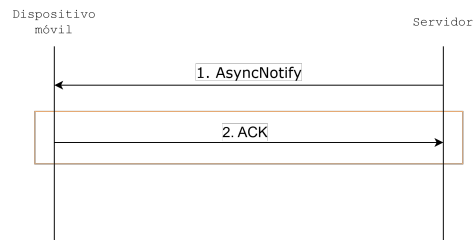
(i) StreamReq



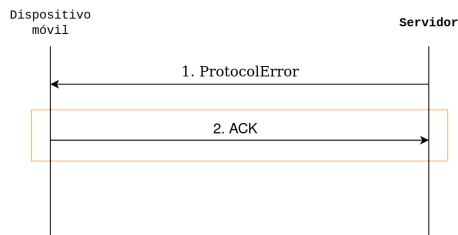
(j) StopStreamReq



(k) PingReq



(l) AsyncNotify



(m) ProtocolError

Figura C.1: Resumen de los diagramas de secuencia de los mensajes del protocolo R2Pilot (Continuación).

Anexo D

Grafos de alcanzabilidad

En este anexo se recogen, en primer lugar, los diagramas de estados completos detallados en el Capítulo 4, correspondientes tanto al cliente como al servidor. A continuación, se incluyen los grafos de alcanzabilidad generados para la validación formal del protocolo, junto con las tablas de leyenda que definen los estados y mensajes de cada máquina de estados.

D.1. Máquina de Estados Principal

Estados del Protocolo (C: Cliente / S: Servidor)		Mensajes y Eventos	
A (C)	Comienzo cliente	1	PingReq [n=1]
B (C)	Esperando conexión backend	2	CommandReq [action=change_vars]
C (C)	Conexión backend (Cliente)	3	CommandReq [action=end]
D (C)	Intermedio CommandReq/Resp	4	CommandReq [action=connect]
E (C)	Esperando inicio sesión	5	CommandReq [action=disconnect]
F (C)	Sesión iniciada (cliente)	6	ProtocolError
G (C)	Intermedio AsyncNotify	7	AsyncNotify
H (C)	Esperando info única	8	QueryReq
I (C)	Esperando Cierre Sesión	9	PingResp [n=1]
J (C)	Esperando desconexión	10	PingErrorResp [n=1]
K (C)	Fin cliente	11 / 12	CommandResp / Error [action=change_vars]
L (S)	Comienzo servidor	13 / 14	CommandResp / Error [action=end]
M (S)	Intermedio PingReq/Resp	15 / 16	CommandResp / Error [action=connect]
N (S)	Conexión backend (servidor)	17 / 18	CommandResp / Error [action=disconnect]
Ñ (S)	Intermedio Autoconexión backend	19 / 20	ProtocolResp / ErrorResp
O (S)	Intermedio CommandReq [connect]	21	AsyncNotifyResp
P (S)	Sesión iniciada (servidor)	22 / 23	QueryResp / ErrorResp
Q (S)	Intermedio Autosesión iniciada		
R (S)	Intermedio CommandReq [disconnect]		
S (S)	Intermedio CommandReq [end]		
T (S)	Fin servidor		

Tabla D.1: Leyenda para el grafo principal (figura D.1).

D.2. Subestado: Movimiento

Estados del Protocolo (C: Cliente / S: Servidor)		Mensajes y Eventos	
A (C)	Comienzo cliente	1	ControlModeReq [event=start]
B (C)	Esperando permiso envío info.	2	ControlReq
C (C)	Enviando info.	3	ControlModeReq [event=stop]
D (C)	Esperando terminar envío info.	4	SessionResetEvent
E (C)	Esperando ejecutar acción	5	ProtocolAbortEvent
F (C)	Esperando detener acción	6	ActionReq
G (C)	Fin cliente	7	StopActionReq
H (S)	Comienzo servidor	8 / 9	ControlModeErrorResp / Resp
I (S)	Intermedio ControlModeReq [start]	10	ActionErrorFeedback
J (S)	Recibiendo info.	11 / 12	ActionFeedback [done_exec=true / false]
K (S)	Intermedio ControlModeReq [stop]	13	StopActionErrorFeedback
L (S)	Intermedio ActionReq / Error	14	StopActionFeedback [done_exec=false]
M (S)	Ejecutando acción	15	StopAction [done_exec=true]
N (S)	Intermedio Autoejecutando acción		
Ñ (S)	Fin servidor		

Tabla D.2: Leyenda para el subestado Movimiento (figura D.2).

D.3. Subestado: Monitorización

Estados del Protocolo (C: Cliente / S: Servidor)		Mensajes y Eventos	
A (C)	Comienzo cliente	1	StreamReq
B (C)	Esperando recibir stream	2	StopStreamReq
C (C)	Recibiendo stream	3	SessionResetEvent
D (C)	Esperando dejar de recibir stream	4	ProtocolAbortEvent
E (C)	Fin cliente	5 / 6	StreamErrorResp / StreamResp
F (S)	Comienzo servidor	7 / 8	StopStreamErrorResp / StopStreamResp
G (S)	Intermedio StreamReq/Resp		
H (S)	Enviando stream		
I (S)	Intermedio StopStreamReq/Resp		
J (S)	Fin servidor		

Tabla D.3: Leyenda para el subestado Monitorización (figura D.3).

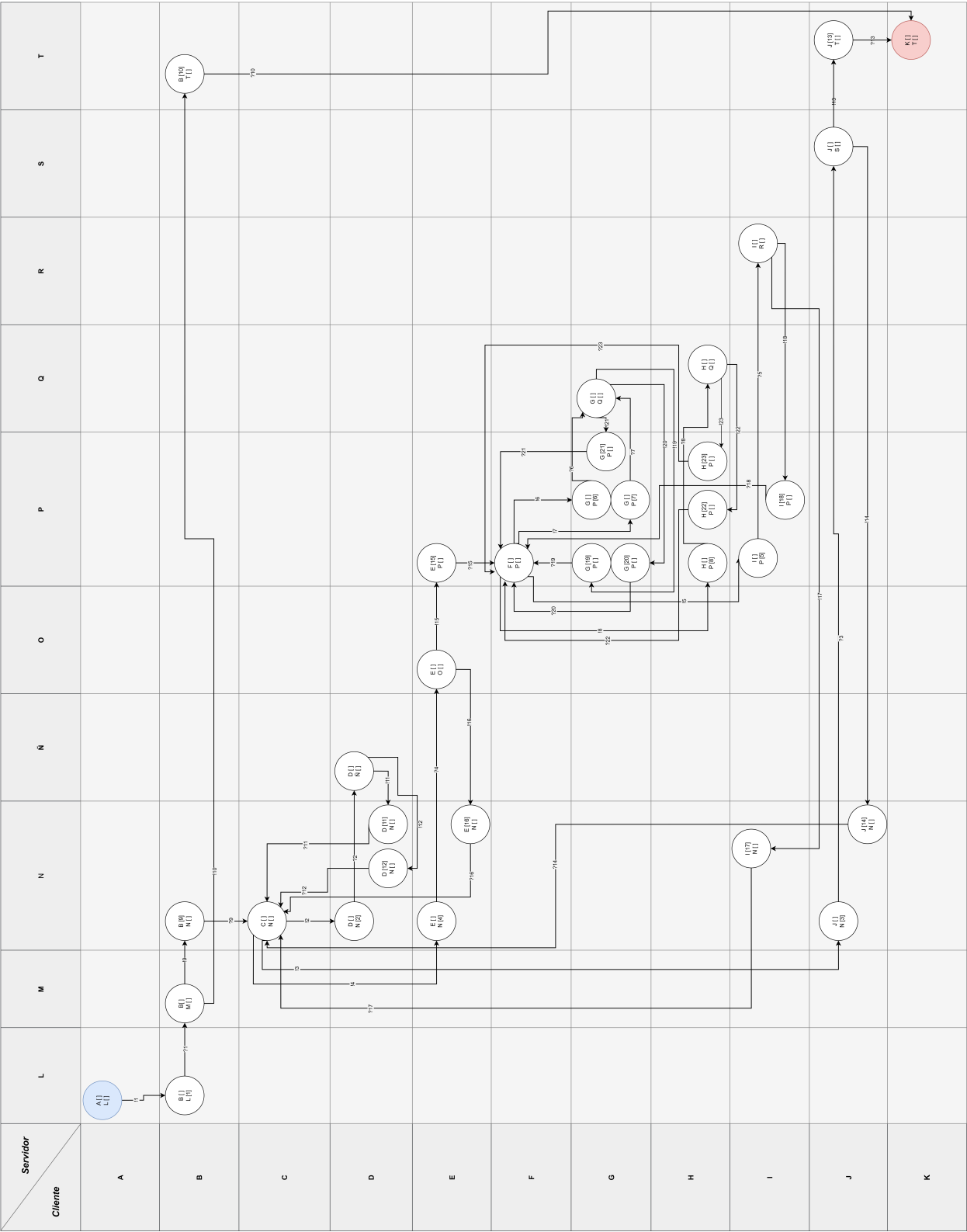


Figura D.1: Grafo de alcanzabilidad principal.

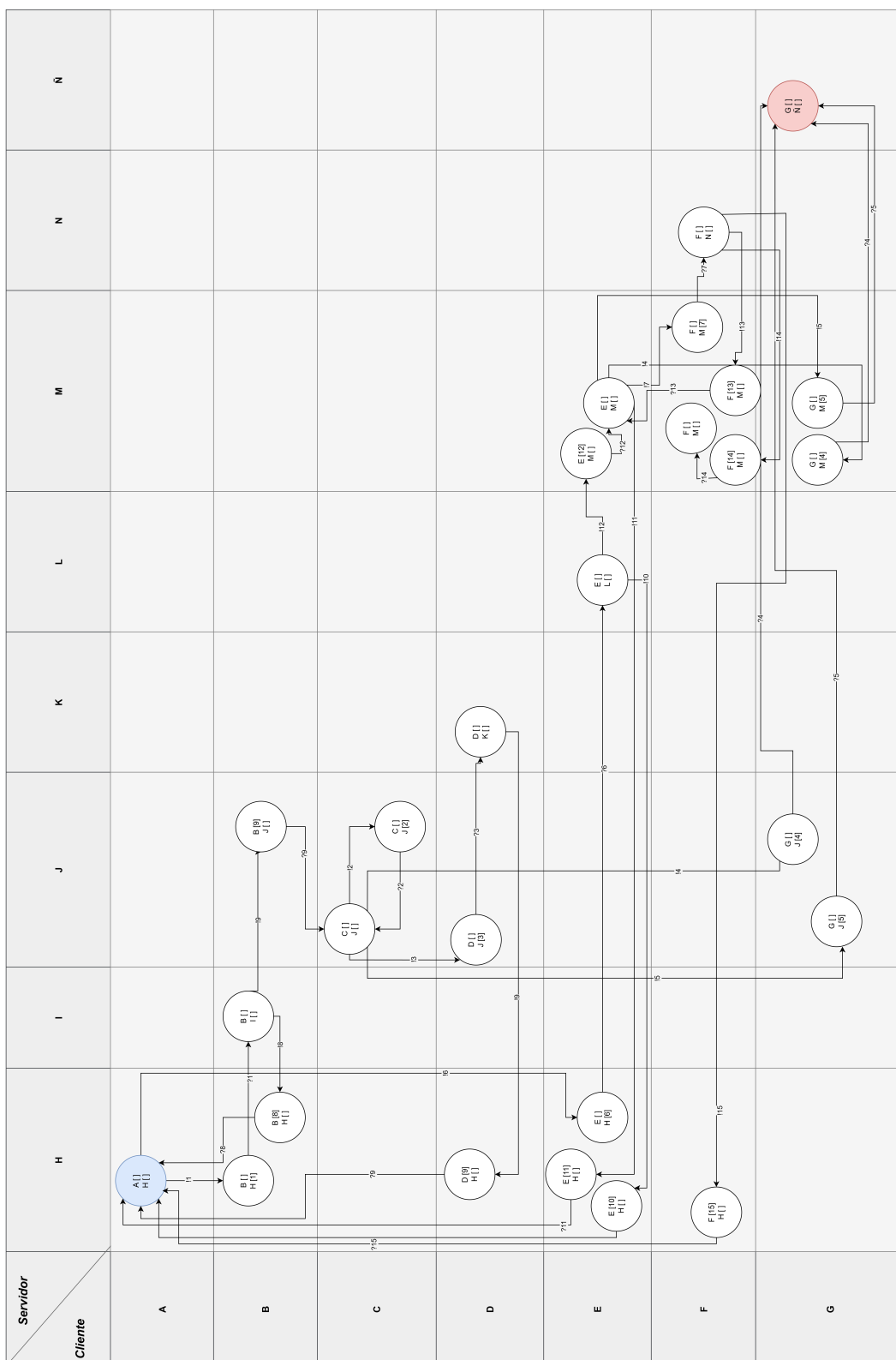


Figura D.2: Grafo de alcanzabilidad del subestado “Movimiento” .

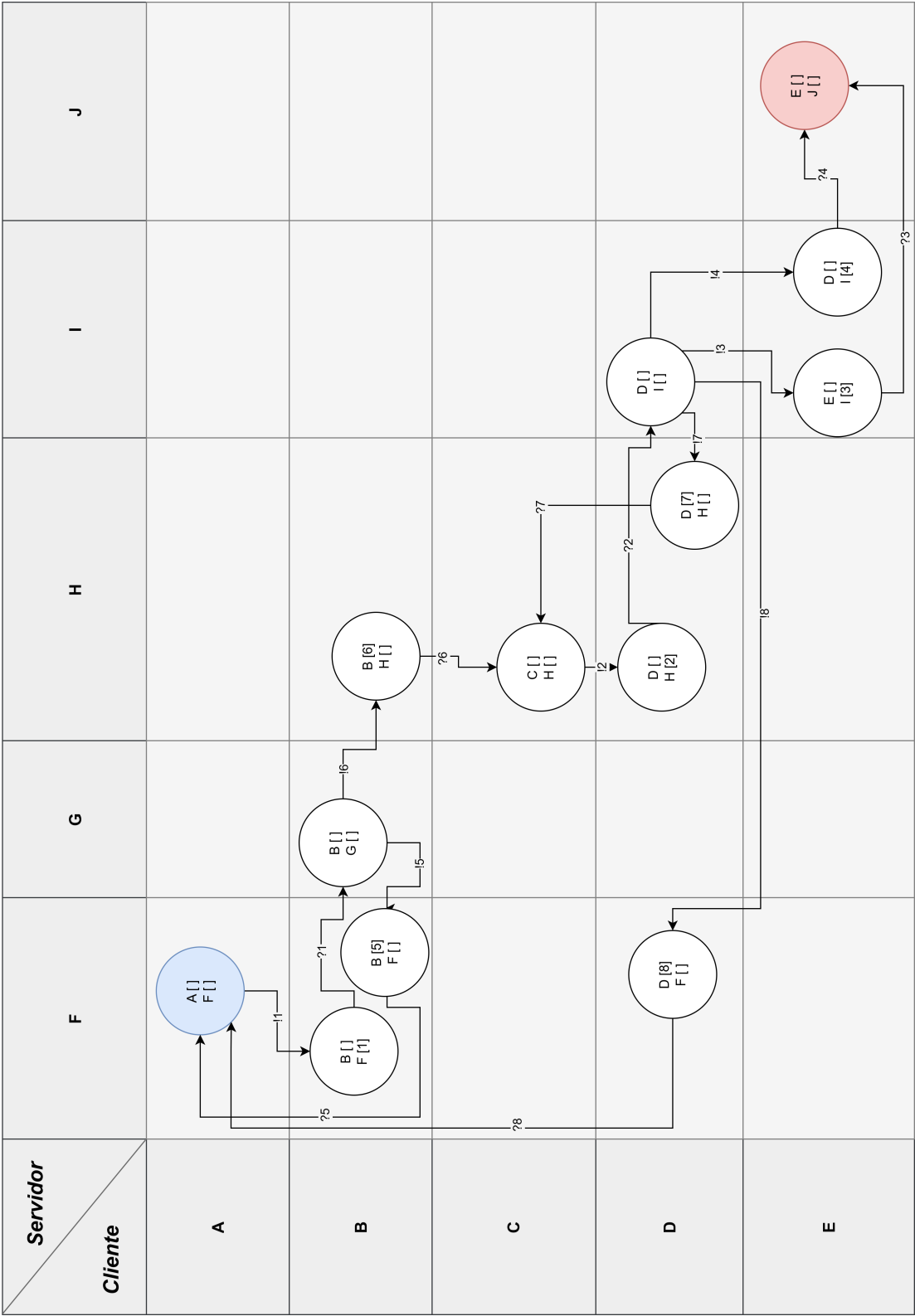


Figura D.3: Grafo de alcanzabilidad del subestado “Monitorización” .

Anexo E

Wireframes

En este anexo se recogen los bocetos iniciales (*wireframes*) desarrollados para definir la estructura, disposición y navegación de la interfaz gráfica de la aplicación R2Pilot.

E.1. Conexión y Configuración

Estas pantallas están destinadas a establecer la conexión con el servidor y a configurar los parámetros de red y la conexión con el robot. La pantalla inicial permite introducir la IP y el puerto del servidor WebSocket, mientras que la segunda permite configurar ROS 2, gestionar la conexión y administrar el apagado o reinicio remoto del robot E.1.

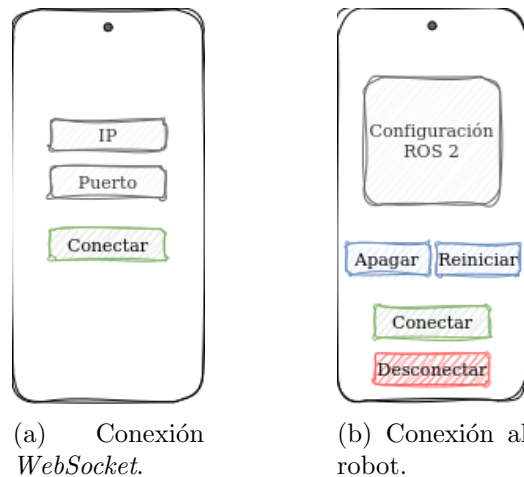


Figura E.1: Bocetos interfaces de conexión y configuración.

E.2. Control del robot

Este bloque agrupa las tres funcionalidades principales de control activo: teleoperación, control de articulaciones y ejecución de movimientos predefinidos E.2. En todas ellas se incluye un botón para alternar entre la vista completa y la vista dividida, permitiendo aprovechar el espacio y usar simultáneamente las funciones de control y monitorización E.3.

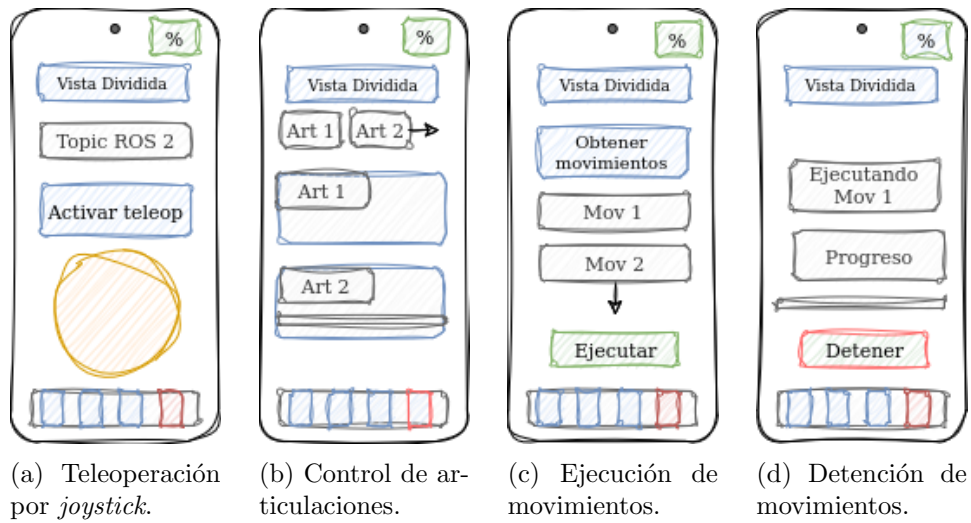


Figura E.2: Bocetos interfaces de control.

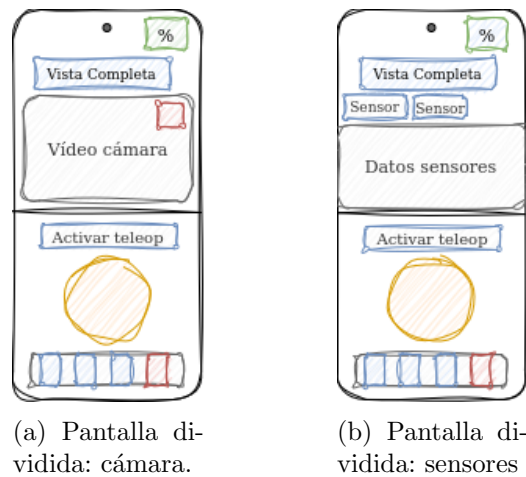
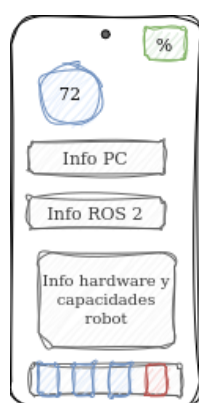


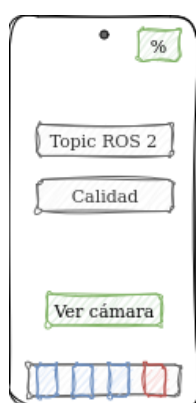
Figura E.3: Bocetos pantalla dividida.

E.3. Monitorización del robot

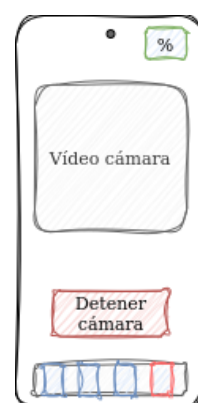
Estas pantallas corresponden a las funcionalidades de monitorización E.4. Incluyen el panel principal de información general del hardware y software, la configuración y visualización de la cámara en directo, la monitorización en tiempo real de los datos de los sensores, y el explorador de tópicos, servicios y acciones de la red ROS 2.



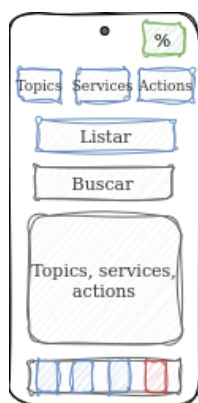
(a) Información general.



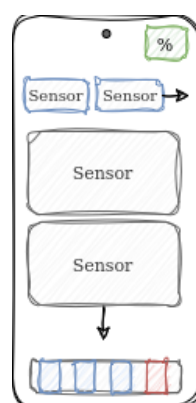
(b) Configuración cámara.



(c) Visualización cámara.



(d) Visualización tópicos.



(e) Visualización sensores.

Figura E.4: Bocetos interfaces de monitorización.

